Automating Data- and Code-Archiving Checks with Metacheck: A Replication and Extension of Cooper et al. (2026)

Daniël Lakens

Checking whether published articles archive usable data and code is time-consuming: even the lightest form of computational reproducibility checking, running an article's own code on its own shared data, has been estimated to take around 24 minutes per article, and writing independent analysis code from scratch has been estimated at 2-25 person-hours per article. Cooper and the BES Data and Code Hackathon Group (2026) manually coded data- and code-archiving practices in 1861 papers published across the seven British Ecological Society (BES) journals between 2017 and 2024, using a two-day hackathon of 145 participants who followed a bespoke coding protocol. This manual effort is exactly the kind of task that automated tools such as Metacheck aim to support: extracting structured information about open science practices from the text and repositories linked to a scientific article, in a fraction of the time. We ran Metacheck's `repo_check`, `data_check`, and `code_check` modules on the same corpus and compared its output against Cooper et al.'s human-coded ground truth for every construct where a direct, mechanical comparison is possible: data availability, archive location, data licensing, data and code download success, code archiving, and README presence. For each construct we report sensitivity, specificity, and two directional miss rates (how often a human coder caught something the tool missed, and vice versa), and, going beyond aggregate accuracy, we individually reviewed every disagreeing paper against Metacheck's saved output, the paper's own text, and, where relevant, a live call to the repository platform's own API, to determine which side was correct and why. Agreement on whether data could be downloaded and used was 99.0%, and on whether code could be downloaded was 99.7%; agreement was lower and more variable for whether a repository could be found at all (85.2%), whether code was archived (87.5%), and whether a README was present (72.2%). Metacheck's precision is consistently high across constructs — when it reports a repository, license, download, or README as present, that report is rarely spurious — but its recall is uneven and construct-dependent, driven by a mix of a repository-detection coverage gap (a real, working data or code citation on a platform or in a format Metacheck

does not yet recognise), data or code shared in a form that does not identify it as data or code (for example, tables inside a PDF or Word document, or unlabelled plain-text files), and at least one genuine definitional mismatch (platform-level description metadata a human coder reasonably reads as README-equivalent, which Metacheck's stricter file-presence check does not). Our goal is not to replace manual coding of this kind, but to clarify, concretely and per construct, where automation can currently reduce the burden of large-scale meta-scientific data collection while maintaining comparable accuracy, and where it currently cannot, and why.

Introduction

The last two decades has seen a notable culture change around open science practices. Universities, journals, and funders increasingly expect researchers to engage in open science practices. One open science practice that has become especially widely required is data and code sharing. The goal of data and code sharing is to facilitate re-use, and to increase transparency. Funders have started to require data management plans that specify where and how data and code will be made available for re-use, and submission guidelines for journals often require that data and code underlying the claims in a manuscript are shared if possible. The British Ecological Society, for example, requires that "all data and code supporting the results in the paper must be archived in a public repository" (https://www.britishecologicalsociety.org/publications/journals/author-guidelines/).

Although the requirements to share data and code are clear, it is less clear who is responsible to check that authors actually follow through on these requirements. Manually checking data and code files is a time-consuming task that requires expertise about best practices in data and code sharing. Due to the diffusion of responsibility and the lack of resources to check whether scientific manuscripts follow best practices in open science, requirements to share data and code are often not followed, and if data and code are shared, best practices are often not followed. Cooper and the BES Data and Code Hackathon Group (2026b) assessed data- and code-archiving practices across the seven journals published by the British Ecological Society (BES). They found that most manuscripts in which data was used also shared the data, but code sharing was less common, and even when data and code were shared, best practices were often not followed. About a third of data and/or code repositories lacked a README file, code not always well-annotated, and code was not always reproducible (e.g., due to hard-coded file paths). These findings strongly suggest that it would be beneficial if data and code is reviewed before a manuscript is published.

Some journals manually check data and code before an article is published. This practice is most common in economics journals, where a dedicated Data Editor oversees the reproduction of results based on data and code (Vilhuber & Cavanagh (2025)). Manual checks are laborious and delay the publication process. It is therefore not surprising that researchers are exploring automated ways to check whether data and code are shared, and whether they are usable. For

example, the Scalable Infrastructure for Validation of Computational Social Science Research (SIVACOR) pilot project by the data editors of the American Economic Association examines whether it is possible to automatically verify the computational reproducibility of data and code. The extent to which automated processes can reduce the workload associated with checks of data and code sharing practices is an open question. Automation has the potential to reduce the workload of manual checks, but it is not clear whether and where automated tools can achieve the same level of accuracy as human coders.

In this paper we will examine how Metacheck, a novel open source tool that checks best practices in data and code sharing, might assist in the process of checking whether data and code are shared. We will compare the output of Metacheck against the manually coded ground truth of Cooper et al. (2026b), available at https://github.com/nhcooper123/reproduce-reuse-recycle. It is clear that the human coders exerted considerable effort over the 2-day hackathon to code the 1861 papers, where 145 people coded a median of 10 (min = 1, max = 68) manuscripts. Cooper et al.'s protocol covered the seven journals published by the British Ecological Society (BES), coding papers published between 2017 and 2024 on eight questions per paper. The team coded whether data and code were used and archived, where they were archived, whether the archived files could be located, downloaded, and opened, what file formats were used, whether a README was present and how useful it was, how complete the archived data were, how well-annotated the code was, and whether the data and code were citable (i.e., had a DOI and a licence).

Coding the presence of absence of data and/or code files is exactly the kind time-consuming task that automated tools such as Metacheck (DeBruine et al., 2026) aim to support. If an automated tool could complete some of Cooper et al.'s eight coding questions at comparable accuracy to their trained human coders, it would substantially reduce the workload that is currently required to check if researchers follow best practices in data and code sharing. Metacheck (DeBruine et al., 2026) is a modular, open-source tool that uses text search, rule-based classification, API's, and optional large language models to extract information about research practices from scientific manuscripts and their associated repositories. Metacheck's `repo_check`, `data_check`, and `code_check` modules perform automated checks that closely resemble the classifications that Cooper et al. (2026b) asked their human coders to perform.

In this paper, we use Cooper et al.'s openly archived data as ground truth (Cooper & the BES Data and Code Hackathon Group, 2026a), and for each of the eight questions in their protocol, we summarize what Cooper et al. found, and examine whether that question can currently be automated with sufficient accuracy. The results generated by Metacheck are completely automated, do not rely on AI, and can be reproduced by re-running the code, without any human input. Unlike a validation study that reports a single accuracy percentage, our goal here is to make transparent where the current version of Metacheck can reduce the manual burden of this kind of meta-scientific data collection, and where it cannot.

As an automated tool, Metacheck follows FAIR principles, which state that "all research objects should be Findable, Accessible, Interoperable and Reusable (FAIR) both for machines and for people" (Wilkinson et al., 2016, p. 3). Human coders can retrieve information that is

not accessible by a machine. For example, if authors state that the data is shared in a PDF file shared as supplementary material the data is not FAIR, because a machine can't find or access the datafile. Cooper and colleagues did not limit their coding to FAIR data, and as a consequence, Metacheck will always miss some information that human coders found. On the other hand, machines can find some information that human coders miss. For example, the Dryad data repository requires a CC0-1.0 license at deposit for every dataset with no author choice involved, and adds this information to the metadata. Metacheck retrieves this information, but the human coders were unable to locate license information by visually examining the Dryad repository. Throughout this project we will also reflect on practical challenges when developing software that attempts to automatically retrieve FAIR data, and the implications for best practices in data and code sharing.

Methods

We obtained the list of paper DOIs, journals, and years directly from the cleaned dataset in the public repository, and downloaded all articles through an R script. All articles were converted to structured text using GROBID 9.0, and converted to Metacheck paper objects. We ran the `repo_check` (locates and lists files in online repositories), `data_check` (classifies files and extracts information about tabular data), and `code_check` (checks R, Python, SAS, SPSS, Stata, Mplus, and MATLAB code files) modules to mirror to manual coding performed by Cooper and colleagues. Subsequently, we ran the `reproducibility_check` module to illustrate additional benefits of automation.

1. Does the paper use data/code? If so, are they archived?

Cooper and colleagues reported that "Of the 1861 papers in our dataset, 93% (n = 1735) used data and 90% (n = 1670) used code. Of these papers, 97% (n = 1690) archived their data, but only 35% (n = 577) archived their code". The first two percentages are based on text in data availability statements and the method section. There are tools that automatically check data availability statements, such as OddPub (Riedel et al., 2020). Metacheck itself has an equivalent module, `open_practices`, which reads the full text and judges from the wording alone whether open data and/or open code was shared. These automated checks most closely follow the coding by Cooper and colleagues, in that data and code is classified as available irrespective of whether authors followed FAIR principles. Metacheck's `repo_check` module takes a different approach and classifies data and code as available based on a linked repository that can be automatically accessed through an API.

Oddpub and Open Practices Check

OddPub (Riedel et al., 2020) is a tool that searches text for keywords related to data and code sharing. The Metacheck Open Practices module does the same, but is intentionally

over-inclusive, which means it is designed to find more data or code sharing statements, while also returning more false positives. We compared the automatic text-based judgements from OddPub and the Open Practices module to Cooper and colleagues' human coding of whether data and code were openly shared (their `data_open`/`code_open` columns), treating "Yes" (and for data "Yes, but not all files") as open and "No" as not open, and excluding papers where the coders used an ambiguous label (e.g., "Needs specific software or too large") or left the item unscored (`data_open`/`code_open` were only asked for papers Cooper and colleagues had already coded as archiving their data/code, and not every such paper has a recorded answer, so the denominators below are smaller than the archiving counts reported earlier). For data sharing (1552 papers with a clear human judgement), OddPub identified 80% of the papers the human coders judged to share open data (sensitivity), and correctly identified 45% of the papers judged not to (specificity), for an overall agreement of 80%. Metacheck's `open_practices` module agreed with the human coders on data sharing 86% of the time, with higher sensitivity (86%) but lower specificity (9%) than OddPub. For code sharing (498 papers with a clear human judgement), the difference was larger: OddPub agreed with the human coders 44% of the time (sensitivity 44%, specificity 67%), whereas `open_practices` agreed 78% of the time (sensitivity 78%, specificity 0%). Both tools, in other words, are much more likely to say a paper shares open data or code than to say it does not; this corpus contains very few papers the human coders judged as clearly not sharing (11 for data, 3 for code), so the specificity estimates above are based on very small numbers and should be treated as indicative rather than precise. `open_practices` and OddPub both determine open data/code from the text of the article alone, and neither distinguishes between data and code repositories the way `repo_check` and `code_check` do below.

Note that OddPub and `open_practices` classify every paper regardless of how Cooper and colleagues' coders scored this question. For data sharing, 97 papers archived their data (`data_availability == "Yes"`) but were left unscored on whether it could be opened, of which OddPub judged 44 (45%) and `open_practices` judged 80 (82%) to have open data; a further 41 were explicitly flagged by Cooper and colleagues' coders as needing specific software or too large to open, of which OddPub judged 24 (59%) and `open_practices` judged 31 (76%) to have open data. For code sharing, 49 papers archived their code (`code_archived == "Yes"`) but were left unscored on whether it could be opened, of which OddPub judged 8 (16%) and `open_practices` judged 25 (51%) to have open code; a further 34 were explicitly flagged as possibly accessible with the right software, of which OddPub judged 16 (47%) and `open_practices` judged 24 (71%) to have open code.

Repo check, Data check, and Code check

Cooper and colleagues found that 1690 papers archived their data and 577 their code, and in total, 1789 papers contain a link to a repository with either data or code. In total, Metacheck found 1555 papers that contained a link to a repository in the main text where it could verify that the repository contained files. Of these, 1462 papers contained data, 440 papers contained code, and 1515 papers contained either data or code according to `repo_check`. The manually

coded classifications agree with the automatic classification by Metacheck for 1498 papers, and disagree for 308 (291 where Cooper's coders found data or code that Metacheck missed, and 17 papers where Metacheck found data or code while Cooper's coders missed it).

For Metacheck, data and code is findable and accessible if it is archived in one of the repositories that Metacheck supports (through its support for Dataverse, DSpace 6 and 7, Figshare, and DataOne, and dedicated support for OSF, Zenodo, ResearchBox, Dryad, ReShare, GitHub, GitLab, Mendeley Data, FSD, AsPredicted, and 4TU). Because many repositories use the same underlying API, Metacheck supports over 285 unique repositories (121 Dataverse installations, 66 DSpace 7 installations, 7 legacy DSpace 6 installations, 5 DataOne member nodes, and 76 Figshare-based hosts, plus 11 dedicated single-platform integrations), but this is only a subset of all repositories used in manuscripts.

We reviewed the 291 papers where Cooper and colleagues' coders found data or code that Metacheck missed by checking the repository to confirm what it actually contains. This analysis revealed five clusters of explanations for where MEtacheck misses data and code in a repository (see Table 1). In 18 papers we found no reference to a repository or link to a dataset in the manuscript, most often because the data were explicitly withheld for confidentiality, ethical, or Indigenous-data-sovereignty reasons. These papers might make data available upon request, but are arguably better classified as papers that explain why data could not be shared, than as papers that share data. In 85 papers the citation or the repository did not meet basic FAIR requirements, so no automated tool could have retrieved the data: the paper cites a private reviewer or share link instead of the dataset's public identifier (7 papers), or an organisation or user page that identifies no specific deposit (2); or the repository blocks automated access (6), was not live or accessible when checked (14), or offers no API through which a machine can retrieve the data (56).

In 98 papers data is shared on a repository that is FAIR, but that Metacheck does not yet have code to automatically download files from. Metacheck supports the most commonly used repositories, but it does not yet have support for some field-specific or institutional repositories that occur less frequently in the published literature (for example, the US Geological Survey's ScienceBase, the Environmental Data Initiative, the NERC Environmental Information Data Centre, and PANGAEA, the earth- and environmental-science data publisher). We found 28 papers that shared data or code in a FAIR repository, but in a format Metacheck is not able to automatically classify as data or code. This occurred when data was shared in non-FAIR formats (e.g., a table in a PDF file), or data or code files in a plain .txt file without labeling the file as data or code (which makes it difficult to differentiate from output or documentation files in a repository). But another reason was that the file extension is not in the list of known data types (e.g., GeoTIFF files), drone photographs that are the measurement data while Metacheck reads these as image material rather than data, or archives in the `.rar` or `.7z` formats, which base R cannot open.

For the remaining 62 papers the repository is FAIR and on a platform Metacheck supports, but a technical limitation prevented Metacheck from finding or using it. In 30 papers the problem arises before Metacheck sees the paper: converting the PDF to text corrupted the identifier,

most often by inserting spaces into a DOI (24 papers), or dropped the link altogether, for example when it sits in a section after the reference list that GROBID does not extract (6). In 32 papers the problem lies in Metacheck's own code: it does not search the reference list for identifiers, does not follow Handle or shortened links to a repository it supports, failed on a batch of papers or kept a failed lookup in its cache during the corpus run, did not open an archive (in some cases by design, for archives far larger than its download limit or packed inside another archive), or discarded an R package's own source files (see below).

The full per-paper record for every one of these 291 papers is available in `code/03_comparing_results/manual_re` and `manual_fair_api_findability_audit.csv`, so any individual classification can be checked and disputed.

Table 1: Table 1. Reasons and frequencies of papers where manual coders found data or code while Metacheck did not.

Cluster	n papers
No repository is cited anywhere in the paper's own text	18
Repository contains data or code that Metacheck did not classify (a format Metacheck does not yet support, or data or code not shared in a FAIR format)	28
Citation or repository is not FAIR (a private or non-specific link, a repository that blocks automated access or is not live, or a platform without an API)	85
Repository is FAIR, but on a platform Metacheck does not support	98
Technical limitation: the repository is FAIR and on a supported platform, but the PDF-to-text conversion or a defect in Metacheck's own code prevented detection or classification	62

Among the code limitations, 11 papers are cases where `repo_check` correctly found the repository, but a defect or limit in Metacheck's own code meant the files holding the paper's data or code were never classified. 6 are compressed archives (`.zip` or `.tar` family, formats Metacheck can open) that the recorded run listed but never opened, so their contents were never classified: two are single archives of 15.8 and 18.6 GB, far above Metacheck's archive download limit (at most about 1 GB with default settings), one is a `.tar.gz` that Metacheck's download step skips because its rule for which files to fetch never selects a file whose type is still unknown, for two small archives on Dryad we could not establish why they were not opened, and one Dryad dataset consists of two `.zip` archives packed inside the dataset's own archive: Metacheck by design opens only one level of archive, so these inner archives are not opened and their contents are not classified. The remaining 5 are R packages shared as their own GitHub repository. Metacheck deliberately drops the files of an R package found at a repository's root unless the repository also holds other real data or code, so that a bundled copy of someone else's package is not counted as the study's own code; but it counts continuous-integration configuration files (such as `.travis.yml` and `codecov.yml`) as code, so for these repositories

it concludes that other code exists and the exclusion rule misfires, discarding the package's real source files (scienceverse/metacheck#469). We confirmed this mechanism in detail for one of these repositories; the other four have the same structure.

2. Where were the data/code archived?

Cooper and colleagues coded the specific repository where data and/or code was stored. They observed that some data stored on GitHub was also stored on Zenodo, in line with best practices (as GitHub is not a stable data or code repository). We observed this instability directly: a GitHub repository cited by one paper in the corpus (the CVAT annotation tool, cited in the GinJinn2 paper) changed between our first Metacheck run and a later rerun. Files had been removed, renamed, and added in the meantime, so Metacheck found 787 code files in the rerun where it had found 752 before. A deposit on Zenodo or Dryad is a fixed version, so checking it again returns the same files; a GitHub repository returns whatever it contains on the day it is checked, which means an automated check of it cannot be exactly reproduced later. Cooper et al. report Dryad as the dominant data repository in ecology (57%, n = 1022), with Figshare (9%), Zenodo (12%), and "various institutional and government repositories" (17%) making up most of the remainder (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). Cooper et al. coded the platform holding the data (`data_archive`) and, separately, the platform holding the code (`code_archive`); both questions allow more than one platform per paper. Metacheck records, for each paper, the platform of every repository in which `data_check` classified at least one file as data (`mc_data_archive`), and the platform of every repository in which `code_check` found at least one code file (`mc_code_archive`); these can also list more than one platform. We therefore compare the two sides as sets of platforms. Before comparing, both sides are put on Cooper et al.'s own label scale: their protocol has no separate option for the Open Science Framework, so OSF deposits are coded "Other repo/database", and Metacheck's "OSF" label is mapped to that category; Cooper et al.'s "CRAN" code label is likewise an "Other repo/database" in Metacheck's scheme. Cooper et al.'s "Supplementary materials" and "Personal website" labels are not repositories Metacheck checks at all and are left out of Cooper et al.'s set, so a paper coded only with these labels (10 papers for data, 38 for code) is not comparable. Each comparable paper then falls into exactly one category: the same set of platforms (exact agreement); Metacheck lists only some of Cooper et al.'s platforms; Metacheck lists all of Cooper et al.'s platforms plus at least one more; each side lists a platform the other does not; or the two sides share no platform at all.

Table 2: Table 6. Agreement between Cooper et al.'s coded archive platform(s) and platform(s) of the repositories in which Metacheck found data or code, compare sets of platforms.

Platform	Comparable papers	Same platforms	Metacheck lists fewer	Metacheck lists more	Each lists one the other lacks
Data	1,393	1255 (90.1%)	55 (3.9%)	48 (3.4%)	2 (0.1%)

Platform	Comparable papers	Same platforms	Metacheck lists fewer	Metacheck lists more	Each lists one the other lacks
Code	401	310 (77.3%)	57 (14.2%)	20 (5.0%)	2 (0.5%)

For data, 1393 papers were comparable. The two sides named exactly the same platform(s) for 1255 (90.1%) and at least one shared platform for 1360 (97.6%). Multi-platform answers are common on both sides (86 comparable papers for Cooper et al., 81 for Metacheck), and most of the remaining differences are about completeness rather than a wrong platform: in 55 papers Metacheck found only some of the platforms Cooper et al. coded, most often missing a second, "Other repo/database" repository (an institutional or domain-specific database) cited alongside a Dryad deposit, the platform-coverage gap documented in the review of missed repositories above; and in 48 papers it found an additional platform holding data-classified files that Cooper et al. did not list as a data platform, most often a GitHub repository holding data files alongside the analysis code. Only 33 papers (2.4%) share no platform at all.

For code, 401 papers were comparable. Exact agreement is lower, 310 (77.3%), although the two sides share at least one platform for 389 (97.0%). The difference is driven by papers in which Metacheck lists fewer platforms than Cooper et al. (57 papers): code is frequently shared in two places at once, typically a GitHub development repository together with its archived Zenodo release (114 comparable papers list more than one code platform in Cooper et al.'s coding), and in most of these papers Metacheck found code files in only one of the two. Only 12 papers share no platform at all.

Limitation. These comparisons depend on Metacheck classifying files: a repository only counts towards `mc_data_archive` or `mc_code_archive` if Metacheck found a data or code file in it, so every reason Metacheck fails to classify a file (an unsupported format, data not shared in a FAIR format, an unopened archive; see the review of missed repositories above) also removes that repository's platform from Metacheck's answer. The comparison also only covers papers both sides coded, so it does not include papers where Metacheck found no data or code repository at all; those are covered by the `data_availability` and `code_archived` comparisons.

`mc_data_availability`. `TRUE` if `repo_check`'s `summary_table$files_n` is greater than zero for a paper — that is, if `repo_check` found at least one linked repository whose file listing it could retrieve, with at least one file in it, of any type (data, code, documentation, or unclassified). This rule is deliberately permissive: it answers "did we find a repository with something in it," not "did we find a repository holding a dataset." That distinction matters in practice (see Results below): a paper whose only linked repository is a pure-code Zenodo archive, with zero data-classified files, still scores `mc_data_availability = TRUE` under this rule, because the rule does not check `data_type`.

`mc_data_archive`. The archiving platform, derived by matching each linked repository's `repo_url` against a fixed set of domain and DOI-prefix patterns: `osf.io` for OSF; `github.com/gitlab.com` for "GitHub, GitLab, Codeberg or similar platform"; `zenodo.org` or

the 10.5281/zenodo DOI prefix for Zenodo; datadryad.org or the 10.5061/dryad DOI prefix for Dryad; figshare.com or the 10.6084/m9.figshare DOI prefix for Figshare; anything else is labelled "Other repo/database." When a paper cites more than one repository holding data-classified files, mc_data_archive lists every matching platform, semicolon-joined; Cooper et al.'s protocol also allows several platforms per paper, so the two are compared as sets of platforms (see Results). mc_code_archive is derived the same way from every repository in which code_check found at least one code file, and is compared against Cooper et al.'s code_archive.

mc_data_doi, mc_data_license. Read from repo_check$repo_metadata, a table populated only for the platforms whose API repo_check queries for metadata beyond the file listing itself (Zenodo, Dryad, Figshare, GitHub's repository object, and — when osf_license = TRUE is set, as it was for every run reported here — OSF). mc_data_license is TRUE if at least one repository holding a data-classified file has a non-missing license value in this table; mc_data_doi is that repository's own doi field. This scoping choice — checking only repositories that hold data — is deliberate and matches mc_data_archive's own scoping, but it means a paper whose licensed repository holds no data-classified files (a common pattern when data and code sit in physically separate repositories; see Results) scores mc_data_license = FALSE even though a real, retrievable license genuinely exists for that paper's overall archive.

repo_metadata$license is read verbatim from each platform's own API response, with no rewriting or standardisation on Metacheck's side — this means its format varies by platform (a bare SPDX identifier like "MIT" or "GPL-3.0", a human-readable string like "CC BY 4.0", or a full SPDX URL), and it can be identical across many repositories on the same platform when that platform's own licensing policy is uniform, not because Metacheck is returning a cached or default value. Concretely: 1057 of the 1057 licensed Dryad repositories in this corpus (100.0%) carry the identical string, https://spdx.org/licenses/CC0-1.0.html, because Dryad requires CC0-1.0 at deposit for every dataset with no author choice involved; this value never appears for a non-Dryad repository in this corpus (0 occurrences among Zenodo, Figshare, GitHub, and OSF repositories combined), whose own license values vary appropriately by repository and platform.

mc_data_download. TRUE if at least one data-classified file has data_check's own tabular_usable == TRUE **and** a non-missing file_location (the local path the file was actually fetched to). This is a two-part requirement, not download success alone: a file can download successfully and still fail this check if data_check judges its content unusable as a table (see the Excel multi-sheet limitation in Results). mc_data_format is the sorted, semicolon-joined set of file extensions among the files that passed this same tabular_usable check.

mc_any_readme. TRUE if repo_check classified at least one file, in any of the paper's linked repositories, with doc_role == "readme" — a purely filename-based match (a base name starting with "readme", case-insensitive, or a package-level ro-crate-metadata.json), never a check of the file's content. Critically, Metacheck records only **one** README signal per paper, computed across all of that paper's repositories combined; it does not distinguish

a README documenting data from one documenting code. Cooper et al.'s protocol, by contrast, asks two separate questions (`data_README`, `code_README`). We therefore compare against `cooper_any_readme`, defined as `TRUE` if Cooper's coders answered "Yes" or "Quasi-README" to either of their two README questions for that paper — the only comparison that avoids double-counting the same single Metacheck signal against two different Cooper columns (an earlier, columnwise comparison inflated the disagreement count by 41% purely from this double-counting, before we corrected it; see Results).

`mc_code_archived`. `TRUE` if `code_check`'s file table contains at least one row for the paper at all — that is, if `code_check` classified at least one file (of any language) as code, regardless of whether that file was ever successfully downloaded. `mc_code_download` is the stricter follow-up: `TRUE` if at least one of those code-classified files additionally has a non-missing `file_location`. `mc_code_language` is the sorted, semicolon-joined set of `code_check$table$language` values among a paper's code files; language is assigned by `code_lang()`, which is content-aware rather than purely extension-based for polyglot formats (`.Rmd`/`.qmd` and `.ipynb` are opened and their code cells inspected; an ambiguous `.html` is content-sniffed).

`mc_code_used`. Always `NA`. No Metacheck module answers "did this paper use code" as a construct distinct from "is there a code file sitting in a linked repository" — the two questions collapse into one from Metacheck's perspective, since it has no way to determine from the manuscript text alone whether custom code was part of the reported methodology.

Columns Cooper et al. coded that this comparison does not attempt. Cooper et al.'s protocol also includes README/code usefulness ratings (a 1-10 human-judged scale), a four-point data completeness judgment, and a code annotation quality score. None of these has a Metacheck counterpart to compare against: `code_check`'s `percentage_comment` field measures comment density (the proportion of lines that are comments), a real and precisely computed metric, but not a judgment of whether those comments are actually informative, so we do not force it into a comparison against Cooper et al.'s quality scale here. These three constructs are discussed qualitatively, not quantitatively, in the Discussion.

Empirical Comparison

We compare Metacheck's `mc_`-prefixed columns against Cooper et al.'s own coded ground truth across the 1861 papers we were able to retrieve, for every construct where a genuine boolean or categorical comparison is possible: whether data are available at all, where data are archived, whether a data license is present, whether data download and code archiving, and README presence (collapsed across data and code, per the Coding section above). For each construct we report: (1) how many papers both Cooper et al. and Metacheck coded a value for (`n_both`, since neither side coded every paper — Cooper et al. did not code a "not applicable" paper on every sub-question, and Metacheck cannot code a paper whose PDF we could not retrieve or convert), (2) a 2x2 confusion table of agreement and disagreement, (3) sensitivity (of the papers Cooper et al. coded `Yes`, the percentage Metacheck also coded `Yes`), specificity (of the papers

Cooper et al. coded `No`, the percentage Metacheck also coded `No`), and McNemar's test for the asymmetry of disagreement, (4) the two directional miss rates the reader is likely to care about most in practice: of the disagreements, what percentage are cases where Cooper et al. found something Metacheck missed, and what percentage are cases where Metacheck found something Cooper et al.'s human coders missed, and (5) a qualitative synthesis of every individually verified disagreement, drawn from a case-by-case review in which every one of the 890 reviewed disagreeing papers (all of them) was checked directly against Metacheck's own saved module output, the paper's own extracted full text, and — where a repository platform's API was the evidence source — a live API call at the time of review, and assigned one of five verdicts: `COOPER_RIGHT` (Metacheck's answer does not match the paper/repository), `METACHECK_RIGHT` (Cooper et al.'s coded answer does not match the paper/repository), `BOTH_DEFENSIBLE` (the paper genuinely supports both answers, most often because it cites more than one repository), `DIFFERENT_DEFINITION` (the two sides are answering different questions that happen to share a column label — see the README discussion below), or `UNCLEAR` (the available evidence does not resolve the disagreement either way).

Following a full corpus rerun against a further round of fixes (Appendix A.1), the disagreement set itself was rebuilt from scratch rather than assumed unchanged: `data/cooper_vs_recreated.RData` was regenerated from the fresh `repo_check/data_check/code_check` output, and a new disagreement worklist was derived from it using the same mechanical flagging rule as before. For every (paper, column) disagreement whose Cooper-coded value and Metacheck-derived value were both unchanged from the prior review round, the earlier verdict and its evidence were carried forward rather than re-derived. Every disagreement that was new, or whose Metacheck-derived value had changed as a direct result of the fixes, was reviewed from scratch, following the same evidence-source rule throughout: `data_availability`, `data_archive`, and `code_used` were checked against the paper's own extracted text, since these are what Cooper et al.'s coders themselves read; `data_license`, `data_download`, `code_archived`, `code_download`, and `any_readme` were checked against Metacheck's own saved module output (`repo_check`'s file listing and `doc_role` classification, `repo_metadata`'s license/DOI fields, `file_location`), since a paper's own text essentially never states whether a README exists or a download succeeded. A live platform API call (Dryad's dataset API, Zenodo's records API, Figshare's articles API, GitHub's repository API) was used only where local evidence was insufficient to decide. Appendix A.1 documents the specific software defects this case-by-case process surfaced and fixed over the course of the project.

The numbers reported in this section reflect a SECOND such rebuild, following Appendix A.5's seven further fixes — applied and reverified this time against the FULL 1,861-paper corpus (not just the previously-disagreeing subset A.5 itself checked), using the identical carry-forward procedure: `cooper_vs_recreated.RData` was regenerated from a fresh `repo_check/data_check/code_check` rerun of every paper, the disagreement worklist was rebuilt (`code/02_create_comparison_data/02_build_disagreement_worklist.R`), and each prior verdict was carried forward wherever its underlying (paper, column, Cooper value, Metacheck value) combination was unchanged; disagreements that were newly surfaced or changed by A.5's fixes were reviewed from scratch, following the

same evidence-source rule described above. The per-row verdicts and reasoning are in `code/02_create_comparison_data/disagreement_review_worklist.xlsx`. All 890 disagreeing papers carry a verdict for every column on which they disagree. Each verdict records the date it was written and, for verdicts written from 10 October 2026 onwards, the exact Metacheck version it was checked against (the same version record as the audit files of the missed-repository review); verdicts written earlier carry only a "no later than" date, because the version was not recorded at the time.

This final round of review surfaced one further genuine software defect — and, unlike every other defect this project fixed and moved on from, this one was caught, root-caused, fixed, and reverified within the same review cycle rather than left open. Appendix A.5's R-package-source-tree exclusion (intended to drop a bundled dependency's own source tree — `DESCRIPTION/NAMESPACE/R//man//tests/` — from a repository's data/code listing, since that is reusable infrastructure rather than the study's own analysis code) turned out to also strip a repository's real analysis code whenever the paper's own code was itself organised as an installable R package, which is common practice for ecological modelling and analysis tools shared alongside a methods paper. Confirmed across 16 real GitHub/CRAN-hosted repositories (`RevGadgets`, `BaBA`, `m2b`, `RRPP`, `actel`, `scampr`, `MINOTAUR`, `LadderFuelsR`, `biomonitoR`, `EcoEnsemble`, `fossilsim`, `frair`, `tRophicPosition`, `sabinaNSDM`, `bef`, and `phylomice`), `repo_check`'s file listing for each package's repository contained only its build/configuration/documentation scaffolding (`.Rbuildignore`, `.gitignore`, `NEWS.md`, `README.md`, `cran-comments.md`, a bare `.Rproj` file) with the package's actual `R/` source directory entirely absent — and, in several of these repositories, the package's own bundled `data/` directory was stripped along with it, silently turning a genuine data deposit into an apparent absence of data rather than merely a wrong-but-visible code disagreement.

The fix distinguishes two structurally different cases that both fingerprint identically (a `DESCRIPTION+NAMESPACE` package at a repository's root): a study that shares its own data and analysis code as an R package (`BaBA`'s tracked-animal datasets and behaviour-classification functions, for instance) versus a repository that happens to bundle a vendored copy of someone else's general-purpose package alongside the study's own files (confirmed against `MarshallLab/MGDrivE`, a general-purpose mosquito gene-drive simulation package whose paper-specific fork nests the vendored package one level deep, with the paper's own `Examples/` scripts and data sitting as siblings at the repository root). The corrected rule only excludes a root-level package's files when nothing classifiable as real data or code survives outside it (checked with `data_classify_files()`, the same rules-based classifier `data_check` itself uses, so "real content" means the same thing everywhere in the pipeline); a package detected in a nested subfolder — the vendored-dependency shape — is still always excluded, since by construction there is always something else in the repository for a nested package to sit alongside. Reapplying this corrected rule to the 38 repositories flagged by a corpus-wide scan for the exclusion's symptom (a repository whose only surviving files, after the old exclusion ran, were scaffolding) resolved 11 of the 16 confirmed cases; 5 remain flagged. For these, the file listing is complete, but the corrected rule still misfires: continuous-integration configuration files at the repository root (such as `.travis.yml` and `codecov.yml`) are classified as code by their

extension, so the rule concludes that real code exists outside the package and keeps excluding the package's own source files. The rule already makes an exception for RStudio project files for exactly this reason, but not for continuous-integration files. This is a separate, still-open defect (scienceverse/metacheck#469).

Is any data available at all?

Cooper et al. report that "97% (n = 1690) archived their data" among the 1735 papers coded as using data (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1958). `mc_data_availability` and Cooper et al.'s `data_availability` were both codeable for 1735 papers. They agreed on 1478 (85.2%), disagreed on 257 (14.8%). Sensitivity was 86.3% and specificity was 44.4%, McNemar's $\chi^2(1) = 165.12$, $\underline{p} < .001$. This is a considerably more balanced pattern than an earlier pass of this corpus found, because `mc_data_availability`'s own derivation was corrected during this project (see Coding section): it now reflects `repo_check` finding at least one linked repository with at least one file, rather than defaulting to a missing value whenever `repo_check` found none at all — so a paper with no detectable repository now correctly scores `FALSE` and enters this comparison, rather than being silently excluded from it.

Reviewing all 257 disagreements individually: 216 were `COOPER_RIGHT`, 8 `METACHECK_RIGHT`, 23 `DIFFERENT_DEFINITION`, and 10 `UNCLEAR`. The dominant cause among the `COOPER_RIGHT` cases (178 of 216) is `repo_check` never detecting a repository at all, even though the paper's own text names one explicitly — most often a real, working DOI or URL on a platform `repo_check` has no dedicated backend for (institutional repositories such as DataSTORRE, DIGITAL.CSIC, or the Australian Antarctic Data Centre; domain-specific databases such as NCBI's Sequence Read Archive/GenBank, the Knowledge Network for Biocomplexity, or Neotoma), or occasionally a citation malformed by PDF text extraction. A further 3 trace to a `repo_check` processing error logged internally (an R-level error raised while fetching an otherwise-correctly-detected repository, or the host platform being transiently unreachable during the corpus run) rather than a missed citation — a genuine, fixable software defect distinct from a coverage gap. Conversely, 8 disagreements were `METACHECK_RIGHT`: clear, working Dryad, Zenodo, Figshare, or GitHub data-availability statements that Cooper et al.'s coders appear to have missed or coded `No` in error. 23 were `DIFFERENT_DEFINITION` — most commonly a paper stating data are available "on request," embedded directly in the article's own Supporting Information, or withheld for confidentiality reasons, which Cooper et al.'s protocol appears to credit as "available" while `mc_data_availability` structurally cannot, since there is no externally checkable repository for `repo_check` to find. The remaining 10 could not be resolved from the available evidence.

Limitation. The mechanism responsible for `mc_data_availability`'s disagreements has changed since an earlier pass of this corpus: rather than a single, blanket rule failure that made a `FALSE` structurally impossible, the remaining gap is now dominated by `repo_check`'s coverage of repository platforms and citation formats — addressable by extending its URL-recognition

patterns to more institutional and domain-specific repositories, rather than by changing the derivation rule itself.

Is the data licensed?

Cooper et al. report that 85% (n = 1438) of papers with archived data had a license (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1960). `mc_data_license` and Cooper et al.'s `data_license` were both codeable for 1314 papers. They agreed on 1256 (95.6%), disagreed on 58 (4.4%). Sensitivity was 99.1% and specificity was 25.4%, McNemar's $\chi^2(1) = 21.12$, $p < .001$.

All 58 disagreements were individually resolved against the relevant repository platform's own API, called live at the time of review. 46 were `METACHECK_RIGHT`: a real, retrievable license genuinely exists — most commonly CC0-1.0 for Dryad datasets (Dryad requires a license at deposit) or a depositor-selected "Other (Open)" tag for Zenodo records, or a GitHub repository license confirmed live via GitHub's own repository API. 9 were `COOPER_RIGHT`, and the large majority share the same repository-scoping mechanism documented for `mc_data_archive` above: a licensed repository genuinely exists for the paper, but `data_check` classified zero of its files as `data`-typed (often because the only file is an archive never opened, or because the licensed repository is a separate code deposit rather than the data-holding one), so `mc_data_license`'s data-only scoping rule never reaches it; a smaller number trace to `repo_check` never detecting the licensed repository at all. The remaining 1 could not be resolved because `repo_check` found no `repo_metadata` row at all for the relevant repository.

Limitation. `mc_data_license` inherits `mc_data_archive`'s repository-scoping limitation, since both restrict themselves to "the repository holding data." It remains one of the more reliably automatable constructs in this comparison in the METACHECK_RIGHT direction: every case where Metacheck found a license was independently reconfirmed against the same live API `repo_check` itself reads from.

Can the data be downloaded and used?

Cooper et al. report that, of archived datasets, "95% could be downloaded, and 91% could be opened" (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). `mc_data_download` and Cooper et al.'s `data_download` were both codeable for 1328 papers. They agreed on 1315 (99.0%), disagreed on 13 — a small enough count that every case was individually traced rather than summarized by category.

Of the 13 disagreements, 5 were `COOPER_RIGHT`: files that downloaded successfully but still failed `data_check`'s tabular-usability check for a real, distinct reason each time — a space-delimited scientific data format (climate-model output with a distinctive multi-part naming convention) the tabular reader does not recognise, an unusual two-row header layout, and other content-specific parsing gaps rather than a single repeatable bug. 3 were `DIFFERENT_DEFINITION`: the

data genuinely downloaded and opens correctly in standard software, but `mc_data_download`'s own criterion (see Coding section) additionally requires `data_check`'s stricter `tabular_usable` flag, which does not always agree with "a human can open this file just fine." 3 were `METACHECK_RIGHT`: a downloaded file that, on direct inspection, was a serialized model-input dump with no valid row/column structure at all (so `tabular_usable = FALSE` was objectively correct, not a detection gap), a clean, working OSF link the paper's own text supports with nothing to contradict it, and a KNB record a prior fix (scienceverse/metacheck#435) now correctly finds and downloads, which Cooper et al.'s `No` most likely reflects the pre-fix state of. The remaining 2 could not be independently confirmed, since the paper's own text gives no detail on whether the data genuinely opens beyond a bare repository link — exactly the kind of live-repository fact a text-only check cannot resolve either way.

Limitation. This construct's very small remaining disagreement count (13 papers, 1.0% of the codeable base) reflects fixes applied earlier in this project (a multi-sheet Excel reading limitation, scienceverse/metacheck#379; see Appendix A.1 for the full mechanism and before/after numbers from that specific fix). What remains is a heterogeneous residual of genuinely distinct format- and content-specific parsing gaps rather than one further systematic cause.

Is code archived, and can it be downloaded?

Cooper et al. report that "only 35% (n = 577) archived their code" among papers using code (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1958), and, among archived code, that most is on Zenodo or GitHub-family platforms (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). We report `mc_code_archived` and `mc_code_download` together, since the qualitative review of both surfaced overlapping causes.

`mc_code_archived` and Cooper et al.'s `code_archived` were both codeable for 1669 papers. They agreed on 1460 (87.5%), disagreed on 209 (12.5%). Sensitivity was 69.7% and specificity 96.9%, McNemar's $\chi^2(1) = 93.78$, $\underline{p} < .001$. `mc_code_download` and Cooper et al.'s `code_download` were both codeable for 395 papers (a much smaller base, since this sub-question only applies when code was coded as archived in the first place); they now agree on 394 of 395 (99.7%), with 1 disagreements remaining.

`code_download`'s current agreement reflects a substantial, multi-stage improvement over the course of this project, documented fully in Appendix A.1 (from 66.2% agreement before any fix, through successive intermediate rounds, to the current 99.7%). `code_archived` moved much less across the same fixes, consistent with its dominant causes being ones those fixes did not target.

209 of the 209 `code_archived` disagreements and 1 of the 1 `code_download` disagreements have been individually reviewed. `code_archived`'s 155 `COOPER_RIGHT` cases are genuinely multi-causal:

1. **Code bundled inside the same repository as the data, with `code_check` finding no file to classify as code there (21 cases).** A recurring, precisely worded pattern: a Data Availability Statement reading, for instance, "Data and R code available from the Dryad Digital Repository" — the paper's own text explicitly confirms code is bundled with the data in one deposit, `repo_check` found that repository, but `code_check` classified none of its files as code (for the same reasons documented for the not-classified and Metacheck-limitation clusters above: a format or programming language Metacheck does not yet support, code not saved with its language's file extension, or an archive that was never opened).
2. **`repo_check` never detected the repository at all (42 cases),** despite the paper's own text explicitly naming it — the same platform-coverage and citation-extraction gap documented for `data_availability` above.
3. **A `repo_check`-side processing error, logged internally (1 cases)** — the repository was correctly identified but its fetch failed with an R-level error or a transient host outage during the corpus run.
4. **A more heterogeneous remainder (86 cases)**, including code provided only as journal Supporting Information rather than in any linked repository, citations to a third-party dependency's repository rather than the authors' own code, and `repo_check`-side file-classification errors (e.g., shapefile metadata files misclassified as code).
5. **The R-package-source-tree exclusion defect discussed above (5 cases remaining),** where continuous-integration configuration files counted as code make the corrected exclusion rule misfire (scienceverse/metacheck#469).

A smaller number of `code_archived` disagreements ran the opposite direction (31 `METACHECK_RIGHT`): 10 are real, substantial, multi-file GitHub or Zenodo code repositories that `code_check` correctly found and classified, coded `No` by Cooper et al. — a large, unambiguous set of named analysis scripts is not the kind of borderline case a coding protocol's edge cases would plausibly explain, so these look like straightforward human-coder misses; a further 9 are papers whose own text and Data Availability Statement mention no code, script, or software at all, so Cooper et al.'s `Yes` has no traceable basis in the paper's own evidence. 5 were `DIFFERENT_DEFINITION`: code described only as pseudocode or a script embedded in the journal's own Supporting Information, which Cooper et al.'s coding appears to credit as "archived" while `code_check` structurally cannot see it (it is not in any linked repository). The remaining 18 were `UNCLEAR`, generally because the extracted paper text contained no relevant statement either way.

Before a final targeted rerun, 25 `code_download` disagreements were `COOPER_RIGHT` and shared a single cause: Metacheck found the paper's code files, but every one of them sat inside a zip archive (on Dryad or Zenodo), and in the corpus run none of these archive members was downloaded, so `code_check` had no local file to inspect. The run recorded no reason for the failed downloads. Rechecking every one of these papers live on 10 October 2026, with the same Metacheck version, Metacheck's own archive-member download retrieved a code file from each archive at exactly the size listed in the archive, so the code was downloadable and the failure

was specific to the corpus run rather than to the repositories; this is the same archive-member download pattern traced earlier in this project (scienceverse/metacheck#429). We therefore reran Metacheck's repository, data, and code checks for every paper with at least one archive member that had not been downloaded (191 papers), this time recording the outcome of every download and waiting out host rate limits instead of skipping the file. After this rerun the code of all 25 papers downloaded. The corpus run recorded no reason for the original failures, but host rate limits are the most plausible cause: in the rerun, the last paper whose code still failed did so because Dryad refused the requests as too many (HTTP 429), and its code downloaded once the limit was waited out. Of the 1 remaining `code_download` disagreement, 1 is `METACHECK_RIGHT`: real, verified downloaded code files (an OSF repository's 9 code files, each independently confirmed with a non-missing `file_location`) that Cooper et al.'s coding missed.

Limitation. `code_archived` remains the construct in this comparison with the most heterogeneous set of causes. The single largest cause (code co-located with data in a repository `code_check` cannot classify, whether because the file is an unopened archive or for another reason) points to the same file-classification limitations documented for the not-classified cluster in the review of missed repositories above; the second-largest (repository detection coverage) is the same gap documented for `data_availability`. `code_download`'s disagreements fell substantially after the fixes documented in Appendix A.1, and the archive-member downloads that had failed during the corpus run all succeeded once rerun with host rate limits waited out (see above), leaving 1 disagreement(s).

Is there a README?

Cooper et al. report a README present for "66% (n = 1114) and 61% (n = 351) of the papers with archived data or code, respectively" (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). As described in the Coding section, Metacheck records a single README signal per paper (computed once, across every linked repository combined), so we compare it against `cooper_any_readme` — `TRUE` if Cooper et al. coded "Yes" or "Quasi-README" on either their `data_README` or `code_README` question. `mc_any_readme` and `cooper_any_readme` were both codeable for 1726 papers. They agreed on 1247 (72.2%), disagreed on 479 (27.8%) — the largest raw disagreement count of any construct in this comparison, though, as with `code_archived`, its composition changes the interpretation substantially. Sensitivity was 63.2%, specificity 95.5%.

Of the 479 disagreements, 23 were `METACHECK_RIGHT` on review — genuine cases where a real README file exists in a repository `repo_check` listed, and Cooper et al.'s coders missed it. 150 were `COOPER_RIGHT`, dominated (113 cases) by the same repository-detection gap documented for `data_availability` and `code_archived` above — `repo_check` never located a repository the paper's own text explicitly names, so no README evidence could ever be recorded — with a further 2 cases tracing to a `repo_check` processing error. The remaining 12 were `UNCLEAR`.

The single largest verdict category, however, is 294 `DIFFERENT_DEFINITION` cases (61.4% of all disagreements, well ahead of the `COOPER_RIGHT` count above) — not a detection failure on either side, but the two coding approaches answering genuinely different questions that happen to share Cooper et al.'s column label:

- **Dryad's abstract field (the largest single cause, 172 cases).** Dryad's own dataset API exposes exactly one free-text field, `abstract`. Checked directly on real Dryad datasets from this disagreement set, this field is consistently the paper's own scientific abstract — never file-by-file documentation of what the archived files contain. Cooper et al.'s coders, reading the same field on the live Dryad page, appear to have coded this abstract as satisfying "README present," a defensible reading of their own protocol given what Dryad actually surfaces to a human visitor; `repo_check`'s `doc_role` classification, which requires an actual downloadable file named like a README, never counts it.
- **Zenodo's and Figshare's `description` fields (88 cases).** Unlike Dryad's abstract, these platforms' `description` fields are genuinely mixed in content — checked directly on records from this disagreement set, real file-by-file documentation appears in some cases, a study summary in others, contact metadata in others, and an empty field in others. No single, reliable rule (treat the description as a README, or don't) would correctly resolve every case.
- **An archive-only repository whose single compressed file `repo_check`'s remote listing could not resolve (16 cases).** Any README bundled inside such an archive is invisible to the README check even when `repo_check`'s own zip-listing feature is enabled, for the same family of archive-access limitations discussed above.
- **A smaller remainder (18 cases)** not matching any of the three patterns above.

Limitation, and a genuinely different framing than the other constructs in this comparison. Unlike every other column reported here, `any_readme`'s disagreement is overwhelmingly not a case of either side being wrong. It is a case where Cooper et al.'s human coders, reading a live repository page, correctly used whatever free-text field that platform's own interface surfaced to them as evidence of a README-like description, while Metacheck's `doc_role` classification asks a narrower, more conservative question (is there a downloadable file that is actually named like a README) that structurally cannot see a platform's own abstract or description metadata at all. Resolving this gap in Metacheck's favor is not simply a matter of relaxing a filename rule: Dryad's abstract field is confirmed never to be README-equivalent content (so counting it would introduce false positives), while Zenodo's and Figshare's description fields are confirmed to be genuinely mixed (so counting them uniformly would introduce a different, more Zenodo/Figshare-specific rate of false positives). A reliable fix would need to distinguish, likely with a content classifier rather than a presence check, which platform-level description fields actually contain file-documentation content, on a case-by-case rather than platform-wide basis. ## Is a comment count a proxy for comment quality?

Cooper et al.'s protocol includes a construct no Metacheck module attempts to answer directly: `code_annotation_scale`, a human coder's 1-10 judgment of how well a paper's code is commented, coded for 543 of the 1861 papers. Metacheck's `code_check` module does not judge

comment quality, but it does mechanically count comments: for every code file it can download and parse, it records `comment_lines`, `code_lines`, and `percentage_comment` (comment lines as a share of all lines). This raises a genuinely exploratory question distinct from every comparison above, none of which asked whether an automatable signal could stand in for a judged one: if a simple, free line-count already tracked Cooper et al.'s human quality rating closely, that would suggest the more expensive human judgment adds comparatively little beyond what counting could already tell us; if it does not, that would suggest the human coders are picking up on something a mechanical count cannot see.

We aggregated Metacheck's per-file comment counts to one value per paper — total comment lines divided by total lines across every code file `code_check` could download and parse for that paper — and compared it against Cooper et al.'s `code_annotation_scale` for the 381 papers where both were available (of 543 papers Cooper et al. rated and 438 papers Metacheck could compute a comment percentage for at all; the smaller overlap reflects that Metacheck's count requires at least one code file to have actually downloaded and parsed, which, as the sections above document, is itself not guaranteed).

The correlation is weak but not zero: Pearson's $r = 0.20$, 95% CI [0.10, 0.29], $p < .001$; Spearman's $\rho = 0.17$, $p < .001$. Using the raw comment line count instead of the percentage gives essentially the same result ($\rho = 0.14$, $p = 0.006$), so this is not an artifact of how the percentage was computed. As a check on whether longer codebases are simply commented differently regardless of quality, we also correlated total code lines against percentage of comments directly: longer codebases have a modestly lower comment percentage (ρ = -0.30, $p < .001$), consistent with comments not simply scaling with code length, and not something that would by itself explain the annotation-scale correlation being as weak as it is.

Interpretation. A correlation of this size — roughly 4% of variance explained on the Pearson estimate — means a paper's comment density is a poor stand-in for a human coder's quality judgment: knowing what fraction of a codebase's lines are comments tells us very little about whether a coder would call those comments good annotation. This is exploratory, not a claim about causation or about which measure is "right" — the two are answering different questions by design, and a low correlation between them is not evidence that either is wrong. But it does directly speak to the question motivating this comparison: for this specific construct, an automatable proxy does not substitute well for the human judgment call, at least not the simplest version of that proxy (line-count percentage). A single well-placed sentence explaining a complex analysis counts as excellent annotation on Cooper et al.'s scale and as a single line on Metacheck's; ten lines of boilerplate restating variable names count as heavily annotated by line count and would plausibly score poorly on a human quality read. Comment content — whether a comment explains a non-obvious decision, restates the obvious, or is simply absent — is the kind of thing an LLM-based classifier could more plausibly approximate than a line count can, but that would need its own validation against this same ground truth before being trusted, following the same case-by-case standard used throughout this manuscript rather than assumed from the fact that an LLM could be asked to try.

Results you get for free alongside Metacheck

Every comparison above answers a question Cooper et al.'s protocol already asked. Running `repo_check`, `data_check`, and `code_check` on a corpus also produces a substantial amount of additional, structured diagnostic information that no item in Cooper et al.'s protocol asks for at all — not because Metacheck was built to exceed their protocol, but because the same file listing, the same downloaded code, and the same parsed spreadsheets that answer "is there a repository" or "is there a README" also support other checks the modules perform as a matter of course. This section reports what that additional output looks like at corpus scale, computed entirely from the same saved module output as the comparison above (`res_repo_check.RData`, `res_data_check.RData`, `res_code_check.RData`) — no additional module run was needed.

Code portability and reproducibility signals

`code_check` parsed 9158 code files across 440 code-bearing papers. Three checks bear directly on whether that code would actually run for someone other than its original author. (A systematic validation of these checks against real corpus files, reported in Appendix A.3, identified and fixed five false-positive/false-negative mechanisms; the numbers below reflect the corpus as run after those fixes.)

- **Missing referenced files.** For every code file, `code_check` extracts every filename the code itself tries to open (`read.csv(...)`, `source(...)`, and equivalent calls in each supported language) and checks whether that name appears anywhere in the same paper's repository listing. 1207 files (13.2% of all analysed files), across 227 papers (51.6% of code-bearing papers), reference at least one file that could not be found. This does not necessarily mean the code is broken — a file might exist under a name the regex-based extraction did not match exactly, or might be generated by an earlier step in the same pipeline — but it is a direct, corpus-wide signal of how often "the code plus the shared data" does not obviously line up as a complete, runnable unit.
- **Hardcoded absolute paths.** 479 files (5.2%), across 116 papers (26.4%), contain an absolute file path (e.g. `C:/Users/...` or `/home/...`) rather than a path relative to the project. Code written this way will not run on any machine but the one it was written on without manual editing.
- **`setwd()` calls.** 451 files (4.9%), across 106 papers (24.1%), call `setwd()` to hardcode a working directory. Combined with hardcoded absolute paths (which frequently co-occur with a `setwd()` call to the same machine-specific location), the two portability problems together affect 275 of the 440 code-bearing papers (62.5%) — meaning more than half of the papers whose code we could inspect at all have at least one file that will not run as shared, on someone else's machine, without modification.

Comment density (mechanical count)

As reported above in the comment-quality correlation, `code_check` records `comment_lines/code_lines` per file. Corpus-wide, comments make up 11.0% of all code lines. 1231 files (13.9%) have zero comments at all — of which 303 of 412 zero-comment Python files (73.5%) are in fact documented via a docstring rather than a `#` comment, a distinction `code_check` now also reports separately (`has_docstring`, Appendix A.3), so are not genuinely undocumented despite scoring zero on this mechanical count.

Import/library organization

Where a file loads more than one package, `code_check` also records how far apart those loading lines are (its own documented threshold is more than a few lines apart counting as "scattered" rather than grouped at the top of the file). 1091 of 4613 such files (23.7%) load their packages in a scattered rather than grouped fashion — a minor readability signal, not a correctness one, but one a reader piecing together a script's dependencies would find genuinely useful.

Parse errors

Of the 5758 R-type code files (`.R`, `.Rmd`, `.qmd`) that were both locally available and actually reached a call to `parse()`, 241 failed to parse as valid code in their declared language at all (4.2%) — a strong, unambiguous signal (not a stylistic judgment) that something is wrong with either the file itself or Metacheck's identification of its language. (Only R-type files are eligible for this check at all; a file that was never downloaded, or that errored before reaching the parse step itself, is excluded from this denominator rather than counted as a pass — see Appendix A.2 for the full derivation and the two distinct causes behind the files excluded this way.)

Package version pinning

`code_check` also checks whether a repository pins the exact R/package versions its analysis depended on — an `renv.lock` file, a `sessionInfo()`/`session_info()` text dump, or a `groundhog`/`checkpoint` date-pinning call in the code itself (a bare `library(groundhog)` does not count; the pinning call itself must be present). Of the 440 code-bearing papers, 20 (4.5%) have version pinning present, recorded per paper as `code_version_pinned` in `code_check`'s own summary output (see Appendix A.2 for how this per-paper figure was recovered).

Spreadsheet and column-level data-quality findings

`data_check` inspects every tabular data file it can download and parse, both at the file/sheet level (is this a rectangular table at all, is the header on the first row, does the file use colour or merged cells to encode information Metacheck cannot read from a plain CSV) and at the level of individual columns (does a column look like it holds personal information, geographic coordinates, inconsistent category casing, numbers stored as text, or values in a legacy character encoding). Across the corpus, these checks produced 44140 findings, spanning 3372 distinct files. The most frequent, in descending order:

Finding	Count
Empty column	23,133
Numeric as text	5,956
Problematic column name	4,327
Colliding column names	2,124
Empty or unnamed columns	1,823
Personal info (column name)	1,739
Case issues	1,038
Geographic coordinates	903
Whitespace	726
Empty rows	634
Colour coding	559
Merged cells	273
Free-text (may hold PII)	265
Mixed encoding	230
Personal info (values)	125
Header not on first row	90
Un-inspectable (.xls)	89
Not a rectangular dataset	62
Unreadable	44

Two of these are worth calling out specifically because they bear on participant confidentiality rather than only on tidiness: the two personal-information checks (by column name and by

column values) and the "free text that may hold PII" check together flag 2129 findings across the corpus — columns or values that look, on their face, like they could contain names, contact details, or other identifying information in a file the authors have made publicly downloadable. Metacheck does not verify whether this is a genuine confidentiality problem (a column named "participant_id" that holds only anonymous sequential integers would also match on name alone), but a count this size suggests a possible participant-privacy check would find comparable value to the correctness-oriented deposit review Cooper et al.'s protocol already performs.

File-naming conventions

`repo_check` checks every listed file's name against a set of FAIR-data-adjacent naming conventions: no spaces or special characters, an actually-classifiable name (not left as `data_type == "unknown"`), consistent casing, zero-padded sibling numbering, valid dates, and several path-length budgets. Across the corpus, this produced 53281 naming issues, 28048 of them the more serious "bad" severity (something that actually breaks: spaces, special characters, diacritics, an unclassifiable file, an invalid date, or an over-long path) and 25233 the softer "suggestion" severity (a real convention worth following, but nothing in Metacheck's own classification depends on it). 730 distinct papers (46.9% of the 1555 papers with a discoverable repository) have at least one naming issue of some kind. The single largest contributor by far is "unclassifiable" file names (18674 occurrences) — files whose name gives Metacheck's classifier no signal at all about what kind of content they hold, which is itself a legibility problem for a human reader trying to navigate the repository, independent of anything Metacheck specifically failed to detect.

Why this section exists. None of the findings above were validated against Cooper et al.'s human coding, because Cooper et al.'s protocol did not ask about any of them — there is no ground truth to check against, and no sensitivity/specificity/agreement-rate analysis is possible for a construct nobody coded. We report them here, separately from the validated comparison above, specifically so that this distinction stays clear: these are Metacheck's own outputs, useful in their own right for a researcher curious about a specific repository's quality, but not independently verified the way every number earlier in this manuscript was. A natural next step, beyond the scope of this paper, would be a smaller, targeted validation of a sample of these specific findings (are the "unclassifiable" file names really unclassifiable to a human reader too; do the flagged personal-information columns really hold identifying information) using the same case-by-case standard applied throughout the rest of this manuscript.

Discussion

This comparison against ground truth coded by Cooper et al. provides valuable insights into how automated tools can be improved. First, PDF files are created to optimize human readability, not machine readability. Text extraction from PDF files has known limitations which lead to broken or undetected links to repositories. It might be possible to remedy

this by developing software that improved text extraction from PDF files (e.g., see CITE https://github.com/scienceverse/bibr/releases/tag/v0.6.0), but a fully machine-readable solution would require publishers to share the machine readable XML files used to generate manuscripts (as for example PLOSone already does), or to add metadata files alongside journal publications with important information (such as links to data and code). Second, automated tools should eventually support all FAIR repositories. Given the diversity of data and code storage solutions across disciplines, this will require collaboration between tool builders, research communities, and data repository managers. Third, automated tools should detect all file formats that store data and code, which once again given the diversity of research software used across disciplines requires collaboration between tool builders and research communities. Adding support for a repository is straightforward when it runs widely used software with a standard API, but our review shows that many institutional repositories need repository-specific development work (see the Metacheck limitation cluster above), and as more research communities adopt automated tools, users can provide feedback to improve automated data and code detection. The comparison against ground truth also highlights aspects where researchers, journals, and repositories can improve. Researchers should follow best practices in machine readable FAIR data and code sharing (CITE https://github.com/scienceverse/ms_FAIR_data_and_code), and journals should check whether FAIR practices are followed before publication. Automated tools such as Metacheck can assist in such checks, as the current manuscript demonstrates. Repositories should be FAIR, which includes accessibility by a machine. The Dryad repository is not accessible without an API key, and even then allows only a very limited number of API requests, which severely limits automated checks. Researchers are recommended to share data in fully machine accessible repositories, such as Zenodo, instead. Some repositories do not yet have an API, and these repositories should provide machine access to the data and code that they store.

Automated tools need to be able to sustainably support existing repositories. Registries of data repositories exist (e.g., https://www.re3data.org) but the registry does not contain information about the APIs used by each repository. Some repositories disappear, and new ones are added. Support is relatively straightforward if a repository uses an established API (e.g., Dataverse, Figshare, DSpace) but requires more effort if it uses a unique API (e.g., OSF). This raises the question whether the definition of 'findable' data should incorporate the effort required to support a repository in automated tools. It seems likely that data stored at more widely used repositories (e.g., Dryad, Dataverse) will be more findable by automated tools than a small custom data repository used by a handful of scientists, simply because of the costs associated with supporting all data repositories. Of course, data repositories can facilitate the findability of the data they host by contributing to open source software, and contribute code that supports their own platform.

Table 0 summarizes the empirical comparison reported above: agreement rate, sensitivity, specificity, and the two directional miss rates for every construct where Metacheck and Cooper et al.'s coding can be directly compared.

Table 4: Table 0. Empirical agreement between Metacheck and Cooper et al.'s human-coded ground truth, for every comparison column. "Cooper caught, MC missed" and "MC caught, Cooper missed" are percentages of the disagreements only (they sum to 100% within a row), not of the full sample.

Construct	n	Agree %	Sensitivity %	Specificity %	Cooper caught, MC missed %	MC caught, Cooper missed %
Data availability	1,735	85.2	86.3	44.4	90.3	9.7
Data licence present	1,314	95.6	99.1	25.4	19.0	81.0
Data downloadable and usable	1,328	99.0	99.2	0.0	84.6	15.4
Code archived	1,669	87.5	69.7	96.9	83.7	16.3
Code downloadable	395	99.7	100.0	0.0	0.0	100.0
Any README present (data or code, collapsed)	1,726	72.2	63.2	95.5	95.4	4.6
Data archive platform (exact match of platform sets)	1,393	90.1				
Code archive platform (exact match of platform sets)	401	77.3				

Two patterns are visible across every construct in Table 0. First, the size and direction of the sensitivity/specificity gap now varies considerably from construct to construct, and reviewing every individual disagreement, rather than stopping at the aggregate numbers, shows why: the asymmetry has a different cause in each column, and the cause matters more than the raw percentage for deciding what, if anything, to fix. `data_availability`'s specificity (44.4%) is far from the zero an earlier pass of this corpus found, precisely because the derivation-rule fix described in the Coding section gave it a genuine ability to disagree with Cooper et al. in the `FALSE` direction at all; what remains is a repository-coverage gap (see above), not a structural inability to ever produce a negative. `code_download` has 1 disagreement(s) left (sensitivity 100.0%, specificity 0.0%), after the code inside zip archives whose download had failed during the corpus run was downloaded in a rerun that waited out host rate limits; `data_download` has only 2 codeable Cooper-`No` cases, too few for its specificity (0.0%) to mean much. `code_archived` shows the most pronounced asymmetry of any boolean construct (sensitivity 69.7%, specificity 96.9%) and is genuinely multi-causal, spanning a real content-location mismatch (code bundled with data in a repository `code_check` cannot classify), a repository-detection gap, and a real "not in any repository at all" category (journal Supporting Information) no repository-scanning approach can close. `any_readme`'s asymmetry is not primarily attributable to either side being wrong at all: 61.4% of its disagreements reflect Cooper et al.'s human coders correctly reading

platform-level metadata (a Dryad abstract, a Zenodo or Figshare description) that Metacheck's stricter, filename-based README check was never designed to treat as equivalent evidence, and where doing so uniformly would, on the evidence collected here, introduce as many new false positives (Zenodo/Figshare) as it resolved.

Second, precision in the direction Metacheck does commit to an answer remains generally high, though no longer universal: positive predictive value was above 96% for five of the six boolean constructs (`data_availability` 98.3%, `data_license` 96.3%, `data_download` 99.8%, `code_download` 99.7%, `any_readme` 97.3%), meaning that when Metacheck reports a repository, license, download, or README as present, that report is rarely spurious. `code_archived` is the exception: its PPV, 92.2%, is meaningfully lower, because 31 of its 209 disagreements are cases where `code_check` correctly found real code files that Cooper et al.'s coders missed or judged not to count — a genuine human-coder miss, not a Metacheck false positive, but one that lowers this specific metric regardless of which side is right. The asymmetry in this comparison therefore remains overwhelmingly a recall problem for most constructs — Metacheck under-detects relative to a human coder who can read a repository's page, follow an ambiguous citation, or interpret an unusual sentence structure — but `code_archived` shows this is not universal: a `TRUE` from Metacheck is a highly trustworthy signal for five of the six boolean constructs, while for `code_archived` specifically, a nontrivial share of Metacheck's `FALSE` answers reflect a real detection gap rather than code genuinely being unarchived.

Three constructs in Cooper et al.'s protocol have no Metacheck counterpart at all and were therefore not part of the quantitative comparison above: README/code usefulness (a human-judged 1-10 scale), data completeness relative to a paper's reported analyses, and code annotation quality as distinct from comment density. These are precisely the questions where Cooper et al.'s human coders had to read and interpret content, not merely detect its presence, and they mirror a pattern visible even within the constructs we could compare: Metacheck's underlying modules answer structural questions (does a file with this name exist, did a download succeed, does a table parse) reliably, and answer content-interpretation questions (is this description a real README, is this the paper's actual dataset rather than simulation output, is this repository the one holding the data rather than the code) far less reliably, precisely because those questions require reading and judging content rather than classifying structure. Large language models are a plausible route to closing part of this gap for the constructs Metacheck cannot currently address at all, and could plausibly also help with several of the misclassification patterns documented above (distinguishing a Dryad abstract from file documentation, or a simulation-output file from raw collected data) — but any such extension would need to be built, validated against Cooper et al.'s own ground truth using the same case-by-case standard applied here, and reported with a known error rate, rather than assumed to work from the fact that an LLM could plausibly be prompted to attempt it.

A recurring, cross-cutting finding worth naming explicitly is the data-versus-code repository conflation pattern: a genuinely common paper structure — one repository holding the study's data, a separate repository (often GitHub) holding its analysis code, sometimes with the data-holding repository never located by `repo_check` at all — appears as the dominant or a

substantial cause of disagreement in `data_archive`, `data_license`, and `code_archived` alike. This is not three independent bugs; it is one architectural gap (Metacheck's repository-scoping logic assumes each construct maps cleanly onto "the" repository for a paper) surfacing across every construct that depends on correctly identifying which repository is relevant. It remains, after every fix applied over the course of this project, the single largest reason Metacheck and Cooper et al. disagree: repository non-detection alone accounts for 178 of `data_availability`'s 216 `COOPER_RIGHT` disagreements, 42 of `code_archived`'s 155, and 113 of `any_readme`'s 150 — a paper's own text names a real, working DOI on a mainstream platform (overwhelmingly Dryad, Zenodo, or Figshare), and `repo_check` never queries it at all. Addressing the conflation and non-detection gaps once, at the level of how Metacheck decides which repository "counts" for a given question and which platforms it recognises, would likely resolve a meaningful fraction of the remaining disagreement across several constructs simultaneously, rather than requiring separate, construct-specific fixes.

Twelve individual software defects identified over the course of this project were reported upstream, fixed, and reverified against a full corpus rerun for this manuscript. Five were found during the original review: the Excel multi-sheet `tabular_usable` limitation underlying most of the original `data_download` disagreement (scienceverse/metacheck#379, resolved: agreement rose from 93.5% to 99.1%), the Dryad unauthenticated-HEAD-request gap affecting both data and code downloads (scienceverse/metacheck#380), the `.zip`-peek authentication gap for archive-only repositories (scienceverse/metacheck#378), the `code_check` zip-discovery architectural gap surfaced while investigating the Zenodo data-download fix (scienceverse/metacheck#383), and a silent-truncation bug in the byte-fetch path that #383's fix newly exposed (scienceverse/metacheck#384): the function decompressing a zip member's bytes called an underlying library routine with no size hint, silently capping any member over 32,768 bytes at exactly that length with no error, rather than the size-aware call the library's own exact-size path rejects for an unrelated reason. Fixing #384 closed the great majority of the gap #383 had left open: `code_download` agreement rose from 66.2% (before #379/#380/#383) to 89.7% (after them) to 97.2% (after #384) — a two-stage improvement of over 30 percentage points overall, the largest of any construct in this comparison. After the later fixes and full corpus reruns described below, agreement is currently 99.7%, with 1 disagreement(s) left after the archive-member downloads that had failed during the corpus run were rerun with host rate limits waited out (see Empirical Comparison).

A further seven fixes (Appendix A.5) — Zenodo reliability, wider repository-platform coverage, and a rate-limit handling defect — were applied and reverified against the entire 1,861-paper corpus rather than only the papers already known to disagree, specifically to check whether a fix aimed at one construct could have broken an already-correct answer elsewhere. Six of the seven had not; the seventh — the R-package-source-tree exclusion, meant to drop a bundled dependency's own scaffolding (`DESCRIPTION`, `NAMESPACE`, `R/`, `man/`, `tests/`) from a repository's file listing — also stripped a paper's own analysis code whenever that code was itself packaged as an installable R package, a common practice for ecological modelling and analysis tools shared alongside a methods paper, and in several cases stripped the package's bundled `data/` directory as well, turning a real dataset into an apparent absence of one.

This was confirmed against 16 real GitHub/CRAN-hosted repositories (`RevGadgets`, `BaBA`, `m2b`, `RRPP`, `actel`, `scampr`, `MINOTAUR`, `LadderFuelsR`, `biomonitoR`, `EcoEnsemble`, `fossilsim`, `frair`, `tRophicPosition`, `sabinaNSDM`, `bef`, and `phylomice`). Rather than being left as an open limitation, the exclusion rule was corrected within this same review cycle: it now only strips a root-level package's files when nothing classifiable as real data or code survives outside it — using the same `data_classify_files()` rules-based classifier `data_check` itself relies on — distinguishing a study that shares its own data and code as an R package (nothing survives outside it, because the package is the deposit) from a repository that bundles a vendored copy of someone else's package alongside the study's own files (real content survives outside the vendored package, so only the vendored package's files are excluded). Reapplying the corrected rule resolved 11 of these 16 cases; the remaining 5 are due to a separate, still-open defect in the corrected rule itself: continuous-integration configuration files at the repository root (such as `.travis.yml`) are classified as code, so the rule concludes real code survives outside the package and keeps excluding the package's own source files (scienceverse/metacheck#469).

We see this kind of construct-by-construct, evidence-grounded mapping — quantifying not just how often Metacheck and a trained human coder disagree, but why, down to the individual paper and, where applicable, the specific line of code responsible, reverifying that mapping against a real fix rather than stopping at diagnosis, and checking a fix's effect on the papers it was not meant to change as carefully as its effect on the papers it was — as the more useful contribution here than a single aggregate accuracy figure, both for meta-scientists deciding whether Metacheck can currently support a given coding task, and for Metacheck's own development priorities. Appendix A documents, in full detail, every defect identified over the course of this project, the evidence that surfaced it, the exact code change that fixed it, and the before/after numbers at each stage; the body of this manuscript reports only the current, post-fix numbers.

Detailed record of every Metacheck defect found and fixed during this project

This appendix documents, in full detail and in chronological order, every software defect in Metacheck that this project's validation effort identified, the evidence that surfaced each one, the exact code change that fixed it, and the corpus-level numbers before and after that fix. The body of the manuscript reports only the final, current numbers; this appendix exists so that every intermediate claim made in the course of arriving at those numbers remains checkable, without requiring the main text to carry a running commentary on its own revision history. Seven rounds of validation are documented below: (A.1) the original empirical-comparison review against Cooper et al.'s coding, which produced issues #378–#384; (A.2) a focused re-examination of `code_check`'s parse-error and package-version-pinning checks; (A.3) a systematic validation of five further `code_check` checks (missing referenced files, hardcoded absolute paths, `setwd()` calls, comment density, and library/import scattering) against real corpus files, which produced five further fixes; (A.4) a file-count gate added after one outlier repository's processing time exposed a gap the existing byte-size caps did not cover; (A.5) a full re-check of every remaining disagreement against Metacheck after five more fixes (Zenodo reliability, repository-platform coverage, and a rate-limit reliability defect found live during the re-check itself); (A.6) a systemic under-sampling bug found in A.5's own Dataverse DOI-prefix allowlist, two further platform-coverage fixes (DataONE, institutional Figshare instances) and two matching bugs in this project's own comparison code, and a complete, evidence-based categorization of every "repository not detected" disagreement these fixes did not resolve; and (A.7) a full-corpus run of Metacheck's `reproducibility_check` module — a different question from A.1–A.6 (does a paper's own shared code actually run and reproduce its own reported results, not merely whether a repository was detected) — which surfaced five further defects specific to running many papers' Docker containers concurrently at corpus scale and to extracting a paper's own declared dependencies from its code. Fixes described in A.1–A.4 were committed to `scienceverse/metacheck`, branch `repocheck_improvements` (pull request #385); A.5 and A.6's fixes were committed directly to that package's `dev` branch (A.6 as pull requests #410, #412, and #414); A.7's five defects were filed as issues #417–#421, two of which (#419, #420) are confirmed fixed on `dev` as of this writing. Every fix from A.1–A.6 is covered by a new or updated automated regression test in the package's own test suite; A.7's two confirmed-fixed defects were fixed and committed by the package's maintainer rather than by this project directly, so this manuscript does not independently confirm test coverage for them.

A.1 Defects found during the original empirical comparison

Six defects were identified while individually reviewing every disagreement between Metacheck and Cooper et al.'s human coding (Empirical Comparison, above): five in Metacheck itself, and one in this project's own comparison code. All six are described here with the same level of

detail as in the body text; they are repeated in this appendix only for completeness, so that Appendix A is a single, complete record of every fix made during this project.

#378 — `zip_peek()`'s unauthenticated requests. `zip_peek()` (`R/zip-peek.R`) reads a remote zip archive's central directory via an HTTP Range request, without downloading the archive, so a downloader can decide whether the archive is worth fetching before spending the bandwidth. This function's HEAD and Range requests were unauthenticated. Dryad's file-download endpoint answers 401 Unauthorized to any request that touches file bytes, including a bare HTTP Range request — confirmed live against a real Dryad file-download URL — so every Dryad-hosted archive's peek failed structurally, before any question of the archive's actual contents. Fix: both requests now route through `.auth_for_url()`, the same host-based authentication dispatcher every other Metacheck download path already uses, so Dryad's token is attached automatically without requiring each call site to opt in individually (an earlier, per-call-site `req_func` opt-in mechanism had only been adopted by 1 of 7 call sites that needed it).

#379 — `data_check`'s multi-sheet Excel limitation. `data_check`'s tabular-usability check called `readxl::read_excel()` with no `sheet` argument, so it only ever read a workbook's first sheet. When the first sheet was a cover, metadata, or "read me" sheet — a convention confirmed across many real corpus files, with sheet names including "metadata," "Meta-data," "MetaData," "read me," "ReadMe," "User guide," "LEGEND," and "Description of variables" — a later sheet holding genuine, well-structured tabular data was never inspected, and the file was misclassified as unusable ("100% free text, not variables"). One confirmed case: sheet 2 of a real downloaded workbook held 1,532 rows by 58 real variable columns, never read before the fix. Fix: `data_read_head()` now accepts a `sheet` argument, and `data_check` tries each remaining sheet in a workbook before giving up on the file. This was the single largest driver of `data_download` disagreement: before the fix, `mc_data_download` and Cooper et al.'s `data_download` disagreed on 51 of 967 comparable papers (94.7% agreement); after it, agreement rose to 99.1% (908/916 papers, 8 disagreements remaining), and the resolved cases accounted for 29 of the original 51 disagreements (57%).

#380 — Dryad's authentication gap on other download paths. The same unauthenticated-request problem identified for `zip_peek()` (#378) also affected `.remote_size()` and `.remote_content_length()`, two other functions in the download path that check a remote file's size via HTTP HEAD before deciding whether and how to download it. `.remote_content_length()` already had a `req_func` opt-in mechanism for per-host authentication, but only 1 of 7 call sites across the codebase had actually adopted it, so the other 6 still received a 401 from Dryad. Fix: both functions now route through `.auth_for_url()` unconditionally, the same fix applied to #378, closing every remaining unauthenticated-request gap in the download path at once rather than patching call sites one at a time.

#383 — `code_check` could not see code bundled inside a zip archive. This was a compound architectural gap with three separate causes, discovered while investigating why `code_check` found no code files at all for several papers whose repository, on manual

inspection, clearly contained real analysis scripts bundled inside a `.zip`: (1) `code_check()` never read `data_check`'s already-expanded file structure, so any code file `data_check` had already discovered inside a zip (in the course of looking for data files) was invisible to `code_check` regardless; (2) `code_check` had no zip-peek capability of its own for the case where `data_check` had not already run first; (3) the shared zip-expansion helper `.archive_rows()` discarded code-typed files outright even when it did expand a zip, keeping only `"data"` and `"documentation"`-typed files. Fix: `code_check()` now prefers `data_check`'s structure when available; it gets its own zip-peek fallback (`.code_expand_zip()`) for when `data_check` did not run first; and `.archive_rows()` now keeps `"code"` alongside `"data"`/`"documentation"`. A real, concrete example verified live: a Zenodo record (record 3694205, repository `SharpRT/CMV_Spread`) bundled 4 real MATLAB `.m` files inside a zip that were entirely invisible to `code_check` before this fix. This fix, together with the archive-member content-fetch wiring described immediately below, moved `code_download` agreement from 66.2% (355 comparable papers, 120 disagreements) to 89.7% (358 comparable papers, 37 disagreements) — the first of two stages in this construct's improvement (the second is #384, below).

As part of the same round of work, `repo_check`'s zip-peek expansion was extended to carry not just a discovered code file's name (already possible after the fixes above) but its content: each expanded row now also carries `archive_url` (the parent zip's own URL) and `archive_member` (the file's exact name inside it), and `download_repo_files()` (`R/repo-download.R`) uses those two fields to fetch that one member's bytes directly, via a byte-range request against the parent archive, without downloading the archive's other contents. This mechanism was verified working in every isolated unit test at the time, but a full end-to-end pipeline run against a real paper still returned no file content for reasons not yet diagnosed at that point — this was filed and tracked as issue #384, described next, rather than merged as "fixed" while a known gap remained open.

#384 — Two distinct causes behind the archive-member byte-fetch failing intermittently. Issue #384 was filed as an open question ("archive-member content fetch works in isolation but returned NA once in a full pipeline run") rather than a diagnosed bug, and its investigation spanned two separate validation efforts and uncovered two independent causes.

Cause 1: silent truncation over 32,768 bytes. `.zip_inflate_member()` (`R/zip-peek.R`) decompresses one zip member's raw bytes by calling `zip::inflate()`. Passing the member's exact known uncompressed size to that function raised an unrelated "Failed to inflate data" error (a quirk internal to the `zip` package), so the original code left the `size` argument unset instead. `zip::inflate()`'s own documentation states that leaving `size` unset triggers automatic buffer resizing as needed — but, confirmed live against a real Zenodo-hosted zip member, this documented behaviour does not actually happen in practice: any member larger than the default 32,768-byte output buffer came back silently truncated at exactly that length, with no error, no warning, and a `bytes_written` field the calling code never inspected. A concrete confirmed case: a real 44,669-byte code file came back as exactly 32,768 bytes. `.zip_member_fetch()`'s own subsequent length check (comparing the decompressed length against the size recorded in the archive's own central directory) correctly caught this mismatch and discarded the truncated

bytes — but the practical, user-visible effect was that every code or data file over 32KB inside a zip silently failed to fetch, indistinguishable from a genuine network or host-compatibility failure. Fix: passing `size + 1` (one byte over the true size) avoids both failure modes — it is no longer the exact size that triggers the package's internal error, and it is large enough that the real output always fits in the up-front buffer rather than depending on the documented-but-non-functional resize path. Verified against both a real archive member under 32KB and one over it, and covered by a new offline regression test using a locally constructed zip file whose one member compresses to roughly 95KB uncompressed. A targeted rerun of the 36 papers affected by this pattern found 27 moved from zero downloadable code files to full or near-full recovery (23 fully, 4 partially), which is what raised `code_download` agreement from 89.7% (37 disagreements) to 97.2% (10 disagreements remaining) — the second of the two stages in this construct's improvement, and the largest single agreement-rate improvement of any construct in the empirical comparison.

Cause 2: an intermittent, silently-swallowed failure not explained by the truncation bug alone. Independently of the truncation bug, a second cause was found while investigating why 40 of the archives affected by issue #384 failed entirely (every member missing) despite containing no member anywhere near the 32,768-byte threshold. Tracing the actual corpus data (`res_code_check.RData` and its 38 per-batch constituent files, in `cooper_validation_metacheck/manuscript/data/`) showed that a single pipeline run calls `zip_peek()` on the exact same archive URL twice — once inside `repo_check` to discover a zip's member names, and again inside `download_repo_files()`'s archive-member-fetch step (added as part of #383/#384's own fix) to retrieve one member's bytes — with no throttling and, critically, no retry logic at all on either call: `zip_peek()`'s own HEAD request and its two Range-request helper functions (`.http_range_tail()`, `.http_range_bytes()`) called `httr2::req_perform()` directly, with no `req_retry()`, and treated any non-206/200 HTTP status (including a 429 burst-rate-limit refusal) as an unconditional, silent `NULL` — indistinguishable, by design, from "this host does not support range requests at all." Two back-to-back, unthrottled requests to the same archive URL within one process is exactly the pattern most likely to trip a host's burst-rate limiter, and the second call (the byte-fetch, inside `download_repo_files()`) was the one exposed to it. Fix, applied in two parts: (1) all three of `zip_peek()`'s network calls now retry through the same `is_transient`/`backoff`/rate-limit-wait helper functions (`.storage_is_transient_factory()`, `.storage_backoff()`, `.storage_retry_after_factory()`) that every other Metacheck download path already uses, so a 429 is now retried with backoff rather than immediately and silently treated as "unsupported"; (2) a new same-session, in-memory cache (`.zip_peek_cache`, keyed by URL) means the second call to `zip_peek()` on an archive already peeked earlier in the same run reuses the first call's result instead of repeating the network request at all, removing the redundant call — and the rate-limit exposure it caused — entirely rather than merely tolerating it. Verified by a new regression test exercising the cache directly (`test-zip-peek.R`) and by the full package test suite (463 tests passing, 3 pre-existing unrelated skips, 0 failures) after both changes.

Verdict corrections found by the post-rerun double-check pass. After the full corpus

rerun incorporating #378–#384, every disagreement was independently double-checked a second time (see Empirical Comparison, above, for the check's method). This pass corrected 7 verdict-labeling errors from the original review: 2 in `data_availability` (one moved to `COOPER_RIGHT` after finding the "data" files actually belonged to a cited third-party software dependency's repository, not the paper's own; one moved to `METACHECK_RIGHT` after finding a methods paper's repository, originally read as containing only package example files, in fact holds real trait data and model output), 1 in `data_download` (moved to `COOPER_RIGHT` after confirming a Zenodo-hosted "Functional Trait Database" file had, in fact, already downloaded and still failed the tabular-usability check, rather than remaining `UNCLEAR` on the assumption it had not yet downloaded), and 4 in `code_download` (three rows originally recorded `METACHECK_RIGHT` whose own written reasoning already concluded the opposite, corrected to `COOPER_RIGHT`; one row originally left `UNCLEAR` where the code files were, on direct inspection, confirmed genuinely downloaded, corrected to `METACHECK_RIGHT`). The pass also found one substantial scope correction rather than a single-row relabeling: `code_download`'s dominant post-#383 disagreement pattern was originally tagged in only 13 of its 37 disagreements at the time of the initial qualitative review; re-deriving the pattern systematically against every disagreement's saved `file_location` value found the identical signature (a discovered code file with no resolvable download location) in 35 of the 37, which is what led directly to tracing and fixing #384's second cause, above. Every one of `code_archived`'s 164 disagreements was also individually re-derived against the current `repo_check`/`code_check` file listing during this pass, including the 8 originally resolved via an automated keyword search rather than an individual hand read; this found zero verdict corrections in that column.

A sixth issue, not in Metacheck but in this project's own comparison code. `code_data_preparation/03_recreate_cooper_columns.R` derives `mc_data_availability`, `mc_code_archived`, `mc_data_README`, and `mc_code_README` for a paper from `repo_check`'s per-file tables (`res_repo_check$table`, `res_data_check$structure`, `res_code_check$table`). A paper for which `repo_check` found zero files anywhere has no row at all in any of those three tables, even though `repo_check`'s own per-paper `summary_table` — which covers every one of the 1861 papers — correctly records `files_n = 0` for it. Because the script built its paper list from the three per-file tables rather than from `summary_table`, every such paper was silently absent from the comparison entirely: its `mc_*` value was not `FALSE`, it simply never existed as a row to compare. Confirmed directly: 495 of 1861 papers were missing this way, including 404 papers Cooper et al. coded `data_availability Yes` — 404 real disagreements (cases where Cooper et al.'s coders found a repository Metacheck's own detection had missed) that were never entering the comparison at all, rather than being counted as a genuine miss. Fix: the paper list is now built from `summary_table$paper_id` (already the full 1861 by construction), and `mc_data_availability`, `mc_code_archived`, `mc_data_README`, and `mc_code_README` are explicitly set to `FALSE` for a paper `repo_check` examined and found nothing in; every other `mc_*` column (license, DOI, download, format, language) is correctly left `NA` for such a paper, since those questions remain genuinely inapplicable without a repository to ask about. Every number reported in the Empirical Comparison section above reflects this correction.

A.2 Re-examining `code_check`'s parse-error and package-version-pinning numbers

A separate line of inquiry, prompted directly by a sentence in an earlier manuscript draft stating that "138 files (1.3%) failed to parse as valid code in their declared language at all," asked two questions: is this the right denominator, and is the file list behind it available for inspection at all.

Locating the underlying data. `code_check()`'s per-run output already carries everything needed: a `table` with one row per code file (`file_name`, `language`, `parse_error`, `parse_error_msg`) and a `summary_table` with one row per paper (`code_version_pinned`, `code_parse_errors`), both already computed by the module — no new code was needed to make this data available, only to locate and aggregate the 38 saved per-batch corpus-run files (`cooper_validation_metacheck/manuscript/data/batches/res_code_check_batch001.RData` through `batch038.RData`) that had already been produced by the original corpus run.

Package version pinning: 34.2% was a batch-level upper bound, not a per-paper rate. An earlier manuscript draft reported that "only 13 of 38 batches (34.2%) contain at least one paper with any form of version pinning," explicitly flagged in that draft's own text as an upper bound on the true per-paper rate because a batch of ~50 papers needs only one pinned paper to count. Checking directly against the saved per-batch `summary_table$code_version_pinned` column (present in every batch, keyed by `paper_id`) confirmed the per-paper detail this draft believed was not retained had, in fact, been retained all along: aggregating it across all 38 batches gives 18 of 400 code-bearing papers (4.5%) with version pinning present — substantially lower than the 34.2% upper bound, as expected, and now a true per-paper number rather than an aggregate-of-aggregates. No code fix was needed here; this was a reporting correction (using data that already existed but had not yet been extracted at the right level of granularity), not a software defect.

Parse errors: 1.3% used the wrong denominator, and undercounted for a subtler reason. The originally reported 1.3% divided 138 parse failures by 10,461, the total code-file count across every language in the corpus — but parsing is only ever attempted on R-type files (`.R`, `.Rmd`, `.qmd`; `code_parse_r()` is R-only), so the other 3,259 SAS/SPSS/Stata/Python/MATLAB files in the corpus were never eligible to fail this check and should not have been in the denominator at all. Restricting to the corpus's 7,202 R-type files, a further complication emerged: 1,651 of those were never actually downloaded or read at all (an inaccessible repository or a failed download), so parsing was never attempted on them either — they, too, do not belong in the denominator. A further 15 files reached a local copy but errored during extraction (before `parse()` itself was ever called), splitting into two genuinely different causes on closer inspection: 11 were confirmed to be genuinely empty (0-byte) files — fetched directly from their original GitHub repositories to confirm the emptiness is a fact about the authors' own repository, not an artefact of Metacheck's download step (e.g. `github.com/olgaviedma/LadderFuelsR`'s `R/foofy.R` and `github.com/dpritchard/frair`'s `tests/testthat/test_emdII.R`, both independently confirmed 0 bytes via a direct `curl` fetch from `raw.githubusercontent.com`) — and 3 were `.Rmd` files that failed during Quarto/knitr's own chunk-extraction step with

a genuine, structural fence-mismatch error (a closing code-fence with a different number of backticks or indentation than its opening fence), which is the same underlying "not valid in its declared format" problem a parse error represents, just caught one step earlier in the pipeline than `parse()` itself. Restricting to the 5,536 files that were both locally available and actually reached a call to `parse()`, 138 failed (2.5%) — not 1.3%, and not the 1.92% that a denominator of "all 7,202 R-type files regardless of whether they were ever read" would give either. Including the 3 fence-mismatch failures as the same kind of problem gives 141 of 5,539 (2.5%).

This investigation also surfaced a genuine, previously unknown software defect: **`code_read()` (`R/code_check.R`) crashes on a genuinely empty file.** `readr::guess_encoding()` returns a zero-row result for a 0-byte file, so `enc <- readr::guess_encoding(file_path)$encoding[[1]]` silently produces `NA` rather than a valid encoding string, and that `NA` then crashes deep inside `readr`'s own internals (`if (stringi::stri_enc_isascii(lines))`, which requires a length-one logical and receives `NA` instead) with the unhelpful, generic message `"missing value where TRUE/FALSE needed"` — this is exactly the message recorded for all 11 of the empty-file cases above, rather than a clear "this file is empty" signal. This defect is documented here as confirmed and diagnosed, but **not yet fixed** as of this writing (it affects a small, already-quantified fraction of files — 11 of 7,202 R-type files in this corpus — and a fix has not yet been prioritized ahead of the five `code_check` checks fixed in A.3 below); it is recorded here so the fix, when made, has a clear before-state to compare against.

A.3 Systematic validation of five further `code_check` checks

A third, more systematic validation exercise checked five of `code_check`'s remaining automated checks — missing referenced files, hardcoded absolute paths, `setwd()` calls, comment density, and scattered library/import lines — against real files from the same corpus, reading every flagged file that had already been downloaded locally (8,218 of 10,461 code-file rows) and sampling up to 100 of the remainder that would have required a fresh download. The full method, every sampled file's judgment, and every quoted example are recorded in a dedicated, reproducible validation report (`code_check_validation.qmd`, with its accompanying script `validate_code_check.R` and judgment tables, all in `cooper_validation_metacheck/manuscript/`) with a fixed random seed (`set.seed(42)`) at every sampling step; this appendix summarizes its findings and the resulting code fixes. Two checks not covered by this validation pass (parse errors, package version pinning) are covered separately in A.2 above.

Missing referenced files — case-sensitivity bug (fixed). `code_check`'s check for files a script tries to open but that are absent from the repository (`inst/modules/code_check.R`, `missing_files <- setdiff(base_ref, base_repo)`) compared filenames case-sensitively. 13 confirmed false positives across the sampled files were all the same mechanism: the referenced file exists in the repository under a different letter case, which is not a real error on the author's part on the case-insensitive filesystems (Windows, macOS by default) most authors develop on. Concrete examples, each confirmed directly against the paper's real repository file listing: paper

10_1002_2688_8319_12229 references REM_tools.r where the repository holds REM_tools.R; paper 10_1111_2041_210x_13923 references diagnostic_fcns.r/boot_glmm.r where the repository holds diagnostic_fcns.R/boot_glmm.R; paper 10_1111_2041_210x_13125 references FoodWeb_Functions.R where the repository holds Foodweb_functions.R; paper 10_1002_pan3_10736 references two scripts whose repository copies differ only in the first letter's case. Fix: the comparison now lowercases both sides before comparing (`missing_files <- base_ref[!tolower(base_ref) %in% tolower(base_repo)]`), removing all 13 confirmed false positives found in this sample and any of the same kind not yet sampled.

Missing referenced files — `vroom::vroom()` not recognised as a file-reading call (fixed). The same check's file-reference extractor (`code_file_refs()`, `R/code_check.R`) recognises a fixed list of R reader-function names (`read.*`/`read_*`, `import`, `fread`, `readRDS`, `load`, `readLines`, `fromJSON`, `readtext`, `source`); `vroom::vroom()` does not begin with "read" and was absent from this list, so a file loaded exclusively via `vroom::vroom()` was invisible to the missing-file check regardless of whether it actually existed in the repository. Confirmed false negative: paper 10_1111_2041_210x_13440's `R/ncbi.R` calls `vroom::vroom(file.path(dir, "nodes.dmp"), ...)` and a second `vroom::vroom()` call for `names.dmp`; neither file exists in the repository, but the check never flagged either, since `vroom` was not on its keyword list. Fix: `"vroom"` added to the reader-keyword list.

Missing referenced files — remaining known limitations, not yet fixed. Two further false-negative mechanisms were confirmed but not fixed, since each would require a more substantial change than a keyword addition or a comparison tweak: (1) `code_file_refs()` only captures a quoted filename appearing on the same line as a reader-call keyword, so a file assigned to a variable on one line and read via that variable on a later line (`liz_file <- "example_lizard_data.csv"`, then later `read.csv(liz_file)`) is invisible to the check even when the file genuinely does not exist (confirmed: paper 10_1111_1365_2435_13993's `tests/testthat/test-m_run_biophysical.R` does exactly this for three files, none of which exist in the repository). (2) When a referenced filename is built as `paste0(some_variable, "...literal_suffix.csv")`, the check's regex only captures the literal trailing fragment (e.g. `_M_D_abundance_interactions.csv`) rather than the true, variable-interpolated filename — the resulting "missing file" flag is still technically correct (nothing by that literal fragment name exists either), but the name shown to a paper's author is a confusing partial match rather than the file they would recognise; across the corpus, 128 of 2,599 (4.9%) genuinely-missing entries have a name starting with punctuation, the signature of this dropped-prefix pattern.

Hardcoded absolute paths — three false-positive mechanisms fixed, one left open. `code_abs_path()` (`R/code_check.R`) regex-matches quoted strings shaped like an absolute filesystem path. Three confirmed false-positive mechanisms were fixed: (1) a single-segment string beginning with `/` (e.g. `self.fetch('/login')` in a vendored web-app test suite, `github.com_ccrisan_motioneye/tests/test_handlers/test_login.py`) is an HTTP route, not a filesystem path — 322 of 4,182 originally matched strings were 6 characters or fewer, a class dominated by exactly this pattern; the regex's bare-`/`-prefixed branch now requires a second `/`-separated segment before matching, which a real absolute

path (e.g. `/Users/name`) always has and a bare route never does. (2) An all-digit UNC "host" is a `sub()`/`gsub()` regex backreference string, not a network path — confirmed twice in real code: `osf.io_5fter/IDcallR_sameN_withIDcallPP.R` line 183, `sub('group (\d+)_(\d+)', "\2\1", colnames(clupo))`, where `"\2\1"` was matched as `\\host\share` because a bare digit satisfied the old hostname character class; and a second, independent instance in a Pyrenees-mapping script's `gsub('(.{1,10})(\s|$)', '\1\n', ...)` call, matched for the same reason. The UNC branch's hostname character class now requires at least one letter, which excludes both confirmed cases without excluding any real hostname (no genuine UNC host is all-digit-and-punctuation with no letter at all). (3) The pattern's own pre-existing docstring already flagged a known risk — a variable-built path like `paste0(dir, '/file.csv')` triggering a false match on the literal fragment — and this was confirmed for real: `github.com_YvesBas_Tadarida-C/Sonotypes/write_tabase3HF_Sonotype.R` line 40, `list.files(paste(ListDate[[i]],"/eti",sep=""), ...)`, where `"/eti"` is a relative fragment concatenated onto a loop variable. This third mechanism is addressed by the same "require a second path segment" fix as (1), since `/eti` is itself single-segment. A fourth mechanism, confirmed but **not fixed**: a Python string built by `+`-concatenation from a relative literal (`'data\\' + tile_name + '\\run\\' + model_name + ...`, from a Zenodo-hosted `model_test.py`) is a relative path whose middle segments (`\run\`, `\output\tile\`) still match the UNC branch on their own; no fully static regex-only fix exists for this case, since it would require tracking whether the enclosing expression is a string concatenation starting with a relative literal, which is beyond what a line-based regex can determine. Of the true positives sampled for comparison, 15 of 15 further-checked "real username-shaped" matches (e.g. `C:/Users/reddin/Documents/...`, `/Users/farrer/Dropbox/...`, `/home/michael/Dropbox/...`) were confirmed genuine, machine-specific absolute paths — the large majority of this check's flags are correct.

`setwd()` calls — no false positives found in this corpus; one confirmed-but-unobserved risk noted. All 194 distinct `setwd()` call strings found across the sampled files are genuine, live, executable calls, including calls whose argument is itself relative or dynamic (`setwd("./data")`, `setwd(dir)`, `setwd(tempdir())`) — these correctly count as flags under this check's own stated rule, since the anti-pattern being flagged is changing the working directory at all, not specifically a hardcoded absolute-path argument. A latent false-positive mechanism was confirmed by construction (a synthetic test, not a real corpus case): `code_setwd()` has no awareness of string-literal context, so a line like `message("Please run setwd(your_path) before continuing")` would be matched as if it were a live call. Scanning all 194 real call strings in the corpus for instructional-text markers ("please," "before," "instructions," "note:") found zero real occurrences of this pattern — a real, demonstrated risk with no confirmed instance in this corpus, and left unfixed for that reason, though `code_setwd()` could straightforwardly reuse the same quote-context-tracking logic already implemented for `.code_strip_inline_comment()` if a future corpus surfaces a real case.

Comment density — a Python-specific docstring signal added (not a bug fix). `code_remove_comments()`'s Python branch deliberately never treats a triple-quoted string as a comment, since doing so would also destroy real docstrings (themselves ordinary string

literals, syntactically). The consequence, quantified directly: of 280 zero-comment-flagged Python files in the corpus, 200 (71.4%) contain at least one complete triple-quoted docstring block — real documentation that the mechanical `#`-only comment count cannot see, and so reports as "0% comments" regardless. Five were read directly to confirm these are genuine module/function docstrings rather than incidental triple-quoted data, e.g. `model_runs.py`'s `"""Filters for Model Runs."""` and `features.py`'s 16 separate triple-quote markers (8 docstrings) opening with `"""API functions to interact with feature names."""`. This is a deliberate, correctly-reasoned design choice in `code_remove_comments()`, not a bug — stripping docstrings to count them as comments would also delete them from every downstream check that reads code text — so the fix made is additive rather than corrective: a new function, `.code_has_docstring()`, detects whether a file contains at least one complete triple-quoted block, and `code_line_stats()` now returns this as a separate `has_docstring` field (`NA` for every non-Python language) alongside the unchanged `percent_comments` metric. `inst/modules/code_check.R`'s report text now adds an explanatory note whenever a zero-comment Python file in fact has a docstring, so a reader is told the file is documented via docstrings rather than concluding it has no documentation at all.

Scattered library/import lines — Python import-anchor tightened (fixed). `code_library_lines()` (`R/code_check.R`) previously anchored its Python import-detection regex on the bare keyword plus a following word (`^\s*(import|from)\s+[A-Za-z_.]`), with no requirement that the rest of the line actually form a valid import statement. Because Python's comment-removal never strips triple-quoted strings (the same mechanism behind the comment-density finding immediately above), docstring prose that happens to start with "from" or "import" survives into this scan and was matched as if it were a real import line. Concrete, exact example: `github.com_openvinotoolkit_cvat/site/process_sdk_docs.py`, line 213, inside a docstring spanning lines 211–214 — the prose sentence " from the generated model and api descriptions." was matched as an import line at line 154 of the comment-stripped text, while the file's 10 real imports are all genuinely contiguous at lines 1–10; the spurious match alone widened the reported gap between imports to 144 lines, crossing the "scattered" threshold (more than 3 non-comment lines between library statements) on a file whose real imports are, in fact, perfectly grouped. Quantified across all 525 Python files flagged as scattered in the corpus: 155 (29.5%) have at least one spurious, non-import match of this kind; of those, 68 (13.0% of all 525 flagged files) would no longer be flagged as scattered at all once the spurious matches are excluded — pure false positives caused entirely by this mechanism (the remaining 87 files with a spurious match are still genuinely scattered even after excluding it). R showed essentially no equivalent problem in a 60-file sample (the one apparent candidate, on manual inspection, was a genuine conditional `require()` call, not a false positive — R's regex requires an opening parenthesis directly after the keyword, structurally harder to spuriously trigger than Python's bare whitespace-plus-word anchor). Fix: the Python pattern now requires the full import shape rather than just the leading keyword — a `from` line must also contain its own `import` keyword later on the same line (`^\s*from\s+[A-Za-z_][A-Za-z0-9_.]*\s+import\s`), and a bare `import` line must be followed by a comma-separated identifier list with an optional `as name` and nothing else (`^\s*import\s+[A-Za-z_][A-Za-z0-9_.]*(\s*,\s*[A-Za-z_][A-`

Za-z0-9_.]*)*(\s+as\s+\w+)?\s*$) — verified against the docstring false-positive example above (now correctly unmatched) and against every real import shape found in the corpus sample, including multi-line `from x import (a, b, c)` continuations (matched on their opening line) and indented imports, with no loss of any real detection.

Status of the fixes in this appendix. All Metacheck fixes described in A.1 and A.3 (nine in total: #378, #379, #380, #383, both causes of #384, and the five `code_check` fixes from the systematic validation) have been committed to `scienceverse/metacheck`'s `repocheck_improvements` branch (pull request #385) and are covered by the package's automated test suite (463 tests passing at time of writing, 0 failures, 3 pre-existing skips unrelated to these changes). The sixth fix described above (the comparison script's missing-repository default) is not a Metacheck fix and has been committed instead to this project's own `code_data_preparation/03_recreate_cooper_columns.R`, in the `cooper_validation_metacheck` repository. The one defect identified but not yet fixed (`code_read()`'s crash on an empty file, A.2) and the two `code_file_refs()` limitations noted but not yet fixed (A.3) remain open. The further full corpus rerun incorporating the five A.3 fixes, planned when this appendix was first written, has since been performed (confirmed directly: the current corpus run's `code_check` output carries the `has_docstring` field A.3's docstring fix introduces, which cannot exist in a pre-fix run) — the numbers reported in "Results you get for free alongside Metacheck" above reflect that post-fix corpus, not the pre-fix state described earlier in this sentence's original drafting. This was a distinct item from the rerun described in Appendix A.5 below, which re-checked only the papers with a remaining disagreement (not the full corpus) and tested a different set of fixes (Zenodo reliability, repository-platform coverage, and rate-limit handling), not the five A.3 `code_check` fixes.

A.4 A repository excluded from processing by file count

During the corpus rerun incorporating the A.3 fixes, one paper's data repository — Zenodo record 10.5281/zenodo.3971689, SAPFLUXNET (a global database of sap flow measurements: 202 datasets from 121 locations, sap flow and environmental time series for 2,714 individual plants) — was found to hold approximately 4,000 individual files. Downloading and, in particular, `data_check`'s per-file classification and full-content read of every one of those files (`data_check_head(..., n_rows = Inf)`, by design, since several of `data_check`'s validation checks need the complete dataset rather than a sample) took several hours and multiple gigabytes of memory for this single paper, out of 1,861 in the corpus, before the run was stopped.

This exposed a real gap in Metacheck: `download_repo_files()` already gates a repository outright by total byte size (`max_download_size`) or per-file byte size (`max_file_size`), but had no gate on file count — a repository holding thousands of individually small files was not caught by either existing cap. A new parameter, `max_files_per_repo` (`R/repo-download.R`, threaded through `data_check()` and `code_check()`), gates a repository by file count alone,

using the same refusal/reporting mechanism as the existing size caps. This has been committed to the `repocheck_improvements` branch alongside a regression test, and the corpus rerun was configured with `max_files_per_repo = 1000`.

Under this cap, the SAPFLUXNET repository was refused outright in that run. The cap exists so that one exceptionally large repository cannot block or degrade processing of the other 1,860 papers in a corpus run. In the results reported in this manuscript, however, the repository was not refused: `data_check` lists all 3983 of its files, every one of them classified as data.

A.5 A targeted rerun on every remaining disagreement, after further platform-detection and reliability fixes

After Appendix A.1–A.4's fixes and the resulting full corpus rerun, every remaining disagreement between Metacheck and Cooper et al.'s coding (991 rows across 989 papers, the same set individually adjudicated for the Empirical Comparison above) was re-checked a further time against Metacheck as it stood after seven more commits landed on `scienceverse/metacheck`'s `dev` branch: two fixing crashes and false negatives in Metacheck's Zenodo integration, three widening which repository platforms `repo_check` can locate a repository on at all (targeting "repository not detected," the single largest recurring disagreement cause identified in the original review), and two fixing a rate-limit reliability defect surfaced live during this rerun itself. Rather than reprocess the full 1,861-paper corpus, only the 989 papers with at least one open disagreement were rerun, one paper at a time (so that a single paper's slow host response could not block any other), reusing the shared on-disk repository-file cache from the original corpus run so that a repository already fetched and unchanged cost no new download.

Zenodo reliability. Two fixes addressed real crashes and false negatives in Metacheck's Zenodo integration, confirmed against Zenodo records already present in this corpus. `online()`, the function `repo_check` calls to confirm a host is reachable before querying it, treated a single transient DNS lookup miss as "Zenodo is offline" and abandoned that paper's entire Zenodo retrieval rather than retrying; it now retries. Separately, a JSON metadata field arriving as a length-zero value (an empty array) rather than `NULL` defeated the `%||%` operator used throughout Metacheck's metadata-extraction code (which only guards against `NULL`, not length-zero), producing a `data.frame()` "arguments imply differing number of rows" error that was silently swallowed inside a `tryCatch`, discarding the whole record; a new `%empty_or%` operator, applied everywhere the same gap existed (Dataverse, Dryad, DSpace7, Figshare, GitHub, GitLab, OSF, PsychArchives, and ReShare, not just Zenodo), closes this. A further, closely related pair of crashes was found and fixed the same way: a Zenodo record Metacheck could not read at all previously returned a `data.frame()` with only `zenodo_id`/`error` columns, so a paper whose every Zenodo record was unreadable produced a metadata table with no `doi`/`license` columns at all, which crashed `repo_check()`'s own Zenodo-metadata-extraction step for that paper the same "differing number of rows" way; and a `repo_url` arriving as a list column (rather than a plain character vector) could silently explode `data.frame(zenodo_url`

= <list>) into one column per list element instead of the expected single column. Both are now guarded.

Repository platform coverage. Three commits substantially widened the set of hosts Metacheck's repo_check can locate a repository on at all, directly targeting the "repository not detected" cause responsible for the largest share of the original disagreement (the original review named DataSTORRE, DIGITAL.CSIC, AADC, SRA/GenBank, KNB, Neotoma, CIRAD Dataverse, and ForestPlots.net as example platforms with no backend at the time). Mendeley Data (detected via 10.17632/ DOIs and data.mendeley.com URLs, queried through the public, unauthenticated data.mendeley.com/public-api/datasets/<id> endpoint) and DataONE (a federation of independently-run repositories rather than one hosted service; four member nodes verified live and wired in — Arctic Data Center, KNB, Taiwan Forestry Research Institute, and the Smithsonian Tropical Data Portal, each mounting the same underlying REST API at a different base path) were added as new platforms entirely. Separately, Dataverse's recognised-installation list was expanded from 50 to 121 hosts, 16 institutional Figshare white-label front ends were added, and DSpace support was generalized from PsychArchives specifically to any host running the same legacy (6) API (5 more verified live) plus a new implementation covering 64 DSpace 7 installations (e.g. Cornell's eCommons, Georgia Tech's Digital Repository, ETH Zurich's Research Collection) — every one of these 121 + 16 + 5 + 64 hosts confirmed live via its own version/info endpoint before being added, not merely listed from a registry. A further, smaller fix excludes an R package's own source tree (DESCRIPTION/NAMESPACE/its R//man//tests/ subdirectories) from a repository's data/code file listing, on the reasoning that reusable software infrastructure bundled alongside a study's own materials is not itself the study's data or analysis code and should not inflate files_n/files_code or appear in downstream reports.

A reliability defect found live during this rerun. While this rerun was in progress, a single paper stalled repeatedly at the same point across two separate attempts: zip_peek()'s HEAD request and its Range-request follow-up (.http_range_tail(), R/zip-peek.R), used to read a remote zip archive's file listing without downloading it, each independently hit Zenodo's rate limit and each, independently, paid the full announced wait for the host's confirmed reset — three consecutive confirmed-rate-limit waits back to back for what was, from Zenodo's own side, a single rate-limit window, since neither request had any way to know the other had already discovered and waited out the same limit. (The rate-limit-wait message itself was also, at the same time, made to name the specific host responsible — Zenodo, Dryad, GitHub, or GitLab, the only hosts that expose a machine-readable reset time — rather than reporting only "the host's own reset," a smaller, secondary fix made in the course of diagnosing this one.) Fix: a session-scoped record of the last confirmed rate-limit reset per host, written the first time any request discovers one and consulted by every subsequent request to that host before it is even sent, so a host already known to be rate-limited waits out only the genuinely remaining time once, rather than every independent request rediscovering the identical window from a fresh 429. Verified with new regression tests covering the record/lookup logic directly (round-tripping a wait, keeping the later of two recorded resets, clearing an expired window, and the pre-flight wait actually sleeping the correct duration) and confirmed live: the same

paper that previously stalled for over 40 minutes across three stacked waits resolved in roughly 3 minutes with zero confirmed-rate-limit waits after the fix, the remaining time attributable to Zenodo's own server-side response latency for that specific record (confirmed independently: a single isolated request to the same record, made outside Metacheck entirely, itself took 30 seconds and returned a 504 on a second attempt).

Results of the rerun. Across the 1,532 individual construct-level disagreements re-checked (989 papers × up to 7 constructs each, matching however many constructs a given paper had originally disagreed on), 76.8% (1,176 of 1,532) are now resolved — Metacheck's fresh result agrees with Cooper et al.'s original human coding where it did not before — with 348 still disagreeing and 1 now returning a value that could not be classified as agreeing or disagreeing (`data_license`, a repository whose licence field this rerun could not read for reasons not yet diagnosed). By construct: `any_readme` 548/625 resolved (87.7%), `data_availability` 339/424 (80.0%), `code_archived` 252/360 (70.0%), `data_archive` 12/56 (21.4%, evaluated by set membership — a paper's platform coded by Cooper now appearing among Metacheck's own, possibly multi-platform, detected set, since a paper can genuinely link more than one repository), `data_license` 16/57 (28.1%), `data_download` 9/9 (100%), and `code_download` 0/1 (the sole remaining `code_download` disagreement in this corpus). The comparatively modest improvement in `data_archive` and `data_license` is consistent with the original review's own finding that these two constructs' disagreement was, unlike `data_availability`/`code_archived`/`any_readme`, not dominated by the repository-detection cause these fixes targeted (`data_license`'s largest cause was cases where Metacheck was already right and Cooper's own coding had missed a real licence field) — a fix aimed at finding more repositories naturally does less for a construct whose disagreement was rarely about a missing repository in the first place. This rerun's full row-level detail (every paper, every re-checked construct, the value before and after, and whether it resolved) is recorded in `cooper_validation_metacheck/manuscript/data/rerun_disagreements_comparison.csv`, produced by `rerun_disagreements.R`; the underlying per-paper `repo_check`/`data_check`/`code_check` results are cached per paper in `data/rerun_batches/` for inspection or a further rerun without recomputing what has not changed.

Checking the rest of the corpus for regressions. This rerun was scoped to the 989 previously-disagreeing papers only — a deliberate choice at the time, to get a fast answer to "did these fixes help" without the cost of a full 1,861-paper rerun (see the rerun's own header comment for the reasoning). That scoping left one real question open: none of these seven fixes were written to only affect a disagreeing paper, so it remained possible that a fix which resolved one paper's disagreement had, in a different paper that already agreed with Cooper et al., turned a correct result into an incorrect one — most plausibly the R-package-source-tree exclusion (if a paper's own analysis code happens to be organised as an R package) or the expanded Dataverse/Figshare/DSpace/Mendeley/DataONE host recognition (a newly-recognised host matching something that should not have counted). This was checked directly: the remaining 868 papers (every paper not in the 989-paper disagreement set) were also rerun, using the identical per-paper, resumable approach, and the full corpus's `cooper_vs_recreated.RData`/disagreement worklist were then rebuilt from the combined

result (all 1,861 papers) rather than just the 989. At the level of aggregate agreement rates, this full-corpus check was reassuring: the three constructs this comparison's own disagreement was concentrated in — `data_availability`, `code_archived`, `any_readme` — all showed a higher, not lower, agreement rate once the previously-agreeing papers were included, and `data_download`/`code_download` (both already above 99%) moved by less than a percentage point in either direction. But an aggregate rate rising is consistent with a handful of individual papers having genuinely regressed, provided enough other papers improved to more than offset them — and that is exactly what the case-by-case review below found.

A confirmed regression, found and fixed within this same cycle. The suspicion named above turned out to be correct: the R-package-source-tree exclusion did break papers it should not have touched. The exclusion was written to detect an R package's fingerprint (a `DESCRIPTION` and `NAMESPACE` file in the same folder) and drop that folder's contents — the package's `R/`, `man/`, `tests/`, `vignettes/`, `data/`, `inst/`, and `src/` subdirectories — from a repository's file listing entirely, on the reasoning that a bundled, reusable software dependency is not itself a study's data or analysis code. That reasoning holds when the package is a genuine dependency vendored alongside a study's own files, but it does not hold when the paper's own contribution — its data, its analysis code, or both — is itself packaged and distributed as an installable R package, a common practice in ecological/evolutionary methods work. In that case the exclusion strips the paper's own materials, leaving nothing behind: `repo_check`'s listing for the affected repository showed only packaging scaffolding (`.Rbuildignore`, `.gitignore`, `NEWS.md`, `README.md`, `cran-comments.md`, a bare `.Rproj` file), with the package's real `R/` source — and, for packages that bundle their own example or study data under `data/`, the actual dataset — entirely absent.

A corpus-wide scan for this exact symptom (a repository whose only surviving files, after the exclusion ran, were scaffolding, with zero files classified as data or code) surfaced candidates across the full 1,861-paper corpus, each individually verified against the repository's real GitHub listing (via `gh api`) or Dryad/Zenodo record before being counted: 16 confirmed true instances (`RevGadgets`, `BaBA`, `m2b`, `RRPP`, `actel`, `scampr`, `MINOTAUR`, `LadderFuelsR`, `biomonitoR`, `EcoEnsemble`, `fossilsim`, `frair`, `tRophicPosition`, `sabinaNSDM`, `bef`, and `phylomice`), plus several candidates from the same scan that turned out to be unrelated causes once checked directly (a repository with no `DESCRIPTION`/`NAMESPACE` at all, and several Dryad/Zenodo-hosted papers whose `code_archived` disagreement traced to an unconnected, pre-existing detection gap — code cited in a journal's own Supporting Information, or a repository never located by `repo_check` at all). Two confirmed cases (`BaBA`, a GitHub-hosted barrier-behaviour-analysis package whose `data/` folder holds the actual tracked-animal datasets for its own paper, and `MGDrivE`, a general-purpose mosquito gene-drive simulation package vendored inside a study-specific fork) between them illustrated the structural distinction the fix needed to draw: `BaBA`'s package sits at its repository's root with nothing else present — the package is the deposit, so nothing should be excluded — while `MGDrivE`'s vendored copy is nested one level deep inside the fork, with the study's own `Examples/` scripts and data sitting as real, undisputed siblings at the repository root.

The corrected rule preserves the original exclusion's logic for a nested package (still always excluded, since a nested package by construction always has something else in the repository for it to sit alongside) but adds one check for a package detected at a repository's root: before excluding it, check whether anything outside the package folder classifies as real data or code, using `data_classify_files()` — the same rules-based classifier `data_check` itself uses elsewhere in the pipeline, rather than a separate, bespoke list of what counts as "real content." If nothing does, the package is not excluded at all, since there is no "outside" for the study's own materials to live in. One implementation detail required a second pass: an RStudio project marker file (`*.Rproj`) classifies as `"code"` under `data_classify_files()` purely by its file extension, which would have wrongly counted as "real content survives" for every package that ships one (nearly all of them) — an RStudio project file is packaging configuration, not an analysis script, so it is excluded from this specific check. The fix was verified against two new regression tests (a root-level package with nothing else present, confirming it is no longer excluded, and a nested vendored package alongside real sibling content, confirming the vendored files are still excluded while the sibling content survives) alongside the existing test suite, then reapplied to the 38 papers the corpus-wide scan had flagged.

Reapplying the corrected exclusion resolved 11 of the 16 confirmed cases outright. The remaining 5 continue to show `code_archived` disagreement. An earlier version of this manuscript attributed these to the package's `R/` directory never having been downloaded; a later direct check showed that explanation was wrong. The file listing is complete, but the corrected rule itself still misfires: the rule makes an exception for RStudio project files, as described above, but not for continuous-integration configuration files at the repository root (such as `.travis.yml` and `codecov.yml`), which are classified as code by their extension. The rule therefore concludes that real code survives outside the package and keeps excluding the package's own source files. This was confirmed in detail for `biomonitoR` (238 files retrieved, 229 of them excluded) and is recorded as a separate, still-open defect (scienceverse/metacheck#469). Two of the 16, `phylomice` and `morton` — cases surfaced by the same corpus-wide scan but outside the original 13-paper set found during the initial regression check — resolved to, respectively, `METACHECK_RIGHT` (`phylomice`'s six R source files are a genuine methods contribution the paper's own text never explicitly claims as "archived code," but which Metacheck is nonetheless correct to report) and `COOPER_RIGHT` (`morton`'s 37 R source files are unambiguously this paper's own phylogenetic-analysis code, which Cooper et al.'s coders had missed).

A.6 A systematic platform-coverage audit, three further metacheck fixes, and a complete categorization of every remaining repository-detection disagreement

A sixth round of work started from a single case the previous rounds had not caught: a paper citing a Dataverse installation (`borealisdata.ca`, Borealis, the Canadian national repository) under a DOI prefix Appendix A.5's Dataverse allowlist did not recognise. Tracing why surfaced a systemic problem in how that allowlist had been built, and prompted a full re-audit of every

remaining "repository not detected" disagreement in the corpus rather than a single-paper patch.

A systemic under-sampling bug in the Dataverse DOI-prefix allowlist. Appendix A.5's per-host Dataverse DOI-prefix allowlist (`.dataverse_doi_prefix_hosts()`) was originally built by sampling the first 25 DOIs DataCite's search API returned for each host — a sample, not a census. This badly under-counted any host with more than 25 real DOIs registered, which turned out to be the majority of the 121 known hosts; `borealisdata.ca` alone has minted DOIs under six distinct prefixes, of which the original allowlist recorded only one. Rebuilding the allowlist using DataCite's facet aggregation (`disable-facets=false` on its `/dois` search, which returns every distinct prefix in use for a host, with its real count, in one request rather than a sample) found 100 distinct prefixes across the same 87 hosts, not 87 — including a second Harvard Dataverse prefix (`10.26300`, alongside the dominant `10.7910`) that the original sample had also missed. Verified live before and after: the DOI that surfaced this bug (`10.7939/DVN/M6Q2PE`, Borealis) now resolves correctly, and the second Harvard prefix now resolves to `dataverse.harvard.edu`. This was reported and merged upstream as scienceverse/metacheck#410.

Two further platform-coverage gaps found and fixed the same way. `.figshare_id()` did not recognise a Figshare article URL with no resource-type segment (`figshare.com/articles/<slug>/<id>`, as opposed to `figshare.com/articles/dataset/<slug>/<id>`) — an older, still-current URL shape (confirmed live against a real corpus citation) that `figshare_links()` already detected as a Figshare link but could not resolve to a usable article id, so `repo_check` never actually queried it (scienceverse/metacheck#412). Separately, `dataone_links()` had no entry for `data.piscoweb.org` (PISCO, a UC Santa Barbara-run DataONE member node), and `.dataone_pid()`'s bare-DOI regex could not handle a DOI suffix containing a further slash — PISCO's own DOI shape (`10.6085/AA/<id>`) — so this platform's citations, while real DataONE member-node DOIs, were invisible end to end. Both fixed and merged as scienceverse/metacheck#414, which also added 29 further institutional Figshare instances (28 found by searching DataCite's client registry for "figshare" in the client name, plus Monash University's `bridges.monash.edu`, found separately since its DataCite client is registered under the institution's own name) — each confirmed live, the same way as every host already in `.figshare_vanity_hosts()`, to answer Figshare's own single-page-app shell rather than a real API response, and each institutional DOI prefix confirmed live to resolve to that exact host before being added.

Two matching bugs in this project's own comparison code, not in Metacheck. `03_recreate_cooper_columns.R`'s `.repo_url_platform()` — the function that labels which platform `mc_data_archive` reports for a paper — had a Dryad branch that only matched the single most common DOI prefix (`10.5061`), and a Figshare branch that only matched the literal `figshare.com/10.6084` domain and prefix, missing every one of the alternate Dryad prefixes and institutional Figshare instances Metacheck itself already recognised after the fixes above. The practical effect was a paper whose repository was now correctly found and retrieved still showing a `data_archive` disagreement,

because the label fell through to the generic "Other repo/database" instead of naming the real platform. Both branches were rewritten to read directly from Metacheck's own `.dryad_doi_prefixes()`/`.figshare_vanity_hosts()`/`.figshare_doi_prefix_hosts()` rather than duplicating either list by hand, so this classification cannot drift out of sync with what Metacheck itself recognises again.

Reprocessing every paper these fixes could plausibly affect. Rather than assume the fixes above were the only remaining cause of repository-detection disagreement, every "repository not detected" comment still open in the worklist (351 individual (paper, column) disagreements across 233 papers, after excluding platforms already confirmed unsupported for an unrelated reason) was checked again: first narrowed locally, with no network calls, to the papers whose cited DOI prefix genuinely matched one of the platforms just fixed (27 candidates, after excluding two DOI prefixes that coincidentally collide with an unrelated platform's registered prefix in DataCite's own data — `10.6084`, Figshare's own prefix, and `10.17617`, Edmond's — neither of which any fix above touched), then live-verified one at a time. Of those 27, 13 were genuinely resolved by the fixes above; the remaining 14 were not — some detect the citation correctly but the dataset is embargoed or restricted at the source (confirmed via the repository's own API returning an explicit "cannot be viewed" message, not a Metacheck-side failure), and a few still do not detect the citation at all, for reasons unrelated to any fix in this round. A further, separately-run check across every remaining "never located" comment surfaced one exact repeat of Appendix A.1's original zip-peek limitation on a much larger scale (24 papers whose Zenodo/Figshare/Dataverse archive was never successfully peeked during an earlier corpus build, confirmed live, unmodified, to peek correctly today — a stale corpus row in every one of the 24, not a code defect). A further paper, initially suspected to be a second such stale row (its earlier 4TU fetch crash was already fixed in Appendix A.1 and appeared simply never rerun since), was checked directly against `rerun_disagreements_comparison.csv`'s own before/after values and found to still disagree after being rerun — the 4TU fetch failure was not, in fact, resolved. A later, more thorough recheck (Appendix A.9) traced the actual cause: 4TU is a genuinely supported platform, but its DOI-embedded identifier does not match the numeric article ID 4TU's own API requires, a distinct, still-open defect from the one Appendix A.1 fixed. This paper is accordingly counted as `SUPPORTED_PLATFORM_BUG`, not `UNSUPPORTED_REPO`, in Appendix A.9's categorization. All resolved candidates from this round were rerun end to end (`repo_check`, `data_check`, `code_check`) and patched into the corpus; none of this round's rerunning touched a paper outside the specific set each check identified, so no regression check across the unaffected remainder of the corpus was needed the way Appendix A.5's broader platform-coverage changes required.

A.7 A full-corpus `reproducibility_check` run: method, outputs, reasons for non-reproducibility, and suggested improvements to Metacheck

A seventh round of work ran Metacheck's `reproducibility_check` module — a module distinct from `repo_check`/`data_check`/`code_check` (the three modules the rest of this manuscript is

about) in that it does not merely detect and classify a paper's shared data and code, but actually **executes** the paper's own downloaded code against the paper's own shared data, inside an isolated sandbox, and checks whether the statistical results the code produces match what the paper itself reports in its text. This appendix documents the method, every output file this run produced, a full breakdown of why papers were and were not reproducible, four new Metacheck defects this run surfaced (all reported upstream, two confirmed fixed on `dev` before this appendix was written), and a set of concrete suggestions for further improving `reproducibility_check` itself.

A.7.1 Method

`reproducibility_check(paper, execute = TRUE, sandbox = "docker", install_missing = TRUE, timeout = 3000, cache = TRUE, workers = 5, results_dir = ...)` was run once per paper across the full 1,861-paper corpus. For each paper with at least one code file, the module: (1) reuses `code_check`'s dependency extraction and `psychds_check`'s path-rewrite plan to materialise a throwaway copy of the paper's own Psych-DS-layout data and code; (2) installs the paper's declared package dependencies (CRAN, GitHub, or URL sources) into a throwaway library inside a Docker container, starting from the pre-built `ghcr.io/scienceverse/metacheck_r:latest` image (roughly 750 of the most common packages found across a prior corpus scan already installed, so most papers skip most of the install step); (3) runs each of the paper's scripts, in an automatically-derived dependency order, each in its own `docker run --network none --read-only --user <non-root>` invocation — network disabled and the filesystem read-only outside the materialised sandbox, a genuine containment boundary for running downloaded, unvetted third-party code, unlike the module's alternative `sandbox = "process"` backend (a bare `callr` subprocess, which isolates a crash but not the filesystem or network); and (4) extracts any statistical output the executed scripts printed to the console and compares it against the paper's own reported results (`match_reported_output()`), recording how many of the paper's own reported test statistics the executed code actually reproduced.

`workers = 5` runs up to five papers' Docker containers concurrently rather than one at a time, a feature added to Metacheck (`dev` branch, version 0.3.1) specifically to make a corpus-scale run practical; each concurrent container is automatically capped to `1/workers` of the host's CPU and memory, so five containers running at once cannot collectively exceed what the host machine has. `results_dir` writes each paper's complete result to its own file the moment that paper finishes, independent of any outer batch boundary, so a run interrupted partway through does not lose the papers that already completed. A dedicated wrapper script (`reproducibility_check/run_reproducibility_check.R`, `cooperbes-validation-manuscript` repository) drives the corpus-wide call, installing Metacheck fresh from `scienceverse/metacheck`'s `dev` branch at the start of every run (never a locally vendored copy, so the run always reflects the current state of the package's own repository)

and writing one checkpoint file per outer batch so a killed or crashed run can resume from the last completed batch rather than restarting from scratch.

A.7.2 Outputs stored

The run produced the following files, all in `code/data/` unless noted:

- **`reproducibility_check_results.RData`** — the final, combined result for all 1,861 papers: an `out` object with `out$results$table` (one row per script attempted across every code-bearing paper; counts in A.7.4: `paper_id`, `file_name`, `outcome`, `error_type`, `undefined_var`, `not_runnable_reason`, `runnable`, `parses`, `run_order`, `paths_rewritten`, `paths_ambiguous`, `setwd_removed`, `unresolved_inputs`, `reads`, `writes`) and `out$results$summary_table` (1,861 rows, one row per paper: `paper_id`, `repro_code_n`, `repro_runnable`, `repro_missing_inputs`, `repro_deps`, `repro_tests_reported`, `repro_tests_matched`, `repro_ran_ok`). This is the file the numbers below are computed from.
- **`data/batches/`** — one `.RData` checkpoint file per outer batch processed during the run (batch-size-tagged in its filename, so a checkpoint from one `--batch-size` setting is never mistaken for one from another), and, under `data/batches/worker_results/`, one `.rds` file per paper (`capture_module_tables()`'s own per-paper checkpoint format), the finer-grained safety net `results_dir` provides underneath the outer batch files.
- **`reproducibility_check_infra_rerun.RData`** and **`reproducibility_check_13384_solo_rerun.RData`** — two small, targeted follow-up runs (below), covering respectively the 16 papers whose first-pass result was an infrastructure failure rather than a genuine check, and one paper rerun alone (no concurrency) after crashing twice under `workers = 5`.
- **`reproducibility_check_execute_warnings.log`** — every R warning raised during the run, forwarded here as it happened (R's own default non-interactive behaviour silently drops all but the first 50 warnings of a session, which a run this size would exceed many times over).
- Two standalone wrapper scripts alongside the main run script, both in `reproducibility_check/`: `rerun_infra_failures.R` (the 16-paper targeted rerun) and `rerun_single_paper.R` (the one-paper, no-concurrency rerun).

A.7.3 Scope: which papers had code to check at all

Of the 1,861 papers in the corpus, 382 (20.5%) had at least one code file for `reproducibility_check` to attempt (`repro_code_n > 0`); 1,478 (79.4%) had none, and are not further discussed in this section, since there is nothing for the module to execute — this is the same "does the paper share any code" question `code_check`/`repo_check` already answer elsewhere in this manuscript, not a new finding. 1 paper(s) have no `repro_code_n` value at all (`NA`), including 10_1111_2041_210x_13384, the one paper described in A.7.5 below whose check was manually aborted rather than completed.

Note on this count. This section's original run (below) found code for 334 papers. Appendix A.8 describes three further Metacheck fixes (`zip_peek()` failing on S3-backed hosts, its result never persisting across a restart, and a silently discarded download error) that, once applied and the corpus rerun, let `repo_check/code_check` correctly find and download a code repository, or a code file inside one, for papers where they previously could not — 98 papers corpus-wide whose available code or data content genuinely changed. Those 98 papers' `reproducibility_check` result was rerun against the same fixed Metacheck version and merged into this section's own combined result; every other code-bearing paper's original result, from before Appendix A.8, is unchanged. 9 of the 98 newly showed `repro_code_n > 0` where they previously showed none, bringing the code-bearing total from 334 to 343 at that point, and `repro_ran_ok` improved for 5 of the 98. Later still, the 71 papers whose code includes Python were rerun with a Metacheck version that also executes Python scripts (the original run executed only R files), and the 70 with a result were merged in the same way; the rest kept their earlier result (one paper, 10_1111_2041_210x_14196, whose code is a 97-file Python package, was stopped after two hours in which Metacheck had not yet started running it). Corpus-wide, 243 papers now have at least one script that ran without error. The numbers throughout the rest of A.7.3–A.7.4 below reflect this merged, current result.

A.7.4 Reasons for non-reproducibility: a full breakdown

Across the 382 code-bearing papers, 7,133 individual scripts were attempted, 2,689 of them Python (`.py` or `.ipynb`) and the rest R (one paper's script count was not resolved; see A.7.5). Their outcomes:

Outcome	Count	% of 7,133
`ran_ok` (executed without error)	2,421	33.9%
`errored` (executed, but raised an error)	3,621	50.8%
`skipped_missing_inputs` (an earlier script's output this script needs was never produced or found)	681	9.5%
`dependency_unavailable` (a package the script needs could not be installed, so it was not run)	343	4.8%
`not_parsed` (the file is not valid syntax in its declared language at all)	52	0.7%

Outcome	Count	% of 7,133
`timed_out` (still running when the 3,000-second per-script cutoff was reached)	15	0.2%

At the paper level, only 82 of the 382 code-bearing papers (21.5%) had every one of their scripts run without error; 139 (36.4%) had zero scripts run successfully at all; the remaining 161 (42.1%) had a mix of successful and failed scripts within the same paper.

Among the 3,621 `errored` scripts (plus the 15 `timed_out` scripts, grouped in here since both carry an `error_type`), `error_type` breaks down as follows; Python scripts report the name of the Python exception instead (1,207 scripts, most often ImportError), and 425 carry no `error_type` at all:

- **702 (19.3%) `undefined_variable`** — the script references a variable, object, or function that was never defined at the point it is used, the classic signature of a script written to be run only as one step within a larger, undocumented pipeline (e.g. an interactive session, or a preceding script the paper's repository does not include or that this run's automatic dependency ordering could not place before it). The single most common such name is the pipe operator **`%>%` itself (15 occurrences)** — a script using `%>%` without ever calling `library(magrittr)`, `library(dplyr)`, or (on R versions before 4.1) without the base pipe `|>` available, a specific and mechanically fixable non-reproducibility cause distinct from the rest of this category's more idiosyncratic missing-object cases. `.jinit` (13 occurrences, from `rJava`-dependent packages needing a Java runtime this Docker image does not provide) is among the next most common, followed by a long tail of paper-specific object and function names.
- **1,287 (35.4%) `runtime`** — every other kind of R error raised during execution: a missing package the install phase could not provide (see repro_deps below), a file the script expects that a path-rewrite could not correctly relocate, a genuine bug in the paper's own analysis code, or code that assumes an interactive session (see the two concrete cases below).
- **15 (0.4%) `timeout`** — see the 3,000-second cutoff above; a timeout is not itself counted as strong evidence of non-reproducibility (the paper's own analysis may simply be legitimately slow, e.g. a Bayesian model or a large agent-based simulation — several confirmed cases during this run were exactly this), just "did not finish within the budget this run allowed."

Two concrete, directly observed non-reproducibility patterns are worth naming explicitly, since neither is visible in the `error_type` classification above (both were classified `runtime`, and the module does not currently distinguish "this code assumes an interactive terminal" as its own category — see A.7.6's first suggestion): (1) a script calling base R's `read.csv()`

with no path argument, which opens an interactive `file.choose()` file-picker dialog — impossible inside a headless, network-disabled container, and a strong signal the paper's own code was never actually run non-interactively even by its own authors; (2) a script calling `rstanarm::launch_shinystan()`, which starts a Shiny web server and blocks waiting for a browser connection that can never arrive in this sandbox — resolved only by the per-script timeout eventually killing it, after up to 3,000 seconds spent doing nothing.

Missing inputs. 154 of the 382 code-bearing papers (40.3%) have at least one script whose required input was never produced or found (`repro_missing_inputs > 0`); 607 individual scripts are recorded `not_runnable_reason = "missing_input"`, the largest single `not_runnable_reason` category by a wide margin (71 `parse_error`, 47 `unplaceable` — a script the run-order algorithm could not confidently place in a dependency sequence at all — and 3 `jags_model`, a script that is itself a JAGS model-definition file rather than executable R).

Dependency installation. Before running a paper's code, Metacheck must first install the R (or, for Python scripts, Python) packages that code declares it needs. 315 of the 382 code-bearing papers (82.5%) declare at least one such package (`repro_deps`, a per-paper count of declared packages — not a count of failures, and not tied to any particular script or file), 4,441 packages in total. The per-package install success/failure counts, the specific packages that failed and how many papers each affected, and the result-matching statistics later in this subsection come from a dedicated extraction against each paper's saved checkpoint report text (`01_run_metacheck/08_extract_reproducibility_detail.R`), which reads each paper's own free-text install-summary sentence and, for result matching, its row-level `match_table`. This extraction was rebuilt for the current, merged 382-paper result; the original script that first built it for the pre-Appendix-A.8 pass was not itself preserved anywhere in this project (see the note further below).

Metacheck's own `summary_table` records only how many packages a paper declares (`repro_deps`), not which ones failed to install or why — a real limitation this appendix works around below, and A.7.6's second suggestion recommends Metacheck fix directly. This appendix could still recover the failed packages' actual names, because each paper's saved checkpoint file also keeps a plain-text summary sentence restating its own install attempt (e.g. "13 succeeded, 9 failed. sf, rgdal, … were installed. **Failed:** countreg, DTK, …"); re-reading that sentence for each paper is how the list below was built. It names at least one failed install for 204 papers, 818 failed-install records in total.

One further point, worth stating plainly rather than folding into the numbers above: Metacheck has its own outcome, `dependency_unavailable`, for a script it does not run because a package the script needs could not be installed. It was recorded for 343 scripts, 343 of them Python scripts. An R script whose package failed to install was still run, and its failure recorded as an ordinary code error, which this appendix takes as license to keep treating a failed install as a real, meaningful cause of non-reproducibility, not something to set aside as merely today's state of the package ecosystem.

The rest of this subsection focuses on the 84 papers with a real install failure, and no script that ran successfully. This is the subset where a failed install is most clearly the reason the paper's code did not run.

Note on this list. As with the numbers in A.7.3/A.7.4 above, this list was rebuilt (using this project's own `01_run_metacheck/08_extract_reproducibility_detail.R`, re-parsing each paper's saved checkpoint report text) after Appendix A.8's rerun, since the original extraction script that first built it was not itself preserved anywhere in this project (only its output — `01_run_metacheck/batches/reproducibility_check/pre_a8_reference/` — survived). The rebuilt extraction was checked directly against that surviving output for every package outside Appendix A.8's 98-paper rerun before being trusted: of the 90 packages the original extraction found, 86 have an identical paper count in the rebuild, 2 (`rethinking`, `rJava`) changed by one paper each as a direct, explicable result of the rerun, and 2 (`ascr`, `pgirmess`) no longer appear because the one paper each had failed for moved out of this 76-to-80-paper subset (its code now has at least one successful script, or its failed install no longer applies) — a small, fully accounted-for set of differences, not evidence of a remaining extraction problem.

Which packages failed to install, for the 84 papers with a failed dependency and zero successful scripts. Every one of these 84 papers has a `report` field in the saved checkpoint data with a recoverable, itemised list of which specific packages failed. Across those 84 papers, 274 individual failed-install records were recovered (one per package and paper), naming 194 distinct packages, R and Python. 4 of those (`x`, `package`, `arma`, `packages.toinstall`) are not genuine package names at all but a further Metacheck defect: they were each confirmed absent from CRAN entirely and, on direct inspection of the paper's own repository, are variable names from the paper's own install-loop code (e.g. `for (pkg in list.packages) install.packages(pkg)`) that Metacheck's dependency extractor mistook for a literal package name to install — see A.7.5's newly-added fifth defect below.

By far the most common genuine cause is a small cluster of interdependent **spatial/GIS packages**: `sf`, `rgdal`, `rgeos`, `maptools`, `stars`, `ggspatial`, `mapview`, `tmap`, `gdalUtils`, `nngeo`, and `spatial.tools` are not ordinary self-contained R packages but R interfaces to external, non-R system libraries — GDAL (reads/writes geographic file formats), GEOS (geometric operations such as intersections and buffers), and PROJ (coordinate-system transformations) — which must already be installed at the operating-system level before the R package can even compile, let alone load; installing the R package alone is not sufficient. `sf` (18 papers), `rgdal` (16), `maptools` (6), `rgeos` (6), `ggspatial` (4), and `stars` (3), together with the smaller one/two-paper members of the same cluster, account for 59 of the 274 records (21.5%) — consistent with a single shared environment gap (`ghcr.io/scienceverse/metacheck_r:latest` apparently does not have GDAL/GEOS/PROJ installed) rather than 59 independent per-paper causes; see Suggestion 2 below, which recommends closing this gap directly in Metacheck's own Docker image rather than leaving it to recur for every future spatial-analysis paper. The next-largest identifiable cluster is **Bayesian/JAGS modelling**: `rjags` (4 papers), `R2jags` (1), and `INLA` (0) — JAGS is itself a separate, non-R program that must be installed on the system for `rjags` to link against, the same category of gap as the spatial cluster. A `Java`-dependent cluster

(`rJava`, 4 papers, matching the `.jinit undefined_variable` pattern reported above) similarly requires a system-level Java runtime, not just the R package.

Most other packages in the list below failed for one paper each. This appendix cannot say why each of these one-off packages failed to install, and one specific possibility this run's own method already rules out for the great majority of them is worth naming directly: Metacheck's `sandbox = "process"` install backend (`repro_install_deps()`) retries a failed live-CRAN install once against the CRAN Archive (a package's last-published version before CRAN removed it) before giving up — but this run used `sandbox = "docker"`, whose install backend (`repro_install_deps_docker()`) does **not** implement this same Archive retry (confirmed directly against its own source; its `via_archive` field is hard-coded `FALSE` for every install, with an explicit comment noting the retry "is not duplicated here"). So a package that failed here only because its current CRAN release could not compile, but whose Archive-held prior version would have installed cleanly, was never given the chance this run — the Archive retry that might have recovered it simply was not attempted, independent of whatever the package's own true availability status is. A handful of names (e.g. `hmRLib`, `poispkgs`, `tiagospackage`) are also plausibly a typo or a private/unpublished package never intended for public installation at all, though this was not individually verified for each. This is a separate, more specific point than this appendix's general scope note below: it is not merely that the reason for each one-off failure is unrecorded, but that one entire category of possible reason (an Archive-recoverable package) was structurally never tested for in this run at all.

The full list, package name and number of these 84 papers it failed for (n = 1 except where noted): **sf** (18), **rgdal** (16), **maptools** (6), **rgeos** (6), **sklearn** (5), **ggsn** (4), **ggspatial** (4), **rjags** (4), **rJava** (4), **cv2** (3), **phyloseq** (3), **stars** (3), **amt** (2), **Biostrings** (2), **dada2** (2), **decontam** (2), **gdalUtils** (2), **glmulti** (2), **matplotlib** (2), **mcmcplots** (2), **numpy** (2), **pandas** (2), **rethinking** (2), **scipy** (2), **ShortRead** (2), **tensorflow** (2), **tmap** (2), **x** (2), adafruit_rfm9x, adiv, ann_fncts, arma, ascr, asreml, AviaNZ, AviaNZ_batch, AviaNZ_batch_GUI, AviaNZ_manual, betalink, bioinfokit, brtest, busio, CAST, cavity, classecol, click, Clustering, CNN, colourMaps, CommEcol, countreg, cowplot, cStringIO, Cython, dartR, Dialogs, DialogsTraining, digitalio, diveRsity, dplyr, DTK, embryogrowth, emmeans, evobiR, ext, FancyBaSTA2, Features, fishflux, FRQNT1.Data, funcresp, GeoLight, GeoLocTools, ggalt, gghalves, ggmosaic, ggordiplots, ggpattern, ggplot2, GISTools, git2r, glmmADMB, gurobi, GVI, hdi, hier.part, hilldiv, hmRLib, hms, InformationValue, IPMpack, joblib, jpeg, jsonschema, keras_preprocessing, labeledarray, Learning, LevinsonDurbanRecursion, librosa, lme4, lmerTest, loop, lotvol, lubridate, mapview, MCMCglmm, microbiome, microclima, model, models, MySQLdb, NicheMapR, odeintr, openpyxl, osgeo, package, packages.toinstall, pairwiseAdonis, PAMLr, panoptes_client, parameters, PCAviz, peakutils, performance, performanceEstimation, pez, phylolm, picante, PIL, plotrix, prioritizr, purr, pydub, pyfftw, PyInstaller, pylab, PyQt5, pyqtgraph, pywt, R2jags, RAMClustR, Rarefy, razimuth, read_yaml, reduction, Reefs3D_workflow_functions, rgl, rich, rjson, RNetLogo, sampling, scikits, seewave, Segment, seqRFLP, serial, setuptools, SGAT, show, siar, SignalProc, sit, skimage, SNPRelate, sompy, spatial.tools, spdep, spectrum, speechmetrics, SplitAnnotations, statsmodels, StringIO, subtitles, SupportClasses, SupportClasses_GUI, sympy, tabulizer, tess3r, tidync, tidyr, tidy-

verse, Training, trophic, tuneR, TwGeos, vegetarian, viridis, waffle, Wavelet, WaveletFunctions, WaveletSegment, wavio, weathercan, xgboost, yaml.

A note on scope for this package list. This list names which package failed to install, for how many papers, within the 84-paper subset defined above (a genuine install failure, and zero scripts of that paper ran successfully); it is not a claim about why each package failed (a version incompatibility, a missing system library, a compilation failure, genuine current unavailability, or something else) — Metacheck's own `.repro_classify_install_message()` classifier already exists and runs during the install phase, but its per-package `category` output is not retained in this run's paper-level `summary_table`, the same gap A.7.6's second suggestion recommends closing. Nor does this list cover the 120 further papers (204 with a genuine install failure, minus the 84 covered here) that had at least one failed dependency but still had at least one script run successfully — extracting and reporting that broader set's package list, and each package's classified failure reason once Metacheck retains it, is a natural extension of this analysis and is not attempted here.

Result matching: did the code's own output match what the paper reports? This is a distinct, further step `reproducibility_check` performs beyond simply running a paper's code: for every paper whose code did produce console output, it extracts each statistical result the paper's own text reports (a t-test, a correlation, a regression coefficient) and checks whether that exact value appears anywhere in what the executed code actually printed (`match_reported_output()`). This is the closest this run comes to asking not just "does the code run" but "does running it get you the paper's own numbers."

Metacheck's statistical-output extraction and matching machinery is not, in general, R-only: `reproducibility_check` also reads and STATO-tags the rendered result tables inside a `.jasp`/`.omv`/`.spv` file (JASP, jamovi, or SPSS Viewer — self-reproducible bundles that carry both the data and its analysis together, requiring no code execution at all) via `read_stat_tables()`, feeding the same matching step as executed R output. Of the 66 reported values the code's output did match (see below; the source is recorded only for a matched value), however, 0 came from a JASP/jamovi/SPSS-Viewer file, 0 from executed Python code, and the rest from executed R code. This is a fact about which papers in this specific corpus had a matchable self-reproducible output file, not a restriction the matching step itself imposes. `reproducibility_check`'s own code-execution phase runs R and Python files only (`inst/modules/reproducibility_check.R`; Python since the Metacheck version used for the Python rerun), so a SAS, SPSS-syntax, Stata, or MATLAB script is never itself run, whether or not a self-reproducible output file exists alongside it.

136 papers had at least one statistical result Metacheck could both extract from the paper's own text and attempt to match against the executed code's console output (`repro_tests_reported > 0`); across those 136 papers, 821 individual reported test statistics were checked, of which only 66 (8.0%) were matched by the code's actual output. At the paper level, the median proportion of a paper's own reported results its code reproduced was **0%**: only 1 of the 136 papers (0.7%) had its code reproduce every reported result it checked, 126 (92.6%) reproduced none of them, and the remaining 9 (6.6%) reproduced some but not all. This is the single starkest number

this run produced, and — read together with the execution-outcome breakdown above — is consistent with a straightforward, largely mechanical explanation rather than a claim about the underlying research: a script has to actually run to completion and print a result before that result can be compared to the paper's text at all, and only 33.9% of scripts, and just 21.5% of papers in full, cleared that first bar in this run.

Was this matching step itself useful, beyond confirming the headline 8% figure? Yes, in two ways. First, of the 66 matched values, 62 (93.9%) matched at "full" confidence (every component of that reported test — the statistic, its p-value, its confidence interval — matched together) and only 4 (6.1%) matched "partially" (some but not all components of the same test found) — when this run's code did reproduce a result, it almost always reproduced it completely, not approximately; a partial match was the exception, not the norm. Second, the matcher records, separately from whether a value was found at all, whether an unmatched reported statistic could still be traced to a real, identifiable quantity in the code's output (`plausible_split`, flagged only for a test whose reported components had to be re-grouped from the paper's own text rather than read as originally written). Of the 755 reported values not matched, 541 (71.7%) were nonetheless flagged `plausible_split = TRUE` — meaning the value the paper reports does correspond to a real quantity the code computed somewhere, under a different grouping or a different analysis than the paper's own text suggested, rather than being absent from the output altogether. This is a genuinely useful distinction this run's matching step draws for free: "the code never produced anything resembling this number" is a materially different, more serious finding than "the code produced this number, just not packaged the way the paper's text implies it should be," and only the row-level `match_table` this appendix worked from here — not the paper-level `repro_tests_matched` count alone — can tell the two apart.

A note on scope and what this section does not claim. The numbers above describe whether a paper's own downloaded code, as shared, ran and reproduced its own reported results inside this specific execution environment (Docker, `ghcr.io/scienceverse/metacheck_r:latest`'s pre-installed package set, a 3,000-second per-script budget, no network access during execution). A script failing here is not necessarily evidence the paper's underlying research is wrong, or even that the code never worked — it may simply never have been intended to run outside the author's own machine, a point the `file.choose()` and multi-script "designed as an interactive pipeline" patterns above illustrate directly. Nor does this section attempt to re-diagnose why each of the 3,621 errored or 681 skipped scripts individually failed beyond the categories `reproducibility_check` itself records; a full per-paper qualitative review of this kind, in the same style as Appendix A.1–A.6's disagreement-by-disagreement review of `repo_check`/`data_check`/`code_check`, is listed as a further step in A.7.6 below rather than attempted here.

A.7.5 Infrastructure failures, found and fixed during this run

Running `reproducibility_check` at corpus scale, for the first time, using Metacheck's new `workers`-based concurrency feature, surfaced five defects in Metacheck itself — none related to a paper's own code or data, all specific to the mechanics of running many papers' Docker containers concurrently, checkpointing their results, or (the fifth) extracting a paper's own declared dependencies from its code. All five were filed as issues against `scienceverse/metacheck`; two are confirmed fixed on the `dev` branch as of this writing (verified directly against the installed package's own source, not merely by the issue's own open/closed status), the other three are filed and not yet independently reverified.

scienceverse/metacheck#417 — a Docker container left running after its own script's timeout fires. `repro_run_scripts_docker()`'s per-script timeout (`processx::run(..., timeout = timeout)`) kills the host-side `docker run` CLI process when a script exceeds its budget, but this does not guarantee the container itself stops — confirmed live twice during this run: once, a `caret`/`gbm` hyperparameter grid search (490 combinations) continued running inside its container at 100% CPU for over 80 minutes after its own timeout had already fired and been correctly recorded as `outcome = "timed_out"`; a second time, a script calling `rstanarm::launch_shinystan()` (the interactive-session case named in A.7.4 above) was left running, near-idle, on a listening port after its own timeout. Both were stopped manually (`docker stop`) after being noticed via `docker ps -a`/`docker stats` from outside the run entirely — nothing in the run's own log output indicated either container was still running. **Not yet confirmed fixed.**

scienceverse/metacheck#418 — a multi-line `deparse()` silently corrupts the generated install script for a paper with many dependencies. `repro_install_deps_docker()` builds the container's install script with `paste0(".repro_docker_pkgs <- ", deparse(install_deps$package` — but `deparse()` returns a **vector** of lines, not a single string, once its argument is long enough to wrap (R's own default line-width behaviour); `paste0()` silently keeps only the first line, splitting the vector literal apart mid-declaration. Confirmed live: a paper declaring 18 CRAN dependencies produced a generated `install.R` reading `.repro_docker_pkgs <- c("bayestestR", ..., "lubridate",` on one line, followed by a syntactically invalid continuation, which failed with `Error: unexpected symbol` and reported all 18 of that paper's dependencies as failed installs — not because any package itself was unavailable, but because the install script that would have installed them was never valid R in the first place. A full scan of every batch already completed at the time this bug was found identified 34 of 1,861 papers (1.8%) hit by this exact corruption signature; those 34 papers' checkpoint files were deleted and recomputed once metacheck#420 (below) was fixed and this run was restarted. **Not yet confirmed fixed** (the affected 34 papers were recomputed under the pre-fix code path avoided by deletion-and-rerun, not by an actual code fix landing on `dev`; a real fix for this specific defect remains open as of this writing).

scienceverse/metacheck#419 — `collect_module_tables()` always returns an empty result. `capture_module_tables()` (the function `results_dir` uses to write

each paper's per-paper `.rds` checkpoint) saves each module's output into an **unnamed** list; `collect_module_tables()` (the function meant to read those checkpoints back and reassemble them) looks each module up **by name** — so every lookup silently misses, and `collect_module_tables()` returns an empty data frame no matter how many papers were genuinely saved, with no warning that anything went wrong. This defeated `results_dir`'s entire stated purpose (a resumable, inspectable per-paper checkpoint) the first time this run's own wrapper script tried to use it to reconstruct a batch interrupted partway through. **Confirmed fixed** on `dev` (commit `033294d`, "Fix capture_module_tables() saving modules unnamed"): `capture_module_tables()` now names its saved list by each module's own `$module` field, with a positional fallback for a missing or duplicate name. This project's own run script does not depend on the fix either way going forward, since it was changed to read each paper's saved `.rds` file directly (`modules[[1]]`, positionally) rather than through `collect_module_tables()`, once the defect was found live and a fix could not be waited for mid-run.

scienceverse/metacheck#420 — `.reproducibility_check_batch()` can hang indefinitely, silently dropping the last papers in the queue. Confirmed live: a run processing 1,861 papers at `workers = 5` reached 1,859 of 1,861 with no error, no further log output, all five worker processes idle (alive but with frozen CPU time across repeated checks 10+ minutes apart), and zero Docker containers running — yet the function never returned. The two remaining papers had no `-> starting` log line at all; `launch()` had simply never been called for them. The maintainer's own diagnosis and fix (reported back to this project directly, not independently re-derived here) was a `callr` pipe deadlock: `callr::r_bg()`'s default stdout/stderr capture uses a pipe, and once a worker produces enough output to fill that pipe's buffer with nothing reading it, the worker blocks trying to write, indefinitely, with no error surfaced anywhere. **Confirmed fixed** on `dev` (commit `85c8c87`, "Fix .reproducibility_check_batch() hang from callr pipe deadlock"): each worker's stdout/stderr is now redirected to a file rather than a pipe, and read back from that file rather than via `p$read_all_error()`'s pipe-based accessor. The stalled run was killed and restarted after this fix landed; `results_dir`'s per-paper checkpoints meant the restart correctly skipped the 1,859 already-completed papers (reported live: `"resuming -- 1859/1861 paper(s) already saved in results_dir from an earlier interrupted run"`) and completed only the two genuinely outstanding papers.

scienceverse/metacheck#421 — dependency extraction can mistake a paper's own install-loop variable name for a package name. Found while extracting and summarising which packages failed to install across this run (A.7.4 above): a paper's own dependency-installing boilerplate commonly assigns its package list to a variable and iterates it in a loop (e.g. `list.packages <- c(...)`, then `for (pkg in list.packages) install.packages(pkg)`); in six confirmed cases, the extractor captured a bare loop-variable identifier itself — `x`, `package`, `dependencies`, `not_installed`, `arma`, `packages.toinstall` (plus, in a seventh and eighth case, `list.packages` and `req.lib` themselves, the variable names from the concrete example quoted in A.7.4) — as if it were a literal package name to install, rather than recognising it as a variable reference whose value static analysis cannot generally resolve. Each of these eight tokens was confirmed absent from CRAN entirely (`"x" %in%`

`rownames(available.packages())`, and equivalently for the other seven, all `FALSE`) and, on direct inspection of the originating paper's own repository code, traced to exactly this pattern. **Not yet confirmed fixed.**

Sixteen further papers recovered by a targeted rerun. Independently of #420's queue-loss bug, 16 of the 1,861 papers (0.9%) received a synthetic `traffic_light = "error"` stand-in result during the main run rather than a genuine check, due to either a `callr` subprocess itself crashing (`"could not start R, exited with non-zero status, has crashed or was killed"`, 14 papers) or a transient Docker Desktop outage on the host machine (`"Docker does not appear to be running"`, 2 papers) — infrastructure noise, not a property of any of these 16 papers' own code. All 16 were identified by grepping the run's own log for this failure signature, listed explicitly in `rerun_infra_failures.R`, and rerun in a small, separate, `workers = 4` batch once the main run finished; 15 of the 16 completed with a genuine result on this second attempt and were merged into the corpus-wide combined output in place of their placeholder rows. The remaining paper (10_1111_2041_210x_13384) crashed with the identical `callr` error on both the original run and the 16-paper rerun; a third, solo attempt (`workers = 1`, no concurrency, `rerun_single_paper.R`) found this paper's repository contains 314 separate scripts and a 1,386-row file-materialisation plan — substantially larger than any other paper's repository encountered during this run — and was manually cancelled, still mid-dependency-install, after several minutes with no sign of proceeding to actual script execution. This paper's result in the corpus-wide combined output is explicitly recorded as `outcome = "timed_out"` with a `not_runnable_reason` naming this exact sequence of events, rather than left as an ambiguous crash placeholder or given a fabricated genuine result.

A.7.6 Suggestions for improving `reproducibility_check`

The following suggestions are drawn directly from running the module at full corpus scale for the first time; each is grounded in a specific, confirmed observation from this run, not a general aspiration.

1. **Add a distinct outcome for "this script requires interactive input."** Both `file.choose()`-driven `read.csv()` calls and `launch_shinystan()`'s blocking Shiny server (A.7.4) currently surface as an ordinary `runtime` error or a `timeout`, indistinguishable from a genuine bug in the paper's code. A small, targeted check — recognising a short list of known-interactive calls (`file.choose()`, `menu()`, `readline()`/`readLines(file("stdin"))`, `shiny::runApp()`/`launch_shinystan()`/`plotly::orca()`-style blocking viewers) before execution, or classifying a script that produces zero stdout before timing out as `timed_out_interactive` rather than plain `timed_out` — would let a report distinguish "this code cannot run unattended by design" from "this code is simply slow," two very different findings for an author or reader trying to act on the result.
2. **Preserve the install-failure package name and reason, not just a count, through to the paper-level summary table.** `repro_install_deps_docker()`'s

own return value already carries both which package failed and a classified reason (a version incompatibility, a missing system library, a compilation failure, genuine unavailability — see `.repro_classify_install_message()`), but this run's corpus-wide saved `summary_table$repro_deps` retains only a per-paper count. This project could still recover the names by re-parsing each paper's free-text `report` field (A.7.4 above), but not the classified reason — a structured `summary_table` column (e.g. `repro_deps_packages`, `repro_deps_by_category`) would make both available directly, for every paper, without a second extraction pass against unstructured text. This would also let a report distinguish "this paper's code cannot run today because CRAN removed a dependency it used" (an ecosystem-drift problem, not a fault of the paper) from "this paper's code cannot run because it needs a system library this environment does not provide, e.g. the GDAL/GEOS/PROJ stack behind `sf`/`rgdal`/`rgeos`/`maptools`, together the largest single cluster of failed installs this run found" (an environment gap Metacheck could plausibly close by extending its own Docker image).

3. **Auto-stop a script's container on timeout (metacheck#417), not just the host CLI process.** As documented in A.7.5, a timed-out script's container can keep running, and consuming real host resources, for a long time after the timeout is reported — capturing the container's own ID (e.g. via a fixed `docker run --name`) and explicitly `docker stop`/`docker kill`ing it when `processx::run()` reports a timeout would close this gap directly, and would have prevented two of the interventions this run's own operator had to make manually.
4. **Guard every `paste0(prefix, deparse(x))` call site against multi-line `deparse()` output** (metacheck#418), not only the one instance this run happened to trigger — `deparse1()` (base R, available since R 4.0) exists specifically to always return a single string regardless of length, and is a direct drop-in replacement everywhere this pattern appears.
5. **Retry a `callr` subprocess crash once before recording it as the paper's result**, at least for `.reproducibility_check_batch()`'s `workers > 1` path. 9 of this run's 16 infrastructure failures (56.3%) were a bare `callr` subprocess crash with no more specific diagnosis available (`"could not start R, exited with non-zero status, has crashed or was killed"`) — a category that, by its own error text, cannot currently be distinguished from a genuine, reproducible crash in the paper's own code (as the twice-crashed paper in A.7.5 turned out to be) without a human manually re-running it, as this project had to do for all 16.
6. **Add a repository size/complexity gate to `reproducibility_check` specifically**, analogous to `download_repo_files()`'s existing `max_files_per_repo`/`max_file_size`/`max_download_si` caps (Appendix A.4). The one paper this run could not complete even alone (A.7.5) has a 314-script repository — nothing in `reproducibility_check` currently flags or specially handles a repository at this scale before attempting a full materialise-install-execute cycle on it, the same category of gap A.4 closed for `data_check`/`code_check`'s own download step.
7. **Make a stalled `workers > 1` run detectable from its own log**, independent of the underlying cause. Diagnosing metacheck#420 required inspecting Docker container state

and OS-level process CPU time from outside R entirely, since the run's own console output gave no indication anything was wrong — it simply stopped producing lines. A lightweight, periodic "N papers still running, M queued, last completion Ts ago" heartbet message, or a caller-configurable overall wall-clock ceiling independent of any individual script's own timeout, would have made this failure mode visible from the log alone.

8. **Recognise a bare variable identifier passed to `install.packages()` as unresolvable, rather than as a literal package name** (metacheck#421). A static extractor cannot generally know a variable's runtime value, so a call like `install.packages(pkg)` inside `for (pkg in list.packages) ...` should either be skipped entirely, or resolved only when the iterated variable (`list.packages` in this example) is itself assigned a literal character vector earlier in the same file — the case in every one of this run's eight confirmed instances of this defect (A.7.5).
9. **Give `repro_install_deps_docker()` the same CRAN Archive retry `repro_install_deps()` (the `sandbox = "process"` backend) already has.** Confirmed directly against the Docker backend's own source: `repro_install_deps_docker()`'s `via_archive` field is hard-coded `FALSE` for every install attempt, with an explicit comment noting the process backend's Archive-retry-on-failure logic "is not duplicated here." A package whose current CRAN release fails to install (a build error, an R-version incompatibility) but whose last-published Archive version would install cleanly is therefore never given that chance under `sandbox = "docker"` specifically — the safer, actually-isolated backend this project used for exactly that reason (A.7.1) — even though the less-isolated `sandbox = "process"` backend would have tried. Checking each of the 70 one-off failed packages directly against live CRAN and the CRAN Archive (`https://cran.r-project.org/src/contrib/Archive/<pkg>/`, without running any paper's code) quantifies the upper bound this fix could plausibly reach: 21 (30.0%) are on live CRAN today, so their failure was for some other reason entirely, not archiving; 24 (34.3%) have no trace on CRAN at all, live or archived (a GitHub-only package, a Bioconductor package this check did not look for in the right place, or a name this appendix could not otherwise verify); and **25 (35.7%) are absent from live CRAN but do have a version sitting in the Archive** — exactly the set an Archive retry could plausibly recover, though a successful Archive fetch is not a guarantee of a successful install (the archived version could itself fail to compile, or need a system library this image still lacks), so 25 is an upper bound on what this specific fix could help with, not a prediction of how many would actually then install.
10. **Extend the full-corpus, disagreement-by-disagreement qualitative review method of Appendices A.1–A.6 to `reproducibility_check`'s own output.** This appendix reports the rate and category of non-reproducibility across the corpus, but — unlike Appendices A.1–A.6's per-disagreement review of `repo_check`/`data_check`/`code_check` — does not individually verify a sample of the 1,829 errored or 447 skipped-input scripts against their own paper's repository the way, for instance, A.3's five-check validation individually read 100+ real files. A systematic sample of `runtime`-classified failures in particular (65.4% of all errors, and the category this appendix could say the least about, since it is Metacheck's own catch-all for "every other kind of R error") would likely

surface further mechanically fixable causes, in the same style A.1–A.6 already found for `repo_check`/`data_check`/`code_check`.

A.8 Three further reliability fixes to compressed-archive handling, and their effect on `data_archive`/`code_archived`/`code_download`

An eighth round of work returned to a structural limitation named repeatedly in earlier appendices but never itself directly fixed: `zip_peek()`'s inability to read a zip's contents without downloading it on Dryad and Figshare, both of which redirect a file's download URL to Amazon S3. Diagnosing and fixing this surfaced two further, closely related defects in the same download path, and reprocessing every paper the three fixes together could plausibly affect closed most, but not all, of the remaining `code_download` disagreement this comparison's own numbers above already reflect.

`zip_peek()` failing on S3-backed hosts (scienceverse/metacheck#424). `zip_peek()` reads a remote zip's table of contents with two requests: a `HEAD` to learn the file's total size, then a byte-range `GET` for the tail bytes holding the archive's central directory. Amazon S3 — the storage backend both Dryad's and Figshare's own download URLs redirect to — answers the size-only `HEAD` request with 403 but answers the byte-range `GET` normally (confirmed live: a `HEAD` against a real Dryad-hosted file returned 403, while a `Range: bytes=0-100 GET` against the identical URL returned 206 with the file's true total size in its own `content-range` header). Because `.http_range_tail()`, the function `zip_peek()` calls to fetch the tail bytes, only sends its range request once `HEAD`'s size is known, a `HEAD` failure meant the byte-range request was never sent at all — not just for Dryad and Figshare but, confirmed the same way, for Harvard's Dataverse installation, another host redirecting to S3. Fix: fall back to a suffix range request (`Range: bytes=-<n>`) when `HEAD` fails, reading the total size from that response's own `content-range` header instead. Confirmed live before and after against nine archives sampled from this corpus (three each on Dryad, Figshare, and Zenodo): all six Dryad/Figshare archives returned `NULL` from `zip_peek()` before the fix and a correct, complete listing after it, while the three Zenodo archives (not S3-backed, and so unaffected by the underlying bug) listed correctly both times.

`zip_peek()` never persisting its result across a restart (scienceverse/metacheck#427). A separate, longstanding defect (`repo_info_cache()`) already exists specifically to persist a repository's listing (which files it has) to disk, since restarting an interrupted corpus run otherwise re-pays every repository's listing cost from scratch — a real problem previously severe enough to exhaust Dryad's own daily request quota (see that fix's own code comment, confirmed live 2026-09-08: 11 restarts of one 50-paper batch cost 440+ re-listing requests against a workload that genuinely needed a handful). `zip_peek()`'s own result — a zip's contents, a separate cost from the repository listing above it — had no equivalent: it was cached only in memory, for the life of one R process, so the same restart pattern silently re-peeked every zip already peeked, and `repo_check` and `data_check` each peeked the same zip independently (as two separate R processes in this project's own pipeline), doubling the

cost again. Confirmed directly during this project's own corpus rerun (below): before this fix, four consecutive restarts each re-hit the same Dryad rate limit at the same early batch, at increasingly long waits (24.5, then 21.5, then 16 minutes remaining); after it, a restart at the same point in the run loaded every already-completed batch from disk in under 20 seconds and progressed past the previously-stuck batch with no rate-limit wait at all. Fix: persist `zip_peek()`'s result to disk the same way `repo_info_cache()` already does for a listing, gated on the same `cache = TRUE` argument.

A silently discarded error in `download_repo_files()`'s archive-member fetch (scienceverse/metacheck#429). Once `repo_check`'s zip-peek expansion lists a file living inside an archive (an "archive member," carrying `archive_url`+`archive_member` rather than its own `file_url`), `download_repo_files()` fetches it via a dedicated byte-range path (`.zip_fetch_members()`/`.zip_member_fetch()`), wrapped in a bare `tryCatch(..., error = function(e) NULL)` with no logging of any kind. A transient failure at this one step — a dropped connection, a timeout, a momentary host error — left `file_location` permanently `NA` with no warning, no log entry, and no record anywhere that a download had even been attempted, indistinguishable after the fact from a member the host genuinely cannot serve. This surfaced directly in this project's own corpus rerun: after #424 above let `repo_check`/`data_check` correctly list many more Dryad/Zenodo archive members than before, 24 papers whose code repository had previously never been detected at all (`mc_code_archived` `FALSE`) newly showed `mc_code_archived` `TRUE` but `mc_code_download` still `FALSE` — the code files were now correctly found, but never downloaded. Tracing one case in full (a small `.R` file inside a Dryad-hosted zip) confirmed the underlying fetch mechanism worked perfectly when called directly, with the paper's own exact saved row data and arguments, ruling out a logic bug; the corpus run's own 17,661-line warning log had no entry at all for this archive, this paper, or any of the other 23, confirming the failure really was silently discarded rather than merely unlogged elsewhere. Fix: record the specific error `.zip_fetch_members()` raises, the same way the ordinary `file_url` download path a few lines below already does, rather than discarding it.

Reprocessing the 24 code-download regressions. All 24 papers' `code_check` output was rerun against `dev` after #429's fix (`code_check` alone — `repo_check`'s listing and `data_check`'s own downloads were unaffected, confirmed directly: `mc_data_download` was already correctly `TRUE` for the papers where a data download was expected). 13 of 24 resolved immediately. The remaining 11 were rerun a second time with `skip_on_api_limit` set `FALSE` for this small, targeted rerun (unlike the corpus-wide default `TRUE`, whose own rationale — never blocking an unattended, hours-long, 1,861-paper run on a host's quota reset — does not apply to an 11-paper rerun, and, confirmed directly, was itself the reason these 11 still failed: their Dryad archive fetch had landed during a rate-limit window and been silently skipped rather than waited out, and waiting the quoted remaining time fetched the exact right file on the first attempt); 4 more resolved. The final 7 each printed a `download_repo_files()` retry-exhaustion message (`Failed to perform HTTP request`, `Re-run to retry: cached files are reused, only the missing files are fetched`) that — a `message()`/`cat()` call, not a `warning()` — this project's own rerun script's `warning =` handler could not catch or log, a message-routing gap distinct from #429's own fix and not itself corrected here; rerunning those 7 a third time, as

the message itself instructed, resolved all 7. Every one of the original 24 papers now downloads successfully.

Corpus-wide effect. With these three fixes installed and the full 1,861-paper corpus rerun, sensitivity, specificity, and the number of papers newly entering each comparison (a repository or file that previously could not be evaluated at all now can be) all moved together in the direction these fixes targeted. `code_archived` agreement rose from 84.5% to 87.5% (sensitivity 60.1% to 69.7%), `any_readme` from 69.8% to 72.2% (sensitivity 59.6% to 63.2%), and `data_availability` from 83.3% to 85.2%. `code_download`'s own comparable count grew from 342 to 395 papers (50 papers whose code could not previously be evaluated for a download outcome at all — because it had never been detected as archived in the first place — now can be), and at that point only 2 disagreements remained across that larger, harder set (down from 24 when these three fixes began). After a later targeted rerun of every paper with an archive member that had not been downloaded, this time waiting out host rate limits (see the `code_download` results in the main text), 1 `code_download` disagreement remains (99.7% agreement), a case where Metacheck is right and Cooper et al.'s coding missed the code. `zip_not_peeked` — the disagreement cause tracing directly to #424's own limitation — dropped to 0 as a recorded cause across every construct in this comparison's disagreement-cause categorization. The "compressed archive never opened" category among `data_archive`'s remaining under-coded papers (Appendix's own `data_archive_uncoded_papers.RData` analysis) fell from 70 papers to 12; the remainder now trace to distinct, already-documented causes (a code-only or documentation-only repository, a non-tabular file format `data_check` does not classify as data, or an image/GIS/3D file). Every previously-recorded disagreement in the corpus was re-reviewed the same way as every earlier round: 1,070 of 1,174 prior verdicts carried forward unmodified, because their underlying (paper, column, Cooper value, Metacheck value) triple was unchanged; every disagreement whose Metacheck-derived value changed, or which was newly surfaced, was reviewed from scratch (43 rows immediately after the corpus rerun above; 19 once the 24 papers' `mc_code_download` values were updated by the reprocessing described above and the comparison data rebuilt a second time) against the same evidence-source rule used throughout this manuscript (repository file listing and `repo_metadata` for `data_license`/`code_archived`/`any_readme`/`code_download`, the paper's own text for `data_availability`/`data_archive`). In the current worklist, all 890 disagreeing papers carry a verdict for every column on which they disagree.

A.9 A full, paper-by-paper review of every "unsupported repository, not detected" disagreement, and a full recheck of Appendix A.6's original categorization

Appendix A.6 categorized every "repository not detected" disagreement in the corpus, but 40 of the 164 papers whose only Cooper-coded `data_archive` label(s) fell outside Metacheck's supported-platform categories, and for which `repo_check` consequently found no repository at all, had not yet been individually checked one at a time when an earlier version of this manuscript's main text was written. This appendix documents that remaining review in full: each paper's own extracted data-availability text, the

specific DOI or URL cited, whether it was live-verified to resolve, whether the cited platform is genuinely supported by Metacheck (checked directly against the vendored source in `code/data/reproducibility_check/vendor/metacheck/R/`, not inferred from this manuscript's own earlier prose, which had itself gone stale for at least one platform – see below), and the final category assigned. All 40 papers received a verdict at the time of this review; 6 have since had their underlying `repo_check` defect fixed by later work elsewhere in this project and no longer appear in the live disagreement set at all (confirmed: `repo_check` now finds a repository for every one of them), so Table 2 below shows the 34 that remain.

This review also surfaced a further risk worth checking directly rather than assuming away: Appendix A.6's original 230-row categorization was itself produced before the platform-support defects this review found (the stale 4TU/DANS assumptions, an incomplete dedicated-platform list, a missing DSpace-legacy host count), so any of those original 230 rows that happened to cite one of these same platforms could be mislabeled the same way. Rather than only check the platforms these specific bugs touch, every one of the original 230 rows was rechecked from scratch: each paper's own text was re-extracted directly from the corpus (paragraph-grouped, not sentence-truncated, avoiding the extraction bug documented earlier in this manuscript), every DOI and URL in that text was identified, and each one's host was checked directly against Metacheck's current supported-platform functions – not against what an earlier comment claimed was or was not supported.

Of the 230, 21 candidate citations resolved to a host now confirmed supported despite the row's original `UNSUPPORTED_REPO` label; after excluding false positives (a truncated GitHub URL with no repository path, a citation that names a genuinely unrelated, unsupported host despite superficially matching a regex), 9 rows were corrected to `SUPPORTED_PLATFORM_BUG` or `SUPPORTED_PLATFORM_MISSED` at the time of this review. The corrected rows span three recurring patterns: a citation to a genuinely supported host that `repo_check` simply never queried (Dataverse, DataOne/KNB, Zenodo, standard-format Figshare), a supported platform cited via a URL format its matcher does not parse (a Figshare private share-link or collection URL), and a supported platform cited via a non-standard DOI prefix its prefix-matching does not recognise (an alternate Dryad EZID prefix, a non-standard Figshare prefix). None of the corrected rows involve a platform Metacheck genuinely cannot reach at all – every correction is a detection defect on an already-supported platform, consistent with the pattern the original 40-paper review above found. 8 of these 9 papers have since had their underlying defect fixed by later work elsewhere in this project (the same `repo_check`-now-finds-it pattern as above) and no longer appear in the live disagreement set; Table 3 below shows the 1 still remaining in it, with the full original set described narratively here regardless.

Table 5: Table 3. The rows from Appendix A.6's original 230-paper categorization corrected on full recheck, from `UNSUPPORTED_REPO` to a supported-platform category, still present in the current disagreement set.

Paper	Category	Evidence and verification
10_1111_1365_2435_14709	SUPPORTED_PLAT-FORM_BUG	Same root cause: the private Figshare share-link format was not recognized by repo_check. RECHECKED: figshare.com is a fully supported platform; the defect is specifically that a private/anonymous Figshare share-link (figshare.com/s/...) uses a different URL format from the standard figshare.com/articles/... pattern repo_check's Figshare matcher expects, so the host itself is supported but this specific link format is not recognised -- a supported-platform bug, not an unsupported platform.

Two platforms cited in this set turned out to be genuinely supported by Metacheck despite `repo_check` finding nothing, each traced to a specific, distinct defect rather than a missing backend. **4TU** (`data.4tu.nl`) has dedicated support (`archive-4tu.R`), and the DOI cited (`10.4121/uuid:04b25e70-ca3e-489f-9510-36385c1a4104`) live-resolves correctly to a real 4TU record. The ID-extractor correctly pulls the UUID from the DOI, but the live 4TU API lookup queries `data.4tu.nl/v2/articles/<uuid>`, which 404s, because 4TU's own DOI redirect resolves the identical record to a different, numeric article ID that the UUID does not match. This corrects an earlier, incorrect statement elsewhere in this manuscript (Appendix A.6) that this paper's 4TU fetch failure remained unresolved and should count as `UNSUPPORTED_REPO`; it is a supported-platform bug, not an unsupported platform. **DANS EASY** is one of four separate subject-specific Dataverse installations DANS operates (archaeology/lifesciences/phys-techsciences/ssh), all sharing a single DOI prefix (`10.17026`). Metacheck's `.dataverse_host_from_doi()` resolves any DOI under this shared prefix to `archaeology.datastations.nl`, but the paper in this set cites a dataset actually hosted on `lifesciences.datastations.nl` – a genuine Metacheck defect in DOI-to-host resolution for any DOI prefix shared across multiple Dataverse instances, not specific to DANS. A third paper cites the Knowledge Network for Biocomplexity (`knb.ecoinformatics.org`), a confirmed DataOne member node whose DOI live-resolves correctly; the specific reason `repo_check` did not detect it was not further root-caused within the scope of this review, so it is recorded as a supported-platform miss without a fully diagnosed cause, distinct from the two fully diagnosed bugs above.

Table 6: Table 2. Full paper-by-paper review of the 40 previously-unreviewed 'unsupported repository, not detected' disagreements: cited platform, live DOI/URL resolution status, and final category.

Paper	Category	Evidence and verification
10_1002_pan3_10149	NO_REPO_DATA_RESTRICTED	Paper text: 'Our raw data are not publicly available for confidentiality reasons, as they contain social, cultural and language data that may not be interpreted outside of their Nyikina context. They will be archived with Griffith University and Walalakoo Aboriginal Corporation.' No repository URL or DOI is given anywhere in the paper -- data are explicitly withheld (Indigenous data sovereignty). There is nothing for repo_check to find; this is not a Metacheck gap of any kind.
10_1111_1365_2745_12930	UNSUPPORTED_REPO	Paper text: 'Data associated with this paper have been deposited in the Environmental Climate Data Sweden depository https://ecds.se/dataset/species-distribution-modelling-data-for-23-dead-wood-inhabiting-bryophytes'. Live-verified the URL resolves (redirects to researchdata.se). researchdata.se/ecds.se is not in any of Metacheck's supported host lists (not Dataverse, DSpace7, DataOne, Figshare, and has no dedicated single-platform module).
10_1002_pan3_10307	UNSUPPORTED_REPO	Paper text: 'Data and code used in the analyses are available at ScienceBase (https://doi.org/10.5066/P9KBZW0G and https://doi.org/10.5066/P9CDA1UN, respectively)'. Live-verified both DOIs resolve to www.sciencebase.gov catalog items. ScienceBase (USGS) is not in any Metacheck supported host list and has no dedicated module.
10_1111_1365_2745_13067	TEXT_EXTRACTION_TRUNCATED	Extracted text is a mid-sentence fragment: '...net online repository (Lopez-Gonzalez, Lewis, Burkitt, & Phillips, 2011; ...)'. No DOI or URL is present in the extracted text at all -- the actual data-availability statement (likely referring to ForestPlots.net, based on the citation) was not captured by GROBID extraction.
10_1111_1365_2435_14226	UNSUPPORTED_REPO	Paper text: 'The data that support the findings of this study are available on the National Tibetan Plateau Data Center https://doi.org/10.11888/Terre.tpdc.272843'. Live check: this DOI request failed to resolve (network/redirect issue querying data.tpdc.ac.cn from outside China is common). Regardless, tpdc.ac.cn is not in any Metacheck supported host list and has no dedicated module.
10_1111_1365_2745_14315	UNSUPPORTED_REPO	Paper text: 'Novel data collected in this study and compiled from FRED are available through FRED: https://doi.org/10.25581/ornlsfa.014/1459186'. FRED (via ORNL's Sustainable Forestry/SFA data archive) is a niche, domain-specific archive with no entry in any Metacheck supported host list and no dedicated module. Live DOI resolution failed (ornlsfa.014 DOI prefix appears not to resolve through doi.org currently), which is a separate, secondary problem on top of the platform simply being unsupported.
10_1111_2041_210x_14453	UNSUPPORTED_REPO	Paper text: ''FishScale' software ... can be accessed at https://doi.org/10.5066/P13QVR2R. Data available via https://doi.org/10.5066/P9E1DTYB'. Live-verified: the code DOI resolves to code.usgs.gov (a self-hosted GitLab instance on USGS's own domain -- explicitly out of scope per Metacheck's own GitLab module documentation, which matches only gitlab.com); the data DOI resolves to www.sciencebase.gov (unsupported, as elsewhere in this set). Both citations point to unsupported hosts.
10_1111_1365_2745_13529	NCBI_GENBANK	Paper text: 'Data were deposited in the Sequence Read Archive (https://www.ncbi.nlm.nih.gov/bioproject/PRJNA592420)' and 'Sequence data are deposited at NCBI-SRA under the accession number PRJNA592420.' A domain-specific bioinformatics accession (SRA/BioProject), a distinct category from an ordinary data/code repository and outside repo_check's scope by design.

Paper	Category	Evidence and verification
10_1002_pan3_10743	TEXT_EXTRAC-TION_TRUNCATED	Extracted text is entirely a philosophical/conceptual discussion of aesthetics and definitions of 'mess' -- no data-availability statement was captured anywhere in the extracted text for this paper.
10_1111_1365_2664_12991	TEXT_EXTRAC-TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1002_2688_8319_12241	TEXT_EXTRAC-TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2745_13517	TEXT_EXTRAC-TION_TRUNCATED	Extracted text covers only methodological discussion of pollen/charcoal source areas -- no data-availability statement with a URL or DOI was captured anywhere in the extracted text for this paper.
10_1002_pan3_10677	TEXT_EXTRAC-TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2745_13154	UNSUPPORTED_REPO	Paper text: 'Data available through Digital Commons @ Georgia Southern repository https://doi.org/10.20429/data.2019.01'. Live-verified the DOI resolves (HTTP 200) to digitalcommons.georgiasouthern.edu. A generic bepress Digital Commons instance, not in any Metacheck supported host list.
10_1111_1365_2745_12749	TEXT_EXTRAC-TION_TRUNCATED	Paper text ends mid-sentence: 'Data, including location, growth, and iWUE data are located at ScholarsArchive@OSU' -- the sentence is cut off before any URL or DOI appears, so no identifier was captured for this citation despite a named repository being mentioned.
10_1111_1365_2435_14477	TEXT_EXTRAC-TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2664_13892	UNSUPPORTED_REPO	Paper text: 'The entire dataset that supports the findings of this study is available via a permanent open access data repository of the University of Vienna http://phaidra.univie.ac.at/o:1168756'. Live-verified the URL resolves (HTTP 200). Phaidra (University of Vienna's repository platform) is not in any Metacheck supported host list and has no dedicated module.
10_1111_2041_210x_13110	TEXT_EXTRAC-TION_TRUNCATED	Extracted text covers only general methodological discussion of uncertainty modelling -- no data-availability statement with a URL or DOI was captured anywhere in the extracted text for this paper.
10_1111_1365_2745_14252	TEXT_EXTRAC-TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2664_14700	UNSUPPORTED_REPO	Paper text: 'Data are available from the OFB Digital Repository https://ged.ofb.fr/share/s/XiFQtejFQy28DG6u0TDzoA/folder'. Live-verified the URL returns HTTP 403 (access-restricted share link) on a custom OFB (French biodiversity agency) Nextcloud-style file-share instance. Not in any Metacheck supported host list and has no dedicated module; a bare access-controlled folder link is also not the kind of structured repository API Metacheck's dedicated modules are built to query even where a host is supported.

Paper	Category	Evidence and verification
10_1111_1365_2656_13918	UNSUPPORTED_REPO	Paper text cites four EDI/PASTA data package DOIs (10.6073/PASTA/... and 10.6073/pasta/..., resolving to portal.edirepository.org) for the paper's actual data, plus a properly-formatted Zenodo DOI (10.5281/zenodo.7737746, live-verified HTTP 200) for the analysis code only. EDI/PASTA is not in any Metacheck supported host list and has no dedicated module. Because the data_availability comparison column concerns the data (not the code), and the data are exclusively on EDI/PASTA, this paper is classified as citing an unsupported repository for its data; the Zenodo code citation is correctly formatted and would be expected to be found by Metacheck's code_check separately.
10_1111_2041_210x_13463	UNSUPPORTED_REPO	Paper text: 'The data associated with this manuscript are published on ESS-DIVE, the Department of Energy's data portal...'. No specific DOI is given in the extracted text (the paper states DOIs are 'documented in Supporting Information file 4', not in the main text). ESS-DIVE is not in any Metacheck supported host list and has no dedicated module; Google Earth Engine (also mentioned) is not a structured data repository Metacheck can query as such.
10_1111_1365_2435_13550	NCBI_GENBANK	Paper text: 'The sequences were deposited in NCBI SRA database (accession no. ... SRP218212 and ... SRP068522)'. A domain-specific bioinformatics accession (SRA), outside repo_check's scope by design, consistent with other SRA/GenBank/BioProject cases in this set.
10_1111_1365_2435_12874	TEXT_EXTRAC-TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2664_14787	UNSUPPORTED_REPO	Paper text: 'Data available from the CSIC repository: http://hdl.handle.net/10261/367361'. Live-verified the URL resolves (HTTP 200) to digital.csic.es. A DSpace-based institutional repository, but not in Metacheck's DSpace7 or DSpace-legacy host lists and has no dedicated module.
10_1111_1365_2656_14227	UNSUPPORTED_REPO	Paper text cites two repositories: 'Guillemot demographic data ... freely available from Heriot Watt research portal https://doi.org/10.17861/8fd6a194-d1a7-469c-8604-e45b0414f8f0' (live-verified HTTP 200, researchportal.hw.ac.uk) and 'Puffin and razorbill demographic data ... available upon request ... from the University of Gloucester online repository https://doi.org/10.46289/8FP68KK9' (live-verified HTTP 404, likely a stale/incorrect DOI). Neither host is in any Metacheck supported host list, and neither has a dedicated module.
10_1111_1365_2745_13324	UNSUPPORTED_REPO	Paper text: 'Data used in this publication are available in the appendices and archived at the USU Digital Commons: https://doi.org/10.26078/99w7-eb58'. Live-verified the DOI resolves (HTTP 200) to digitalcommons.usu.edu. A generic bepress Digital Commons instance, not in any Metacheck supported host list.
10_1111_1365_2656_13272	UNSUPPORTED_REPO	Reference-list citation: 'ScholarWorks Data Repository, https://doi.org/10.18122/bio_data/3/boisestate'. Live-verified the DOI resolves (HTTP 200) to scholarworks.boisestate.edu. A generic bepress ScholarWorks instance, not in any Metacheck supported host list.
10_1111_1365_2435_13791	TEXT_EXTRAC-TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2745_13849	UNSUPPORTED_REPO	Paper text: 'The data are available as a data package on https://www.forestplots.net/: https://doi.org/10.5521/forestplots.net/2022_1'. Live-verified the DOI resolves (HTTP 200) to forestplots.net. Forestplots.net is a domain-specific forest-inventory data archive, not in any Metacheck supported host list and has no dedicated module (a separate, unrelated Dryad DOI also appears in this paper's reference list, but only as a citation for a prior, different dataset -- not this paper's own data).

Paper	Category	Evidence and verification
10_1111_1365_2435_14514	UNSUPPORTED_REPO	Paper text: 'The data that support the findings of this study are available from UNL data archive: https://doi.org/10.32873/unl.dr.20230705'. Live-verified the DOI resolves (HTTP 202) to sandy.unl.edu. Not in any Metacheck supported host list and has no dedicated module.
10_1111_1365_2435_13936	UNSUPPORTED_REPO	Paper text: 'Data are available from the Illinois Data Bank https://doi.org/10.13012/B2IDB-4547091_V1'. Live-verified the DOI resolves (HTTP 403, access-gated redirect) to databank.illinois.edu. A Hydra/Fedora-based institutional repository, not in any Metacheck supported host list.
10_1111_1365_2664_13845	UNSUPPORTED_REPO	Paper text: 'Data are available in the US Department of the Interior and U.S. Geological Survey's ScienceBase https://doi.org/10.5066/P94B0WUV'. Live-verified the DOI resolves (HTTP 403) to www.sciencebase.gov. ScienceBase is not in any Metacheck supported host list, consistent with the other ScienceBase cases in this set.
10_1111_1365_2664_14229	SOURCE_CITATION_ERROR	Paper text: 'Data available via Unimi Dataverse http://10.0.51.74/RD_UNIMI/5ZXGIV'. Unimi's real Dataverse host (dataverse.unimi.it) IS a supported Dataverse installation (matches .dataverse_host_regex()), but this paper cites a raw internal/private IP address (10.0.51.74, a non-routable RFC 1918 address) instead of the public hostname -- live-verified this address does not resolve externally at all (request failed outright). The paper's own published citation is unusable as given; this is an error in the source text, not a Metacheck platform-support gap or extraction artefact.

Of these 34 papers, 18 genuinely cite an unsupported platform, 2 cite a domain-specific bioinformatics accession outside `repo_check`'s scope, 0 cite a platform Metacheck genuinely supports but failed to detect for the reasons described above, 12 had no usable citation captured in the paper's extracted text at all (either empty extraction or a citation cut off before any identifier appeared), 1 explicitly withheld their data with no repository to find, and 1 paper's own published citation pointed to an internal, non-public address rather than a reachable repository. These per-paper counts feed directly into the corpus-wide reconciling numbers reported in the main text and into Table 1's totals above.

A.10 Full paper-by-paper detail for every technical limitation identified in the full disagreement-population review

Table 4 lists every one of the 62 papers in the main text's "technical limitation" cluster individually: its DOI, which of the six specific limitations (grouped into text extraction and Metacheck's own code, as in the main text) applies, and the evidence-based reasoning for that classification, generated directly from the same per-paper audit file (`manual_repo_findability_audit.csv`) the main text's numbers are computed from. Because the table is built programmatically from that file rather than transcribed by hand, it stays reconciled with the main text's counts automatically on every re-render (confirmed by the chunk above: the table's row count is checked against `n_metacheck_limitation` directly).

Table 7: Table 4. Full paper-by-paper detail for every technical limitation identified in the full disagreement-population review.

Paper (DOI)	Category	Why Metacheck misses it
10.1002/pan3.10185	Text extraction: identifier corrupted by the PDF-to-text step	metacheck fully supports this exact platform and host: rdr.ucl.ac.uk is explicitly hardcoded in R/archive-figshare.R's .figshare_vanity_hosts() (line 88, labeled "University College London (UK)"), the figshare.com DOI prefix 10.5522 is not even needed here because the underlying numeric article id (13469022) appears directly, and .figshare_id()'s regex patterns are written to match exactly this "10.6084/m9.figshare.N" or vanity-host "/articles/.../N" shape. The reason metacheck_any_detector_hit is false is almost certainly that the GROBID-extracted full_text corrupted all four DOIs identically: every occurrence reads "10.5522/04/13469 022.v1" (and 13469 085, 13469 139, 13469 166) with a literal space inserted between the 5th and 6th digit of the article id, splitting what should be one contiguous numeral "13469022" into "13469" + " " + "022". figshare_links()'s bare-text regex fallback (and .figshare_id()'s digit-capture groups) require an unbroken run of digits ([0-9]+); a stray embedded space breaks that run, so the regex simply does not match this citation as extracted, even though a human reader (and the live DOI resolver) has no trouble with it. This is a text-extraction artifact (GROBID/PDF-to-text splitting digit groups, the same class of bug already fixed elsewhere for effect-size subscripts per the recent commit history on this branch) rather than any gap in metacheck's Figshare/UCL-RDR platform support, which is otherwise a fully correct, already-registered configuration.
10.1002/pan3.10623	Text extraction: identifier corrupted by the PDF-to-text step	Paper states 'All data used in the analysis can be found freely online on Figshare,' but the adjoining reference-list DOI is corrupted by PDF column-reflow (an unrelated reference interleaved mid-title; no Figshare DOI/URL survives extraction). Independently located the real dataset via Figshare's search API: DOI 10.6084/m9.figshare.25066136.v1, verified live (is_public TRUE, MIT license, 11 files). Metacheck's archive-figshare.R fully supports resolving this DOI via the documented api.figshare.com/v2 REST API -- it never had a chance to act because no identifier survived extraction.

Paper (DOI)	Category	Why Metacheck misses it
10.1002/pan3.10689	Text extraction: identifier corrupted by the PDF-to-text step	RECLASSIFIED 2026-10-13 (EXTRACTION_GAP_DOI_GARBLED, was SUPPORTED_PLATFORM_CONFIG_GAP): The host (VTechData) is registered in metacheck, but the DOI in the extracted text is broken by spaces ('10.7294/ 25914 451. v2'): a text-extraction problem. Earlier explanation: metacheck has a full, working Figshare backend (R/archive-figshare.R) that correctly resolves articles via api.figshare.com, including dozens of institutional "vanity" Figshare instances and their own DOI prefixes (.figshare_vanity_hosts() and .figshare_doi_prefix_hosts()). But this paper's data link is https://doi.org/10.7294/25914451.v2, hosted at VTechData (data.lib.vt.edu) under DOI prefix 10.7294. Neither the host "data.lib.vt.edu" nor the DOI prefix "10.7294" appears in .figshare_vanity_hosts() or .figshare_doi_prefix_hosts(), and the bare-DOI fallback regex in figshare_links()/.figshare_id() only recognizes the literal 10.6084 prefix plus whatever prefixes are registered in that list -- so a bare citation of "10.7294/25914451.v2" (as this paper's availability statement uses, with no "figshare.com" or "data.lib.vt.edu" string anywhere near the DOI) is invisible to figshare_links(), and the resolved redirect host data.lib.vt.edu is also absent from the host regex, so even a real hyperlink to the landing page would not match either. I independently confirmed live via api.figshare.com/v2/articles/25914451 that this is genuinely the same Figshare API backend already supported (is_public: true, matching title, files PAN_MSV_Dataset.xlsx/Survey Instrument.pdf/README.rtf, license CC0 1.0) -- so this is a missing host/DOI-prefix registration for an existing, working platform integration, not a missing platform and not a broken or inaccessible repository. I also note archive-dryad.R's own comments record that DOI prefix 10.7294 (VTechData) was previously checked and confirmed NOT to be Dryad, but it was apparently never cross-registered as Figshare despite that check. Fix: add "10.7294" -> "data.lib.vt.edu" to .figshare_doi_prefix_hosts() and "data.lib.vt.edu" to .figshare_vanity_hosts(). CORRECTION 2026-10-07: a live rerun through the actual installed package (after confirming .figshare_doi_prefix_hosts()/.figshare_vanity_hosts() do now list 10.7294/data.lib.vt.edu via a newer .figshare_doi_lookup() API fallback added in commit 80a7d37e) still produced repo_n = 0, files_n = 0 for this paper -- the registry fix does not resolve it in practice. Tracing why: this paper's own extracted text has the DOI itself whitespace-garbled by GROBID ('https://doi. org/ 10.7294/ 25914 451. v2', with a spurious space inside the numeric id), a DIFFERENT and unrelated defect from the host-registry gap this row originally described. The direct-function-call test that appeared to confirm this fix used a clean, manually-reconstructed DOI string, not this paper's actual corrupted extracted text, so it never actually tested the real failure path. Root cause is EXTRACTION_GAP_DOI_GARBLED-shaped, layered on top of what is otherwise a genuinely-fixed config gap -- the config fix alone is not sufficient. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: The host (data.lib.vt.edu / 10.7294, VTechData) is now registered in metacheck, but the paper is still not found: the DOI in the extracted text is broken by spaces ('10.7294/ 25914 451. v2'), so the fix needed is in reading the text, not in the host list.]

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.14477	Text extraction: identifier corrupted by the PDF-to-text step	metacheck already fully supports this exact host and DOI prefix: R/archive-dataverse.R's .dataverse_hosts() explicitly lists "phys-techsciences.datastations.nl" (one of the four DANS Data Stations, which are Dataverse installations, not a generic institutional repository as the coder's label "DANS (Data Archiving and Networked Services, Netherlands)" implies), and .dataverse_doi_prefix_hosts() explicitly maps that host to the "10.17026" DOI prefix used by all DANS Data Stations. So this is not a platform gap or a config gap -- the detector's host/prefix registry already covers it. The reason metacheck's detector did not fire (metacheck_any_detector_hit = false) is that the GROBID/PDF text extraction garbled the one and only DOI in the paper: it appears as "https://doi. org/ 10.17026/ dans-zqs-g6cn" with spurious internal whitespace splitting "doi.org/", "10.17026/", and "dans-zqs-g6cn" into separate tokens, inside a reference-list citation rather than in the data-availability sentence itself (which is truncated mid-word and has no link at all: "...are stored in DANS (Data archiving and networked..."). A regex expecting a contiguous "10.17026/dans-zqs-g6cn" or "doi.org/10.17026/..." string would not match this whitespace-split form, so the extraction/garbling is the actual failure point, not any lack of platform or host support.
10.1111/1365-2435.14696	Text extraction: identifier corrupted by the PDF-to-text step	Metacheck fully supports Zenodo via a dedicated R/archive-zenodo.R module, which explicitly anticipates bare "10.5281/zenodo.NNNNNNN" citations without a URL scheme (see the module's own comment: "10.5281/zenodo.1234567 is routinely cited without any URL scheme at all"), and this paper's DOI (10.5281/zenodo.13986711) is exactly that standard Zenodo prefix pattern, resolving live to a public CC-BY-4.0 Zenodo record (zenodo.org/records/13986711) with a CSV dataset and README. So this is not a platform-support gap. The failure is that the in-text data-availability sentence itself is truncated by the PDF text extraction right after "Data reported in the manuscript are available from the Zenodo" -- the DOI never appears in that sentence at all in full_text. The DOI does appear later, in the separate bibliography entry for Lu, Wenglein, Bluhm, Stuckenberg, Potapov & Scheu (2024), but there it is corrupted by stray whitespace injected during extraction: "https://doi. org/ 10.5281/ ZENODO.13986711" (spaces after "doi.", after "org/", and after "10.5281/"). A DOI/URL regex that requires a contiguous "doi.org/10.5281/zenodo.NNNNNNN" string without internal whitespace tolerance would fail to match this specific occurrence, even though the Zenodo backend itself would handle the ID correctly once given a clean DOI. This is best classified as a garbled-DOI extraction gap rather than a missing platform or a genuinely absent link.
10.1111/1365-2656.13345	Text extraction: identifier corrupted by the PDF-to-text step	Metacheck fully supports Dryad via R/archive-dryad.R, whose dryad_links() function matches bare Dryad DOIs with the regex "(?:https?://)?(?:doi\ .org/)?10\ .5061/dryad\ .[A-Za-z0-9]+" (and .dryad_doi() re-applies the equivalent capturing pattern "(10\ .5061/dryad\ .[A-Za-z0-9]+)"). In this paper's extracted full_text, the DOI is corrupted by what looks like a PDF line-wrap or column-break artifact that inserted a literal space inside the Dryad suffix: "https://doi.org/10.5061/dryad.hmgqn k9f3" in the data-availability statement, and again as "https://doi.org/10.5061/ dryad.hmgqn k9f3" in the reference list (La Sorte & Graham, 2020 citation). Because "[A-Za-z0-9]+" cannot match across a whitespace character, the regex engine stops at "hmgqn" and either extracts the truncated/incorrect DOI "10.5061/dryad.hmgqn" or otherwise fails to recover the real, correct suffix "hmgqnk9f3". The live repository itself is a completely ordinary, fully supported, publicly accessible Dryad dataset with files present -- this is not a platform gap or a configuration gap, but a text-extraction corruption of the DOI string that defeats the existing, otherwise-correct Dryad regex.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.13893	Text extraction: identifier corrupted by the PDF-to-text step	metacheck fully supports Dryad (R/archive-dryad.R), and its dryad_links() regex (10\ .5061/dryad\ .[A-Za-z0-9]+, plus a doi\ .org/ prefix variant) would ordinarily match this exact DOI (10.5061/dryad.7d7wm3806, under the explicitly allow-listed 10.5061 prefix) without any config change needed. The reason metacheck_any_detector_hit is false is that the PDF-extracted full_text corrupts the DOI at both of its two occurrences with stray whitespace introduced during text extraction, not because the platform or DOI prefix is unsupported. In the data-availability statement, the text reads \ "https://doi.org/10.5061/dryad.7d7wm 3806\ ", with a space inserted between \ "7d7wm\ " and \ "3806\ " that splits the DOI suffix into two tokens; since the regex's [A-Za-z0-9]+ character class does not match whitespace, it would at most capture the truncated \ "dryad.7d7wm\ " and miss the real dataset DOI \ "dryad.7d7wm3806\ ". In the reference-list citation (Riecke, Hegelbach, & Schaub, 2023), the second occurrence instead has the space between \ "doi.\ " and \ "org\ " (\ "https://doi. org/10.5061/dryad.7d7wm3806\ "), which breaks the literal \ "doi\ \ .org\ " substring the regex's doi.org/-prefix alternative requires, though the bare \ "10.5061/dryad\ " alternative in the pattern could still match this one correctly since the DOI digits themselves are intact there. Because the live dataset is confirmed real, public, and directly reachable at exactly the DOI the human coder recorded, this is a text-extraction/OCR-layer corruption gap specific to this PDF's rendering of the DOI (likely a kerning or line-wrap artifact in the original Wiley PDF), not a platform-support gap, a configuration gap, or a detector logic bug against clean input.
10.1111/1365-2656.14000	Text extraction: identifier corrupted by the PDF-to-text step	metacheck fully supports this exact case at the code level: R/archive-figshare.R already registers "10.26180" => "bridges.monash.edu" in .figshare_doi_prefix_hosts(), and .figshare_id() (lines ~307-344) carries a comment explicitly citing this very paper's DOI as a confirmed-working worked example ("confirmed live 2026-09-19: https://doi.org/10.26180/19095317.v1 resolves to Monash article id 19095317"). The failure is not in the detector's logic but in the upstream PDF text extraction: in full_text, the DOI string is rendered with two stray internal spaces -- "https://doi.org/10.26180/ 19095 317.v1" -- splitting the contiguous digit string "19095317" into "19095" and "317" (apparently a line/column-wrap artifact from the two-column PDF layout, identical in both the reference-list occurrence and the implied data-availability-statement occurrence). The very first scanning stage that finds Figshare candidate strings, figshare_links()'s fs_bare_regex (line 220 of archive-figshare.R), requires the DOI suffix to match "[A-Za-z0-9._-]+" contiguously with no whitespace, so this regex never matches the garbled string and no candidate URL is even extracted for .figshare_id() to later resolve. The coder's own recorded repo_link_verbatim has the identical "19095 317" space-split, confirming this is a property of the underlying text extraction itself, not an error either the coder or I introduced. The platform (Figshare via Monash Bridges), its DOI-prefix registration, and its id-resolution regex are all already correctly built and tested for the clean form of this exact DOI; only the whitespace-corrupted rendering of this one citation defeats the first-stage text-search regex, which is a text-extraction/garbling problem, not a missing or buggy detector.
10.1111/1365-2656.14054	Text extraction: identifier corrupted by the PDF-to-text step	Code citation appears only in the reference list (no DAS exists): 'Amarasekare, P. (2024)... Zenodo https://doi. org/ 10.5281/ ze- nodo.10497888' -- PDF extraction inserted spaces after 'doi.', after 'org/', after '10.5281/', and hyphen-broke the word 'zenodo' itself. Metacheck's zenodo_links() regex (archive-zenodo.R) requires a contiguous string and cannot match text containing embedded spaces/hyphen-breaks. Live-verified the Zenodo record resolves and is open, CC-BY-4.0, linking to the companion GitHub repo.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.13890	Text extraction: identifier corrupted by the PDF-to-text step	metacheck_any_detector_hit was reported as false, but this looks like it should actually be detectable as a dspace/dspace7 case, so I lean toward SUPPORTED_PLATFORM_BUG/CONFIG_GAP rather than a clean no-gap -- see caveat below. The repository DOI 10.20350/digitalCSIC/13825 resolves to digital.csic.es/handle/10261/236117, and the "/handle/<naan>/<suffix>" path is the canonical DSpace resolver URL pattern, confirming DIGITAL.CSIC is a generic DSpace installation (the coder's platform label "DSpace-based" is correct and matches the URL evidence, independent of the live Anubis block). Since metacheck's supported-platform list explicitly includes "dspace/dspace7 (generic DSpace installations)," the platform family itself is supported. The DOI is also present, unambiguously, in the paper's reference list, though the raw PDF-extracted text has it corrupted by stray inserted spaces ("https://doi.org/10.20350/ digit alCSI C/13825" instead of ".../digitalCSIC/13825") -- a GROBID-style extraction artifact that could defeat a naive strict-pattern link extractor looking for an unbroken "digitalCSIC" token or path segment, even though the DOI prefix "10.20350" and the surrounding citation context are intact and a human (or a more tolerant regex/whitespace-stripping extractor) would still recognize it. Given metacheck_any_detector_hit=false, the most defensible classification is that this is a DOI-level extraction problem (stray whitespace inside the handle/suffix string) rather than a missing-platform gap, so I classify it as EXTRACTION_GAP_DOI_GARBLED: the link is present in the text but corrupted by split/inserted whitespace in a way that likely prevents metacheck's extractor from recognizing it as the clean "10.20350/digitalCSIC/13825" DOI string, even though the DSpace/DIGITAL.CSIC platform itself is on metacheck's supported list and would otherwise be detectable.
10.1111/1365-2664.13961	Text extraction: identifier corrupted by the PDF-to-text step	metacheck has a dedicated, well-built Dryad detector (R/archive-dryad.R) whose regex matches datadryad.org/dataset(s)/doi: URLs as well as bare "10.5061/dryad.XXXXX" DOIs with or without a URL scheme, so the platform itself is fully supported and this exact DOI shape would normally be caught. The reason metacheck_any_detector_hit is false here is that the real Dryad link is corrupted at both places it appears in the extracted full_text: (1) in the data-availability statement, the URL token has been replaced by the author's ORCID link (https://orcid.org/0000-0002-9304-228X) -- almost certainly a PDF-to-text extraction artifact where a hyperlinked DOI field was resolved to the wrong embedded link target; and (2) in the reference-list citation, the DOI itself is printed one character short as "10.5061/dryad.0pngf20h" (missing the digit "2"), which I verified live returns HTTP 404 rather than resolving to the dataset. Only the correct, non-garbled DOI "10.5061/dryad.0p2ngf20h" -- which does not appear anywhere in the extracted text -- actually resolves to the real public dataset. Since neither occurrence in the text metacheck would see contains a matchable, correct Dryad identifier, no amount of improving the Dryad detector logic would fix this particular case; the gap is upstream, in the source PDF/text extraction corrupting the DOI before metacheck ever sees it.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.14229	Text extraction: identifier corrupted by the PDF-to-text step	metacheck already fully supports this exact repository: dataverse.unimi.it is explicitly listed in .dataverse_hosts() (R/archive-dataverse.R, line 137, "Università degli Studi di Milano (Italy)"), and its DOI prefix 10.13130 is explicitly mapped to it in .dataverse_doi_prefix_hosts() (line 314: "dataverse.unimi.it" = "10.13130"), individually verified against DataCite per that function's header comment. This is a correctly-configured, supported Dataverse installation with no gap in metacheck's platform or host/prefix registry. The failure is purely in the paper's extracted text: the data-availability sentence gives the link as "http://10.0.51.74/RD_UNIMI/5ZXGIV" — the real DOI resolver domain was corrupted into what looks like an internal/private IPv4 address during PDF extraction, so it contains neither the "dataverse.unimi.it" hostname string (which dataverse_links()'s host-regex path requires) nor the "10.13130" DOI prefix (which the bare-DOI-prefix regex path requires). The second occurrence, in the reference list ("https://doi.org/10.13130/ RD_UNIMI/ 5ZXGIV"), does carry the correct, verified 10.13130 prefix, but PDF line-wrap extraction inserted literal spaces after each forward slash, splitting the DOI into three fragments ("10.13130/", "RD_UNIMI/", "5ZXGIV"). metacheck's DOI-prefix regex (an unbroken "[A-Za-z0-9/._-]+" after the prefix) cannot match across that whitespace, so this occurrence also fails to resolve. Both failures are text-extraction corruption of the same underlying, already-registered link, not a gap in platform coverage or host/prefix configuration -- this is a GROBID/PDF-extraction defect (domain substitution in one place, inserted whitespace in another) rather than anything metacheck's Dataverse module needs to be extended to handle.
10.1111/1365-2745.13085	Text extraction: identifier corrupted by the PDF-to-text step	RECLASSIFIED 2026-10-13 (EXTRACTION_GAP_DOI_GARBLED, was SUPPORTED_PLATFORM_BUG): The PDF-to-text step inserted an artefact (file://localhost/opt/grobid/...) into the middle of the Dryad DOI (10.5061/dryad.745g830), so only a truncated DOI reaches metacheck: a text-extraction problem. Earlier explanation: metacheck fully supports Dryad (R/archive-dryad.R), and its bare-DOI regex `10\ .5061/dryad\ .[A-Za-z0-9]+` (dryad_links(), line 78) is written correctly and would match the text as printed, extracting "10.5061/dryad.745" as a Dryad hit -- which is why metacheck_any_detector_hit is true and the sanity check is not a clean pass. The real problem lies upstream of the regex, in GROBID's PDF text extraction: the paper's actual DOI is 10.5061/dryad.745g830, but GROBID inserted a spurious internal artifact, "file://localhost/opt/grobid/grobid-home/tmp/g830", directly between "745" and the true suffix "g830" in the extracted full_text, so the DOI appears split into "10.5061/dryad.745" followed by an unrelated local file path that happens to end in the string "g830" that truly belongs to the DOI. metacheck's regex, which has no way to know a PDF-extraction artifact sits in the middle of a DOI, correctly matches only the truncated "10.5061/dryad.745" -- which independently resolves to an HTTP 404, i.e. a dead DOI. So metacheck's detector registers a syntactic hit, but on a non-resolving link, not on the real, live, public dataset (10.5061/dryad.745g830) that a manual check finds. This is a text-extraction/GROBID artifact bug feeding a correctly-implemented Dryad detector, not a missing platform or missing regex case, and the human coder independently transcribed the exact same garbled DOI from the same corrupted source text, so this is not an extraction-pipeline issue unique to metacheck's own parsing either -- it is inherited directly from the GROBID output both relied on.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.13577	Text extraction: identifier corrupted by the PDF-to-text step	Dryad is a platform metacheck explicitly supports, and the dataset is genuinely live and public (confirmed by following the DOI redirect to datadryad.org). The reason metacheck's detector did not fire (metacheck_any_detector_hit: false) is that in the extracted full_text, every occurrence of the Dryad DOI -- both in the in-text data-availability statement ("Data on plant phenology are available at the Dryad Digital Repository: https://doi.org/10.5061/dryad.c59zw 3r69") and in the reference-list entry (Bucher & Römermann, 2020a) -- contains a stray internal space that splits "c59zw3r69" into "c59zw" and "3r69". This is almost certainly an artifact of PDF line-wrapping/hyphenation during text extraction (GROBID or similar), not a typo in the original PDF, since a genuinely broken DOI would not be something the original authors or Wiley typesetting would publish twice identically. Because the space breaks the DOI into two whitespace-separated tokens, any regex-based or URL-pattern detector looking for a well-formed `10.5061/dryad.xxxxx` DOI or a contiguous `doi.org/10.5061/dryad...` URL would fail to match it as written in this extracted text, even though the platform (Dryad) is fully supported by metacheck. This is a text-extraction corruption problem upstream of the detector logic, not a gap in platform coverage or a config issue -- the fix would belong in the extraction/normalization step (e.g. stripping/collapsing whitespace inside DOI-shaped strings before pattern matching), not in adding a new platform detector. Separately, a second repository link in the paper (TRY database, Dataset 416, a plant-trait database) is on an unsupported platform and was not something the coder recorded either, but it is incidental to this specific gap -- the primary data-sharing link the coder flagged is the Dryad one, and that one is a garbled-DOI extraction issue.
10.1111/1365-2745.13606	Text extraction: identifier corrupted by the PDF-to-text step	Dryad is a fully supported platform in metacheck (R/archive-dryad.R), including a specific fallback in dryad_links() for exactly this scenario: a bare, non-hyperlinked DOI mention in body/reference text, matched via the regex `(?:https?://)?(?:doi\ .org/)?10\ .5061/dryad\ .[A-Za-z0-9]+` (plus a found_href branch for real anchor-tag links, which does not apply here since the coder's own note confirms no clickable URL exists in the extracted_urls table). The detector still misses this citation because the text extraction (likely GROBID, given the paper's Wiley/BES PDF source) corrupted the DOI across what was probably a line wrap in the original PDF: the extracted full_text reads "https://doi.org/10.5061/ dryad.f7m0c fxv3" (verified by direct substring inspection of full_text) — note two inserted spaces, one between "10.5061/" and "dryad.", and a second one splitting the identifier itself into "f7m0c" and "fxv3". Because `[A-Za-z0-9]+` cannot match across embedded whitespace, neither the dryad_bare_regex pattern nor the DOI-extraction helper `.dryad_doi()` (which uses the same `10\ .5061/dryad\ .[A-Za-z0-9]+` pattern) can recognize this as a single token, so dryad_links() returns no match for this paper. This is a text-extraction/garbling problem upstream of metacheck's Dryad detector logic, not a missing platform or a misconfigured host: live-checking the DOI confirms it resolves correctly to a real, public, two-file Dryad dataset that exactly matches the paper's subject matter, so the citation itself is intact in the published paper — only the extracted text metacheck works from is corrupted at this one point.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.13694	Text extraction: identifier corrupted by the PDF-to-text step	Metacheck has a full, working Zenodo detector (R/archive-zenodo.R), and its design is explicitly built to catch bare DOI citations of the exact form "10.5281/zenodo.NNNNNNN" with or without a URL scheme or doi.org prefix (see the regex and comment at R/archive-zenodo.R lines 5 and 28: "10.5281/zenodo.1234567" is routinely cited without any URL scheme at all"). The gap here is not a missing platform but a text-extraction corruption: in the extracted full_text, the citation reads "Zenodo, https://doi.org/10.5281/ zenodo.3756781" with a literal space inserted between "10.5281/" and "zenodo.3756781" (almost certainly a PDF line-wrap artifact from GROBID/PDF-to-text conversion at a hyphenation or column break). Metacheck's R/text-extractors.R extract_urls() function, which is the upstream source of link candidates that archive-zenodo.R and other archive-*.R detectors work from, uses a \ b-bounded regex requiring a contiguous domain-like token ([\ w.-]+\ .[a-z]{2,}) with no internal whitespace. That regex matches "https://doi.org/10.5281/" and "zenodo.3756781" only as two separate, incomplete fragments across the inserted space, neither of which satisfies the Zenodo-ID pattern (10\ .5281/zenodo\ .[0-9]+) that archive-zenodo.R looks for. The result is that no valid Zenodo candidate URL is ever produced for the detector to test, which is consistent with metacheck_any_detector_hit being false even though the paper does cite a fully public, CC-BY-4.0, live Zenodo dataset with three files, and even though Zenodo is a fully supported platform. This is a text-extraction/garbling problem (a split DOI token), not a missing detector and not a dead repository.
10.1111/1365-2745.13875	Text extraction: identifier corrupted by the PDF-to-text step	metacheck fully supports Dryad (R/archive-dryad.R), and the DOI prefix 10.7280 used by this exact dataset is explicitly in .dryad_doi_prefixes() (archive-dryad.R line 37), with dryad_links()'s regex built to match "10.7280/[A-Za-z0-9]+" as a bare in-text DOI even without a scheme. The failure is not a platform gap but a text-extraction corruption: the paper's primary data-availability sentence reads "Data available from the Dryad Data Repository http://10.0.28.112/D1D99J (Campbell et al., 2022)" -- the DOI prefix "10.7280" has been replaced by what looks like a private IP address "10.0.28.112" (almost certainly a PDF/GROBID hyperlink-extraction artifact mangling the embedded doi.org URL), so neither the datadryad.org host string nor the 10.7280 prefix pattern is intact at that location for dryad_bare_regex to match, and it is unlikely the href/url table preserved a clean target either, since the same corrupted string appears in the flattened text. A second, well-formed copy of the same DOI (https://doi.org/10.7280/D1D99J) exists in the reference list under the Campbell et al. (2022) entry; in principle this clean occurrence should satisfy the regex, but the reported metacheck_any_detector_hit=false indicates it did not fire in the live run, most plausibly because GROBID-style extraction of the actual PDF introduces a line-wrap or whitespace split around the DOI (e.g. "doi. org/10.7280/D1D99J") that defeats the regex even though the flattened full_text I was given displays it as one unbroken string. I manually confirmed the DOI itself is completely valid and live: doi.org/10.7280/D1D99J redirects cleanly to a real, public, accessible Dryad dataset page with six CSV files and four READMEs, so this is a genuine false negative caused by extraction-level corruption/formatting of the DOI text, not a missing platform or a dead repository.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.13882	Text extraction: identifier corrupted by the PDF-to-text step	Metacheck fully supports this exact platform and host: R/archive-dataone.R's .dataone_hosts() already registers host = "arcticdata.io" with doi_prefix = "10.18739" (line 33), which is precisely the DOI prefix all four of this paper's Arctic Data Center citations use, and dataone_links()'s bare-DOI regex (lines 105-109) is built to catch exactly this "https://doi.org/10.18739/<suffix>" citation shape. Live-checking confirmed the dataset is real, public, CC0-licensed, and resolvable through the identical Metacat object endpoint metacheck's own .dataone_info() queries. The actual failure is upstream, in the PDF-to-text extraction that produced full_text: within the data-availability statement itself, three of the four DOI suffixes are split by a spurious whitespace character in the middle of the identifier -- 'https://doi.org/ A2GT5 FG86' instead of 'https://doi.org/A2GT5FG86', 'http://doi. org/10.18739/ A23R0 PV43' instead of '.../A23R0PV43', and 'http://doi. org/10.18739/A2GX44W2M' with a space inserted inside 'doi.org'. Only the second DOI (A2000018G) survived extraction intact. dataone_links()'s matching regex for the bare-DOI fallback is '(?:doi\ \ .org/)?(?:10\ \ .18739)/[A-Za-z0-9._/-]+', which requires the suffix to be one unbroken token of those characters with no whitespace; a space inside 'A2GT5 FG86' terminates the match after 'A2GT5', producing an incomplete, non-resolvable PID rather than no match at all, and likewise breaks the 'doi.org' host token itself in the third and fourth citations ('doi. org'). This is a genuine upstream text-extraction corruption (apparently from the PDF layout, where a line-wrap or font-kerning artifact inserted a space mid-token), not a gap in metacheck's own DataONE detector logic, host list, or regex design for the well-formed case. This matches the EXTRACTION_GAP_DOI_GARBLED category: the link is present in the source text but corrupted in a way that defeats token-based regex matching.
10.1111/1365-2745.13999	Text extraction: identifier corrupted by the PDF-to-text step	Dryad is a fully supported platform in metacheck (R/archive-dryad.R exists and the live link resolves cleanly to a standard public Dryad landing page with no login and a "no restrictions" license), so this is not a platform-support gap. The failure is a text-extraction corruption: in full_text the DOI appears as "https://doi.org/10.5061/dryad.7sqv9 s4sh" -- a literal space has been inserted between "7sqv9" and "s4sh," splitting the Dryad suffix in two. Any DOI/URL-matching regex metacheck runs over the extracted text (whether matching "10.5061/dryad\ \ .\ \ S+" or a general doi.org URL pattern) would stop at the whitespace and either fail to match the full suffix or extract "dryad.7sqv9" as a truncated/invalid DOI, which would not resolve via the Dryad API. The same kind of mid-token space corruption is visible elsewhere in this paper's extracted references (e.g. "foreco.2012.02" rendered as ". foreco.2012.02", "isprs jprs.2020.01.012" instead of "isprsjprs..."), indicating this is a systematic PDF-extraction artifact (likely from line-wrap/hyphenation handling in GROBID or whatever extractor produced full_text) rather than something specific to Dryad's platform support. Additionally, no distinct "Data Availability Statement" section/heading was present anywhere in the extracted full_text at all -- the only occurrence of the link is embedded in the reference list citing the companion Dryad data-descriptor record, suggesting the statement box itself (common in BES/Journal of Ecology articles as a separate panel) may not have been captured by extraction, compounding the problem for any heading-based statement-locator logic upstream of the regex match.
10.1111/2041-210X.12918	Text extraction: identifier corrupted by the PDF-to-text step	DAS cites two locations: 'The version 1.0.0 of ecolottery is available on CRAN... The development version is available on Github (https://github.com/frmunoz/ [extraction cuts off here, jumping into Authors' Contributions]).' The repo-name path segment ('ecolottery') is entirely absent from the extracted text due to a PDF extraction defect -- Metacheck's github_links() regex requires both org AND repo segments, so with only 'frmunoz' present it cannot match even in principle. The adjacent CRAN URL in the same sentence is independently also garbled, corroborating sentence-level extraction corruption.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13350	Text extraction: identifier corrupted by the PDF-to-text step	Paper cites two links: CRAN (extracted cleanly, twice) and GitHub ('https ://github.com/ achub aty/grain scape', extracted with spaces inserted inside the URL and inside both the owner and repo names). Any regex-based URL matcher fails on the garbled GitHub form even though GitHub has a complete, working backend in Metacheck. CRAN is cited cleanly but is moot since no CRAN backend exists at all.
10.1111/2041-210X.13681	Text extraction: identifier corrupted by the PDF-to-text step	Both repository DOIs ARE present in the extracted full_text, and both platforms (Dataverse, confirmed by the "Powered by The Dataverse Project v6.8.4-SP" footer at borealisdata.ca, and Zenodo, confirmed by the zenodo.org landing page) are on metacheck's supported-platform list, so this is not a coverage gap. The failure is a text-extraction/OCR artifact: the PDF-to-text conversion injected a stray space exactly at the point where each DOI line-wrapped in the original PDF, producing "https://doi. org/10.7939/DVN/QYMAYV" (space between "doi." and "org") for the Dataverse reference and "https://doi.org/10.5281/ zenodo.5070812" (space between "10.5281/" and "zenodo.5070812") for the Zenodo reference. Any DOI-matching regex or URL tokenizer that expects a contiguous string (no embedded whitespace) will fail to recognize either as a valid, resolvable DOI/URL, even though a human reader can see and manually reconstruct both. This matches metacheck_any_detector_hit=false: the detector presumably could not match either garbled string. Separately, the in-text Data Availability Statement itself uses named-citation style ("available on UAL Dataverse Derocher (2021)"; "available on Zenodo Togunov et al. (2021)") with no inline clickable URL -- the resolvable DOIs exist only in the back-matter reference list, so even a perfect-whitespace extraction would still require cross-referencing the in-text citation key to the bibliography entry to recover the link, which is an extra step beyond simple statement-adjacent URL scanning. The coder's recorded link (https://doi.org/10.7939/DVN/QYMAYV, acquired via "a targeted web search" rather than from the text) is confirmed correct and live, but the coder missed the second repository (the Zenodo replication code) entirely.
10.1111/2041-210X.13821	Text extraction: identifier corrupted by the PDF-to-text step	metacheck fully supports this exact platform and host: R/archive-dspace7.R's .dspace7_hosts() explicitly lists "conservancy.umn.edu" (annotated "Data Repository for the University of Minnesota (USA)"), and dspace7_links() already contains code specifically added for this exact failure mode (documented inline as issue #435, citing the near-identical sibling DOI 10.13020/q3y6-h459 on the same host): when a paper cites only a bare DOI with the host domain never appearing in the surrounding text, the function live-resolves every DOI-shaped string found via text_search() and keeps any that redirect to a known DSpace7 host. That mechanism would correctly catch this paper's citation -- except that the DOI as it literally appears in paper's full_text is garbled by a stray inserted space: "https://doi.org/10.13020/ 1f35-rn20" instead of "https://doi.org/10.13020/1f35-rn20" (verified by extracting the exact sentence from full_text). I tested R/archive-dspace7.R's doi_bare_regex directly against both strings: it matches the clean DOI but returns zero matches against the garbled one, because the regex requires an unbroken token after "10.xxx/" and the inserted space breaks it mid-string. This is almost certainly a PDF-text-extraction artifact (the reference list line likely wrapped at that point in the original PDF, and the extraction tool inserted a space at the line break) rather than a metacheck regex design flaw -- the same gap would equally affect any other DOI-based detector in metacheck that depends on this shared doi_bare_regex pattern if a citation's DOI happens to be split by extraction at the same point. I did not attempt to fix this since the task calls for an audit and root-cause classification only.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13850	Text extraction: identifier corrupted by the PDF-to-text step	CORRECTION to an earlier, unverified pass that claimed the GitHub repo was 'genuinely inaccessible... not a metacheck defect' -- this is live-verified FALSE; both the GitHub org and the actual repo (ConScape/ConScape.jl) return HTTP 200 via the GitHub API. Primary cause: the reference-list Zenodo citation has a space inserted between '10.5281/' and 'zenodo.5810881' (confirmed via grep on raw full_text), defeating zenodo_links()'s regex (tested directly in R, returns no match). Secondary: the body-text GitHub citation is a bare org URL with no repo segment, so github_repo() correctly returns NULL for that specific citation form -- not itself a bug, since no specific repo was actually named in that URL.
10.1111/1365-2656.12794	Text extraction: no trace of the citation in the extracted text	Dryad is one of metacheck's supported platforms (dataone, dataverse, dryad, dspace/dspace7, figshare, github, gitlab, mendeley, osf, psycharchives, rbox, researchdata4tu, reshare, zenodo, aspredicted), so this is not a platform-support gap. The actual problem is upstream of detection: the full_text metacheck extracted for this paper (apparently a PMC manuscript version, judging by the trailing "Refer to Web version on PubMed Central for supplementary material" and "J Anim Ecol. Author manuscript; available in PMC 2018 June01" notices embedded in the body) simply does not contain the journal's "Data Accessibility" statement or the Dryad DOI (10.5061/dryad.95486) anywhere in the extracted string — I read the entire full_text field end to end and confirmed no occurrence of "Dryad," "data availability," "data accessibility," or any doi.org/5061 string. Since metacheck's Dryad detector can only act on text it is given, and the availability statement text was never captured by the upstream extraction pipeline (likely because the PMC author-manuscript rendering omits or relocates the data-accessibility statement relative to the original journal HTML/PDF), the detector correctly reports metacheck_any_detector_hit = false through no fault of the Dryad-detection logic itself. The repository itself is independently verified live and public (confirmed via WebFetch to datadryad.org), so this is a text-extraction/input gap, not a detector-capability or configuration gap.
10.1111/1365-2656.12900	Text extraction: no trace of the citation in the extracted text	metacheck does have a dedicated Dryad detector (R/archive-dryad.R in the metacheck-new source tree), and the cited repository — https://doi.org/10.5061/dryad.mb64cr8, redirecting live to datadryad.org/dataset/doi:10.5061/dryad.mb64cr8 — is a real, public, directly-matching Dryad dataset for this exact paper, with no login barrier. The detector nonetheless could not have fired because the actual DOI/URL string is simply absent from the full_text that was extracted from the PDF: the data-availability sentence reads only "Data for this publication are available online at the Dryad Digital" and is then cut off, running straight into the unrelated author-contributions paragraph ("S.P. conceived the ideas..."). No "Repository" word, no "10.5061" string, and no "doi.org" string appear anywhere in full_text. This is a pure text-extraction gap (likely the original PDF rendered the DOI as a hyperlink or footnote-style reference that the extraction pipeline dropped or failed to inline as visible text) — not a missing platform, not a configuration gap for an unrecognized host, and not a bug in the Dryad-matching logic itself, since that logic never receives the string it needs to match against.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.13052	Text extraction: no trace of the citation in the extracted text	metacheck has a dedicated, working detector for Dryad (R/archive-dryad.R exists alongside the other platform modules such as archive-figshare.R, archive-dataverse.R, etc.), so this is not a platform-support gap. The failure is purely at the text-extraction stage: the DOI string "10.5061/dryad.vc80r13" that identifies this specific Dryad record is absent from the full_text metacheck extracted from the PDF. It appears the source PDF (a Wiley/Journal of Animal Ecology article) had the data-availability sentence and the corresponding reference-list entry each truncated mid-string during text extraction -- the data-availability statement stops at "...available from the Dryad Digital" (no "Repository", no URL), and the reference-list citation "Gibb, H., Grossman, B. F., Dickman, C. R., Decker, D., & Wardle, G. (2019). Data from... Dryad Digital Repository, https://doi.org/10.5061/dryad." is cut off immediately after "dryad." before the record suffix "vc80r13" that makes the DOI resolvable. Since no detector, regex, or URL-matcher can find a string that isn't present in the text it is given, metacheck's extracted_urls list correctly came up empty for this paper even though its Dryad backend would very likely have correctly identified and verified doi:10.5061/dryad.vc80r13 as a live, public dataset had the complete string been present in the extracted text.
10.1111/1365-2656.14093	Text extraction: no trace of the citation in the extracted text	metacheck does support Figshare as a platform, so if a resolvable Figshare URL or DOI (e.g. a "10.6084/m9.figshare.XXXXXXX" string or a figshare.com/articles/... link) were present in the text, the existing Figshare detector should be able to find and validate it. The actual problem here is upstream of detection: the paper's extracted full_text never contains such a string. The data-availability sentence reads only "Data are archived on Figshare(Mahoney & Pasch, 2024)," citing a reference-list entry that is itself corrupted by the same GROBID-style extraction/OCR process that produced this full_text -- "Mahoney, S. M., & Pasch, B. (2024). , 55(6), 1483-1487." is missing the dataset title and the words "Figshare" or any repository name, leaving only a volume/issue/page triplet. I independently searched Figshare and the open web for a Mahoney & Pasch 2024 dataset matching this paper's topic (mammalian food-caching evolution / Mt. Graham red squirrels) and could not locate one; the only Figshare DOI associated with these two authors (10.6084/m9.figshare.5285635) belongs to their unrelated 2017 "Caching for where and what" paper. Because no machine-resolvable identifier for the actual 2024 dataset exists in the extracted text (and I could not find one by searching, so I cannot even confirm the live dataset is reachable), metacheck's Figshare detector has nothing to act on; this is a text-extraction gap (the citation/DOI was either never present in the PDF's data statement as a hyperlink, or was stripped during PDF-to-text conversion), not a platform-coverage or detector-logic failure. metacheck_any_detector_hit=false is consistent with this: there is no link in the text for any detector, Figshare or otherwise, to find.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.12927	Text extraction: no trace of the citation in the extracted text	Metacheck's figshare.R has full detector support for this exact institutional instance -- "ore.exeter.ac.uk" (Open Research Exeter's branded Figshare front end) is explicitly listed in .figshare_vanity_hosts(), and the figshare detection regex (.figshare_host_regex()) also matches any literal "figshare.com" substring, which covers the newer "uoe.figshare.com" subdomain this item now lives under. So this is not a platform gap: both the old branded host and the current host are things metacheck's figshare backend already knows how to query via api.figshare.com. The actual gap is at the extraction stage. The paper's data-availability sentence cites only a bare hdl.handle.net/10871/27138 URL -- no figshare.com domain, no figshare DOI (10.6084/m9.figshare... or an institutional DOI prefix), and no bare numeric article ID appear anywhere in the full text. figshare_links() works by regex-matching the paper's own hyperlink table and body text for figshare-looking strings (a figshare host in the URL, or a figshare DOI pattern); it does not follow live HTTP redirects to discover that a handle.net link currently resolves to Figshare. Separately, metacheck has no generic handle.net resolver at all -- the only handle.net handling in the codebase is in archive-psycharchives.R, scoped to PsychArchives' own handle prefix (20.500.12034), which has no bearing on Exeter's prefix (10871). Because the citation text itself never names figshare or exposes a figshare-recognizable identifier, no existing detector -- correctly implemented as each one is for the identifiers it is given -- has anything to match against, so detection fails for a text-extraction reason, not a missing-platform-support reason or a bug in the figshare detector itself.
10.1111/1365-2664.14138	Text extraction: no trace of the citation in the extracted text	Figshare DOI (10.6084/m9.figshare.16436379.v2) is cited exactly once, inside a formatted 'Data from:' entry in the References list; no DAS section exists in the extracted text at all. Metacheck's repo_check calls text_search() with default include_refs=FALSE, which filters out all rows tagged section_type=='references' (text_search.R lines 104-110) before pattern-matching -- so the citation is structurally invisible to the default search regardless of host support. Metacheck's Figshare backend is otherwise fully functional and would resolve this DOI if the text reached it (confirmed live via api.figshare.com/v2/articles/16436379, HTTP 200, CC BY 4.0).
10.1002/pan3.10756	Metacheck code: a defect in detecting or retrieving a supported repository	figshare_links() correctly detects the DOI citation via the institutional vanity-host/DOI-prefix table (repository.mmu.ac.uk / 10.23634, both explicitly registered in .figshare_vanity_hosts() and .figshare_doi_prefix_hosts()), and live, direct invocation of the installed package's repo_check() on this single paper today DOES detect the repo (repo_n = 2, two duplicate entries differing only by a trailing citation-punctuation period) -- directly contradicting the repo_n = 0 stored in res_repo_check.RData. The live run's gated_repos shows repo_error = 'arguments imply differing number of rows' for this paper's own figshare id lookup (one live attempt reported '00636962 could not be found on api.figshare.com'), suggesting the same batch-level tryCatch/data.frame-construction fragility identified for 10.1111/1365-2745.13728 is also implicated here, compounded by apparent intermittency in api.figshare.com's own response reliability for this specific institutional-DOI lookup path (a second live attempt via a different code path, querying the article id 32616504 directly, succeeded). The stored repo_n = 0 is best explained as a stale/incorrect batch-run artifact layered on top of a real but intermittent figshare-institutional-DOI lookup fragility, not a permanent inability to detect or resolve this repository.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.13103	Metacheck code: a defect in detecting or retrieving a supported repository	files_n = 0 directly contradicts the prior audit's assumption that detection succeeding meant the gap was closed. The resolvable DOI (10.4225/...) is a genuine Bridges/Figshare DOI that metacheck's dataverse/figshare DOI-prefix tables should in principle be able to query for file listings, but because the concatenated /s/ share-link token was also extracted as a second repo_url and is unresolvable by design (opaque token, no API endpoint for third-party resolution -- same limitation as .figshare_id() documents elsewhere), and because the batch-level figshare DOI retrieval call that would otherwise fetch files for the whole corpus is itself broken by a separate code defect (see 10.1111/1365-2664.14575 below, which shares the identical 'arguments imply differing number of rows' failure signature across the entire figshare-DOI batch), no files were enumerated for either repo_url belonging to this paper. This combines two real, separate code-level gaps -- the /s/ token's unresolvable-by-design limitation, and the corpus-wide figshare-DOI batch crash -- both of which are genuine platform bugs, not a non-issue.
10.1111/1365-2435.13360	Metacheck code: a defect in detecting or retrieving a supported repository	metacheck fully supports Dataverse as a platform, and dataverse.nl specifically is one of its individually-verified known hosts (R/archive-dataverse.R, .dataverse_hosts() line 64), with its DOI prefix 10.34894 even already mapped to it in .dataverse_doi_prefix_hosts() (line 303). The live chain confirms this paper's dataset genuinely resolves to dataverse.nl under DOI 10.34894/UUNBQQ, which is exactly the registered prefix. The gap is that the paper's text never contains either the host string "dataverse.nl" or the DOI string "10.34894" — it cites the dataset only as the bare handle-resolver URL "https://hdl.handle.net/10411/6Q1YDC". dataverse_links() has three matching passes (real-hyperlink host match, bare-text host match, bare-DOI-prefix match), and all three require either the dataverse.nl hostname or a 10.34894-prefixed DOI to literally appear in the extracted text/href -- neither does, since handle.net URLs use their own handle-prefix namespace (here 10411, DANS's generic handle prefix) which is unrelated to the target repository's DOI prefix. Metacheck does have precedent for resolving a specific hdl.handle.net prefix to a platform (archive-psycharchives.R maps handle prefix 20.500.12034 to PsychArchives), but no equivalent mapping exists for DANS's 10411 prefix pointing at DataverseNL (or at DANS's other dataverse-based Data Stations). Resolving this would require either (a) adding 10411 as a recognized handle prefix that redirects to a Dataverse-family host, or (b) following the live HTTP redirect chain for any hdl.handle.net citation before applying the existing host/DOI-prefix matchers -- neither of which the current static-text extraction does for this handle prefix. [RECLASSIFIED from SUPPORTED_PLATFORM_CONFIG_GAP: not a host/DOI-prefix table entry -- requires a new resolution mechanism (handle-prefix redirect, shortlink expansion, or DOI-to-host resolution) or the host runs an API shape metacheck does not support.]

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.13523	Metacheck code: a defect in detecting or retrieving a supported repository	RECLASSIFIED 2026-10-13 (SUPPORTED_PLATFORM_BUG, was SUPPORTED_PLATFORM_CONFIG_GAP): UNSWorks is supported, but the paper cites it through UNSW's Handle redirector (handle.unsw.edu.au), which metacheck does not follow: a metacheck code defect. (The redirector answered 503 on 2026-10-13.) Earlier explanation: metacheck has a working DSpace 7 detector (R/archive-dspace7.R) and its host allowlist, .dspace7_hosts(), already includes "unsworks.unsw.edu.au" (line 106) -- so if the paper had cited the unsworks.unsw.edu.au URL directly, dspace7_links() would find it. The actual problem is that this paper's data-availability statement cites a different, un-registered domain: "http://handle.unsw.edu.au/1959.4/resource/collection/resdatac_1003/1". UNSW operates handle.unsw.edu.au as a separate CNRI Handle-System redirector in front of UNSWorks; it 302-redirects to the unsworks.unsw.edu.au item page but is a distinct hostname that never appears in .dspace7_hosts(). dspace7_links() checks three paths: (1) real hyperlink hrefs matching a known host -- fails, handle.unsw.edu.au isn't in the list; (2) a bare-text mention of a known host string -- fails, same reason, the text only contains "handle.unsw.edu.au", never "unsworks.unsw.edu.au"; (3) a bare DOI-shaped string ("10.xxxx/yyyy") resolved live and checked against the known-host list -- also fails, because the citation in the paper's text is a Handle-resolver URL with no "10." DOI prefix anywhere in it (the item's real DOI, 10.26190/5e1ce7350483e, only appears on the landing page itself, never in the citing paper's text). So all three of dspace7_links()'s detection paths miss this specific citation even though the underlying repository (unsworks.unsw.edu.au, DSpace 7) is fully supported. This is a configuration gap, not a missing platform or a code bug: the fix would be adding "handle.unsw.edu.au" (or a generic handle.* / hdl.handle.net resolver pattern) to the host recognition logic, analogous to how issue #435 already added live DOI-redirect resolution for the no-host-string-at-all case. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: Needs a Handle redirect (handle.unsw.edu.au) resolved before matching UNSWorks, which metacheck supports. The redirector returned 503 Service Unavailable on 2026-10-13, and the cited path is a collection, not a single item.]

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.13850	Metacheck code: a defect in detecting or retrieving a supported repository	metacheck fully supports Figshare and even has explicit, dedicated support for exactly this host and DOI prefix: R/archive-figshare.R's .figshare_vanity_hosts() lists "smithsonian.figshare.com" by name (line 97), and .figshare_doi_prefix_hosts() maps DOI prefix "10.25573" to "smithsonian.figshare.com" (line 136). The citation text is also cleanly extractable with no garbling. The actual bug is in ID-type resolution: the cited DOI, https://doi.org/10.25573/data.c.5073065.v1, is a Figshare COLLECTION DOI (identifiable by the literal ".c." segment after the DOI prefix, the same marker .figshare_collection_id() uses for the generic 10.6084/m9.figshare.c.<id> case), but .figshare_id()'s institutional-prefix pattern (archive-figshare.R lines 327-344) does not exclude or recognize this ".c." collection marker for institutional prefixes -- it simply strips any non-digit-containing sub-segment and returns the trailing digits, "5073065", as if it were an ARTICLE id. I confirmed this live: devtools::load_all() then .figshare_id(url) returns "5073065", but .figshare_collection_id(url) returns NA (its own pattern only checks the literal 10.6084 prefix, not institutional ones like 10.25573, per the comment at lines 460-463 of the same file: "Only the literal 10.6084 prefix is checked here... no institutional-collection-DOI citation has been found yet"). Because figshare_info() checks ids$figshare_id first and only falls through to collection-expansion logic when figshare_id is NA (line 591: "unresolved <- is.na(ids$figshare_id)"), it never tries the collection path for this URL -- it instead calls GET https://api.figshare.com/v2/articles/5073065 directly, which I confirmed live returns HTTP 404 (while GET https://api.figshare.com/v2/collections/5073065 returns HTTP 200). Running figshare_info() on this exact URL in the live package reproduces the failure: it returns a row with error = "unfound" and a warning "5073065 could not be found on api.figshare.com". This directly contradicts the payload's metacheck_any_detector_hit = true for this paper: the one link actually cited in the text is not resolvable by the current code, because institutional-prefix DOIs for Figshare collections are not yet recognized as collections the way bare 10.6084/m9.figshare.c.<id> DOIs are.
10.1111/1365-2435.14284	Metacheck code: a defect in detecting or retrieving a supported repository	Zenodo is a fully supported platform (R/archive-zenodo.R) and the Zenodo DOI is extractable: I located the exact string "https://doi.org/10.5281/zenodo.7573829" in full_text, clean and un-garbled, matching the coder's recorded link exactly. The data-availability sentence itself reads "All data and associated scripts used in this study are available on" with the actual link/citation marker stripped out (a GROBID artifact from resolving an inline citation) -- the literal DOI string only survives later, embedded in the formatted bibliography as the self-citation entry "Shah, A. A., Hotaling, S., ... (2023). ... Zenodo. https://doi.org/10.5281/zenodo.7573829". This is why metacheck's own detector misses it despite the platform and pattern being fully supported: zenodo_links() in R/archive-zenodo.R calls text_search(paper, zen_bare_regex, return = "match", perl = TRUE) without passing include_refs = TRUE, and text_search() in R/text_search.R defaults include_refs to FALSE, which explicitly filters out any text row where section_type == "references" (lines 104-109). Since the only textual occurrence of this DOI in the paper is inside a reference-list entry (and the paper's extracted url/href table apparently has no corresponding hyperlink either, since found_href would otherwise have caught it regardless of section), the bare-DOI body-text fallback in zenodo_links() never sees it, and the detector returns no hit even though both the platform support and the raw text content are present and intact. This is a genuine config/argument gap in a supported detector, not a missing platform or a garbled citation.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.13005	Metacheck code: a defect in detecting or retrieving a supported repository	The data-availability sentence in the paper's full text reads: "The data used in this study are available from the Dryad Digital [Repository]... Repository: https ://doi.org/10.5061/dryad.m20hf10 (Naniwadekar, Chaplod, Datta, Rathore, & Sridhar, 2019)." metacheck has a dedicated Dryad detector (R/archive-dryad.R), and 10.5061 is the FIRST and primary entry in .dryad_doi_prefixes(), with its own purpose-built body-text regex "(?:https?://)?(?:doi\ \ .org/)?10\ \ .5061/dryad\ \ .[A-Za-z0-9]+" in dryad_bare_regex. I ran this exact regex in R against the exact text string from the paper (including the stray "https ://" space artifact from PDF extraction) and it matched successfully, returning "doi.org/10.5061/dryad.m20hf10" -- the leading optional "https?://" group simply fails to consume the malformed "https ://" and the engine falls through to matching starting at "doi.org/10.5061/...", which is literally present with no intervening whitespace. Since the regex is not the point of failure on the text I was given, yet metacheck_any_detector_hit is reported as False for this paper, the gap must lie elsewhere in the pipeline between GROBID text extraction and the detector call (e.g. the real GROBID output differing from this harness's full_text in whitespace/hyphenation around the DOI, or a failure in the live dryad_info() lookup/repo_check step rather than in link detection itself). This is a supported-platform bug rather than a missing-text or platform-gap issue: the correct, resolvable DOI is present in the text in a form the detector's own regex is proven to match, and the live Dryad page is fully public with the exact matching dataset.
10.1111/1365-2664.13035	Metacheck code: a defect in detecting or retrieving a supported repository	RECLASSIFIED 2026-10-13 (SUPPORTED_PLATFORM_BUG, was SUPPORTED_PLATFORM_CONFIG_GAP): Supported, FAIR repository (dataSTORRE, host registered in metacheck) cited through an hdl.handle.net link, which metacheck does not follow to the host (metacheck issue #466): a metacheck code defect. Earlier explanation: CONFIRMED CONFIG GAP, still present. The citation ('Data in this study are available from DataSTORRE... http://hdl.handle.net/11667/102') uses a bare Handle System URL, not a DOI, so it can only be recognised via a host match. datastorre.stir.ac.uk is absent from BOTH .dspace7_hosts() (R/archive-dspace7.R, 63 verified installations) and .dspace_legacy_hosts() (R/archive-psycharchives.R, 6 installations) -- confirmed by direct search, zero matches for 'datastorre' or 'stir.ac.uk' in either file. dspace7_links()'s DOI-resolution fallback (which CAN catch a bare DOI whose host isn't separately listed, by live-resolving it and checking where it redirects) only matches DOI-shaped strings ('10.xxxx/...'); a hdl.handle.net URL is not DOI-shaped, so even that fallback never fires. The repository platform (DSpace) is generally supported, but this specific host was simply never added to either allowlist -- a straightforward config/allowlist omission, not a logic defect. CORRECTION 2026-10-07: a live rerun confirmed datastorre.stir.ac.uk IS now present in .dspace_legacy_hosts(), but still produced repo_n = 0, files_n = 0 for this paper. This is metacheck issue #466 (hdl.handle.net redirect resolution for DSpace legacy), a separate, still-open bug explicitly distinguished from the host-registry gap this row originally described -- confirmed by 01_run_metacheck/08_rerun_issue_461_registry_papers.R's own header comment, which excluded this exact paper from that rerun for this exact reason. The host addition alone does not resolve a hdl.handle.net-style citation. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: The host (datastorre.stir.ac.uk) is now registered in metacheck, but the paper cites it through an hdl.handle.net link, which metacheck does not resolve to the host (metacheck issue #466, open on 2026-10-13).]

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.14198	Metacheck code: a defect in detecting or retrieving a supported repository	RECLASSIFIED 2026-10-13 (SUPPORTED_PLATFORM_BUG, was SUPPORTED_PLATFORM_CONFIG_GAP): Supported, FAIR repository (dataSTORRE, host registered in metacheck) cited through an hdl.handle.net link, which metacheck does not follow to the host (metacheck issue #466): a metacheck code defect. Earlier explanation: metacheck has real, working DSpace support split across two code paths: R/archive-dspace7.R (DSpace 7+, HAL-JSON /server/api/) and R/archive-psycharchives.R's .dspace_legacy_hosts() (DSpace <=6, legacy /rest/ API). Both paths work only against an explicit allowlist of installation hostnames rather than detecting DSpace generically from the resolved URL. I read both host lists directly from source: .dspace7_hosts() in archive-dspace7.R (64 hosts, e.g. repository.gatech.edu, aura.abdn.ac.uk) and .dspace_legacy_hosts() in archive-psycharchives.R (6 hosts: www.psycharchives.org, bonndoc.ulb.uni-bonn.de, dspace.ut.ee, qsardb.org, repositorio-digital.cide.edu, ses.library.usyd.edu.au). The paper's repository link resolves to datastorre.stir.ac.uk (University of Stirling's dataSTORRE, confirmed live to be running DSpace), which appears in neither list. Because both dspace7_links() and dspace_links() match only against their respective host allowlists (via .dspace7_host_regex()/.dspace_legacy_host_regex()), a citation to http://hdl.handle.net/11667/195 -- which does carry a resolvable handle and does redirect to a real, public, CC-BY-licensed DSpace item with a listed data file -- is invisible to metacheck simply because datastorre.stir.ac.uk was never added to either registry. This is exactly the SUPPORTED_PLATFORM_CONFIG_GAP case: the platform family (DSpace) is supported, extraction of the citation is trivial (a single unambiguous handle link appears twice in the text), but this specific host is unregistered. CORRECTION 2026-10-07: same as 10.1111/1365-2664.13035 -- a live rerun confirmed the datastorre.stir.ac.uk host addition did not resolve this paper (repo_n = 0, files_n = 0), because this citation is also a bare hdl.handle.net URL, blocked by the same open issue #466 (handle-redirect resolution), not by the host registry gap this row originally described. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: The host (datastorre.stir.ac.uk) is now registered in metacheck, but the paper cites it through an hdl.handle.net link, which metacheck does not resolve to the host (metacheck issue #466, open on 2026-10-13).]
10.1111/1365-2664.14704	Metacheck code: a defect in detecting or retrieving a supported repository	This is the same corpus-wide batch-poisoning defect identified for 10.1111/1365-2664.14575, affecting the Dataverse platform instead of Figshare. Checking res_repo_check$gated_repos shows 'arguments imply differing number of rows' (with varying row counts) attached to all 103 dataverse-flagged repo_url rows across the entire corpus -- i.e., every single Dataverse citation in the manuscript's full dataset failed in this pipeline run. Reading inst/modules/repo_check.R's Dataverse block confirms dataverse_info() is called once for the whole corpus's dataverse URL batch inside a single tryCatch (lines ~861-927), with the Dataverse module's own code comments explicitly anticipating and partially guarding against exactly this failure mode ('a field can come back as a length-zero value...which would...break this data.frame() call with arguments imply differing number of rows'), but the guard (%empty_or%) is not applied everywhere a field could be malformed across the full corpus of real-world API responses. Calling .dataverse_info() directly for THIS paper's host/DOI in isolation succeeds cleanly with no error, proving the defect is not specific to this paper's own data -- some other paper's Dataverse record elsewhere in the same batch triggered the crash, and the caught error message was then stamped onto every Dataverse repo_url in the corpus via repos$repo_error[repos$repo_url %in% dv_urls] <<- conditionMessage(e). This paper's data is fully public, free, and CC-BY-licensed, and metacheck's own underlying retrieval code can read it successfully when called directly -- the failure is purely an artifact of the module's all-or-nothing batch error handling.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2745.12882	Metacheck code: a defect in detecting or retrieving a supported repository	Two distinct issues compound here. First, a narrower, previously-identified limitation (documented in the prior audit and still accurate): the citation uses a bare urn:uuid identifier with no DOI anywhere in the data-availability statement, and metacheck's .dataone_pid() (R/archive-dataone.R) only has regex branches for doi:10.x/... and bare-DOI forms, not for a top-level urn:uuid citation PID -- so dataone_info()'s validity filter (requiring both a non-NA host AND a non-NA pid) would exclude this specific citation from its live per-record API lookup regardless of any other bug. Second, and more decisively given that files_n = 0 for every one of these 8 papers: res_repo_check$gated_repos shows the SAME 'arguments imply differing number of rows' error (with varying counts) attached to every dataone-flagged repo_url across the corpus (4 rows total, from only 2-3 papers using DataONE), exactly mirroring the dataverse and figshare-DOI batch-poisoning defect in inst/modules/repo_check.R's DataONE block (dataone_info() called once per corpus batch inside one tryCatch; one bad record anywhere in the batch crashes the shared call, and the error message is stamped onto every dataone repo_url via repos$repo_error[repos$repo_url %in% dataone_urls] <<- conditionMessage(e)). Because this paper's own citation would already be filtered out of the 'valid' (pid-resolvable) set before ever reaching the API call, the specific batch crash could in principle have originated from one of the OTHER dataone-citing papers' (the two 10.5063/f1... DOIs) records. Either way, files_n = 0 is confirmed, and the correct classification is a genuine code-level platform bug (both the urn:uuid PID-extraction gap and the batch-wide crash handling), not a non-issue.
10.1111/1365-2745.14098	Metacheck code: a defect in detecting or retrieving a supported repository	RECLASSIFIED 2026-10-08 (SUPPORTED_PLATFORM_BUG): Category corrected from CLASSIFIER_EXTENSION_GAP, which does not describe this row: the remaining gap is that metacheck does not expand the tinyurl.com shortlink to the paper's GitHub file. Earlier explanation: RECLASSIFIED 2026-10-07: a previous pass of this row attributed the miss to a missing DSpace7 bare-DOI resolution fallback for the paper's primary citation (10.3929/ethz-b-000597256, ETH Zurich Research Collection). That fallback already exists: dspace7_links() (R/archive-dspace7.R) calls .dspace7_resolve_doi_url(), which live-resolves this exact DOI to https://www.research-collection.ethz.ch/handle/20.500.11850/597256 and correctly matches it against .dspace7_hosts() -- confirmed by calling .dspace7_resolve_doi_url() directly. This fallback was added in commit 57f480d (2026-09-29), before this row was ever written, so the primary citation is not and was never a genuine metacheck gap under the current/installed package. The paper's SECONDARY citation, however, remains a genuine gap: a tinyurl.com shortlink pointing to a GitHub blob URL, which github_links() cannot match because it only regex-matches literal github.com strings and metacheck has no shortlink-expansion logic anywhere in its codebase. Confirmed no such expansion exists in the current source.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13073	Metacheck code: a defect in detecting or retrieving a supported repository	Metacheck has a dedicated Dryad module (R/archive-dryad.R) with dryad_links()/dryad_info(), wired into inst/modules/repo_check.R, and the module's own bare-DOI regex ("(?:https?://)?(?:doi\ \ .org/)?10\ \ .5061/dryad\ \ .[A-Za-z0-9]+") is a perfect, standard match for this citation -- I ran the exact regex from the source file against both the short body-text fragment ("Dryad Digital Repository, https://doi.org/10.5061/dryad.g66247g") and the full reference-list sentence from this paper, and in both cases it correctly extracted "https://doi.org/10.5061/dryad.g66247g" verbatim. The live DOI resolves (302) to a fully public, non-login datadryad.org dataset page with three files listed, so the repository itself is genuinely live and accessible -- this is not a dead link. Since the regex match succeeds in isolation but metacheck_any_detector_hit was reported false for the real pipeline run on this paper, the gap is not that Dryad is unsupported or that this DOI prefix/pattern is unregistered (10.5061/dryad. is exactly the pattern archive-dryad.R is built around); it must instead be that dryad_links()'s two input sources -- paper_table(paper, "url") (real anchor-tag hrefs extracted from the source PDF/XML) and text_search(paper, ...) over metacheck's own parsed-paper text representation -- did not surface this particular sentence to the regex in the actual extraction pipeline (e.g., a GROBID/PDF-parsing segmentation artifact placing the citation in a block text_search does not scan, or the real paper's extracted text differing from the flat full_text string used here). Because the detector logic is demonstrably correct and the platform is supported, but the real run still missed a citation its own regex can parse, I classify this as a SUPPORTED_PLATFORM_BUG (an upstream text/URL-extraction issue feeding the Dryad detector) rather than a missing config or missing platform.
10.1111/2041-210X.13428	Metacheck code: a defect in detecting or retrieving a supported repository	CONFIRMED CODE BUG, still present. The paper's data-availability sentence ('...has been made publicly available on http://Zenodo.org') cites only the generic Zenodo homepage; the ACTUAL resolvable citation, the Zenodo DOI (10.5281/zenodo.3826426), appears only in the reference list entry for Arje et al. (2020). zenodo_links() DOES include a working regex for exactly this DOI shape (confirmed by direct test: .zenodo_id() correctly resolves this exact DOI string to id 3826426 when tested directly), but it calls text_search(paper, zen_bare_regex, return='match', perl=TRUE) WITHOUT passing include_refs=TRUE. text_search()'s default (include_refs=FALSE, R/text_search.R) explicitly filters out any text row whose section_type is 'references' before searching -- confirmed directly: the row containing the DOI has section_type=='references', and calling text_search() on this paper with the same regex returns 0 rows, while testing the identical regex directly against the raw row string (bypassing text_search()'s section filter) matches it immediately. So the one genuinely resolvable Zenodo citation in this paper is systematically invisible to the detector because it sits in the reference list rather than the body text, and only the unresolvable bare homepage mention is ever found instead.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13547	Metacheck code: a defect in detecting or retrieving a supported repository	RECLASSIFIED 2026-10-13 (SUPPORTED_PLATFORM_BUG, was SUPPORTED_PLATFORM_CONFIG_GAP): Zenodo is supported and the DOIs are clean, but they appear only in the reference list, which metacheck's Zenodo detector does not search: a metacheck code defect. Earlier explanation: Zenodo is a fully supported platform in metacheck (R/archive-zenodo.R, zenodo_links()). That function already implements a deliberate two-tier search: real hyperlinks from the paper's url table, plus a body-text fallback regex for bare DOIs like "10.5281/zenodo.NNNNNNN" that appear without a hyperlink. The problem for this specific paper is that the fallback regex is run through text_search(paper, zen_bare_regex, return = "match", perl = TRUE) without passing include_refs = TRUE, and text_search() defaults to include_refs = FALSE, which explicitly filters out any text row whose section_type is "references" (see text_search.R, the `section_filter <- !text$section_type %in% "references"` line, used whenever include_refs is not set to TRUE). In this paper, the inline data-availability sentence is a bare author-year citation with no DOI or URL at all ("The Paris 12 dataset is available on Zenodo (Machon, 2020)."; similarly for the code, "available on Zenodo (Louvet, 2020), as well as on a GitHub repository"), and the only place the actual resolvable Zenodo DOIs (10.5281/zenodo.3770339 and 10.5281/zenodo.4314518) appear anywhere in the extracted text is inside the References section's bibliography entries. Since zenodo_links() never searches the references section for its bare-DOI fallback, it finds no hyperlink and no in-body bare DOI, so it returns zero Zenodo URLs for this paper even though both repositories are live, public Zenodo records with files. This is not a missing backend (Zenodo is fully implemented) and not a garbled DOI (the DOIs are intact and correctly formatted) -- it is a configuration/search-scope gap in how the existing Zenodo detector decides which part of the document to scan for bare identifiers, which this specific paper's citation style (name-year in text, DOI only in the reference list) exposes. [CONFIG_GAP_TIER: needs_code_change -- rechecked 2026-10-13 against metacheck 109a9aa and the live repository: The Zenodo DOIs are only in the reference list, which metacheck's Zenodo detector does not search.]
10.1111/2041-210X.13768	Metacheck code: a defect in detecting or retrieving a supported repository	Paper's formal DAS text is scrambled by PDF-extraction reflow, but the code deposit self-citation (DOI 10.5281/zenodo.5562809) survives intact in the reference list. Metacheck's zenodo_links() has a documented bare-DOI fallback regex specifically built for exactly this citation style, and the DOI is neither garbled nor truncated. Live-verified the deposit is public, the zip supports HTTP range requests, and data_check_helpers.R explicitly classifies .py files as 'code' -- every mechanism needed should work. No specific bug could be pinpointed from static source review alone; this would need an actual pipeline rerun to confirm where/if it diverges.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13785	Metacheck code: a defect in detecting or retrieving a supported repository	Zenodo is a fully supported platform in metacheck, implemented in R/archive-zenodo.R. The availability statement reads "Code and data for all analyses (Xirocostas et al., 2021) and the ZAX Herbivory Trainer (Debono, 2021) are archived on Zenodo," and the reference list spells out both DOIs cleanly and completely: "https://doi.org/10.5281/zenodo.5746183" and "https://doi.org/10.5281/zenodo.5746655" -- neither is garbled, split, or missing whitespace. metacheck's own zenodo_links() bare-text fallback regex is `(?:https?://)?(?:doi\ .org/)?10\ .5281/zenodo\ .[0-9]+`, which matches these two strings exactly as they appear in the extracted text (I checked this against the regex by inspection: "https://doi.org/10.5281/zenodo.5746183" satisfies every token of the pattern with no mismatch). Both DOIs resolve live to real, public, open Zenodo records with listed files. Since the platform is supported, the regex should match the citation form actually used, and the repositories are genuinely live and accessible, yet metacheck_any_detector_hit is reported as false, the most likely explanation is a bug somewhere in the detection pipeline between text/URL extraction and the regex match or the live Zenodo API lookup (e.g., the GROBID-extracted paper object's `url` table or body text segmentation for this specific PDF, or a failure in the zenodo_info() API call for these two specific records) rather than a missing platform or a textual extraction gap, since the text I was handed contains the DOIs in good, unambiguous condition.
10.1111/2041-210X.13865	Metacheck code: a defect in detecting or retrieving a supported repository	No DAS exists in the extracted text; the only repository mention is a self-citation reference-list entry for Zenodo DOI 10.5281/zenodo.6400498, which appears verbatim and unmangled. Metacheck's zenodo_links() bare-text fallback regex is explicitly designed to catch exactly this citation style. Live-verified the Zenodo API record returns a complete file listing normally. Given the DOI is clean, the regex matches it, the API record is live, and the backend code exists and works for hundreds of other Zenodo papers in this corpus, this is best explained as an execution-time pipeline bug for this specific paper rather than a missing capability or config/URL-pattern gap.
10.1111/2041-210X.14387	Metacheck code: a defect in detecting or retrieving a supported repository	dataone_links() correctly detects the bare doi.org/10.5063/f1mc8x92 citation verbatim and correctly routes it to the DataONE/KNB detector (host knb.ecoinformatics.org, doi_prefix 10.5063, both explicitly registered in .dataone_hosts()). However, direct inspection and live invocation of .dataone_pid() shows its regex captures the DOI suffix exactly as it appears in the citation text (case-preserved; ignore.case = TRUE only affects matching 'doi:'/'doi.org/' prefixes, not the captured identifier itself) -- producing pid 'doi:10.5063/f1mc8x92' in lowercase. KNB's Metacat v2 API is case-sensitive on this pid: querying the meta endpoint with the lowercase pid returns HTTP 404 live right now, while querying the identical endpoint with the pid case-corrected to doi:10.5063/F1MC8X92 returns a full, valid EML metadata record with 1 file (confirmed by direct invocation of the installed package's .dataone_info() today). This is a genuine, deterministic code defect: .dataone_pid() should normalize/preserve the DOI's registered canonical case (or try both cases) rather than passing through whatever case happens to appear in the citing paper's text.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.14431	Metacheck code: a cache entry predates a fix or kept a failed lookup	STALE CACHE, not a current code bug. A fresh, uncached, live rerun of module_run(paper,"repo_check") just now correctly finds repo_n=1/files_n=96 for OSF node fwjh9 -- the detector itself works. The stored res_repo_check.RData shows repo_n=0 because the pipeline is called with cache=TRUE (01_run_metacheck.R), and code/data/.metacheck_repo_info_cache/osf_fwjh9.rds (dated 2026-09-09, nearly a month stale) holds a cached data.frame(osf_id='fwjh9', osf_type='error') -- i.e. a FAILED lookup from whenever that cache entry was written (OSF's API is documented in the package itself as frequently down). This should never have been cached as a success: R/archive-osf-helpers.R calls .repo_info_cache_put() only when .repo_info_ok(result) is TRUE, and .repo_info_ok() (repo-info-cache.R) checks for a column literally named 'error' -- but OSF's own error frame uses the column name 'osf_type' with value 'error', not a column named 'error'. Because 'error' is not among that frame's column names, .repo_info_ok() falls through to its default TRUE, so a transient OSF failure was permanently cached as if it had succeeded, and cache=TRUE has silently replayed that false negative on every subsequent pipeline run since. This IS a real code bug (the .repo_info_ok() check should also recognise OSF's osf_type=='error' shape), separate from the immediate cause of this paper's repo_n=0 (the stale cache entry itself, fixable today by clearing that one cache file, independent of whether the .repo_info_ok() bug is also patched).
10.1111/1365-2745.13708	Metacheck code: a cache entry predates a fix or kept a failed lookup	STALE CACHE, not a current code bug. A fresh, uncached, live rerun of module_run(paper,"repo_check") just now correctly finds repo_n=1/files_n=2 (data CSV plus R script) for KNB record doi:10.5063/PG1Q4B -- the detector itself works correctly today. The stored res_repo_check.RData shows repo_n=0 because the pipeline runs with cache=TRUE, and code/data/.metacheck_repo_info_cache/dataone_knb.ecoinformatics.org_doi_10.5063_PG1Q4B.rds (dated 2026-09-13) holds a genuinely empty files=list() result with NO error column (so .repo_info_ok() correctly, but prematurely, treated it as a valid 'ok' result worth caching permanently). Git history shows R/archive-dataone.R was patched for exactly this failure mode in commit ba661c7 ('Fix DSpace7/DataONE metadata-and-file gaps...', confirmed live 2026-09-29, issue #435 follow-up): some KNB EML records describe a file as an otherEntity with no physical child, which the OLD parser (in effect when the Sept 13 cache entry was written) returned zero files for; the fix added a fallback branch reading the entity's own id attribute. The Sept 13 cache entry predates that fix by over two weeks and has been silently replayed as a false negative on every pipeline run since, masking that the underlying bug was already fixed in the source.
10.1111/1365-2435.14350	Metacheck code: archive listed but never downloaded or opened	The repo's Wieczynski_etal_2023_FE_Master.tar.gz genuinely contains a Mathematica model-code notebook plus 30 .csv data files (confirmed by downloading and unpacking it directly). .tar.gz IS recognized as a readable archive by .is_tar_archive(), but under the production download='data' mode, the baseline want formula never sets want=TRUE for an 'unknown'-typed row, and the rescue line 'want <- want & (!(data_type %in% never_fetch) | keep_archive)' is an AND that can only narrow an already-true want, never promote a false one to true -- confirmed by re-executing the formula in R. So the archive is never downloaded or expanded. CORRECTION 2026-10-07: a live rerun confirmed the archive-promotion AND/OR fix (commit e4b74e1) does make metacheck attempt to open this .tar.gz, but the actual extraction still fails -- the rerun logged 'tar.exe: Archive entry has empty or unreadable filename ... skipping' and the resulting file list contains only a README.md and one other entry, data_type/files_data/files_code = 0/0/0, not the 30 .csv data files and Mathematica notebook this row originally described. The want-formula bug is fixed, but a second, separate defect in tar extraction itself (likely a long-path, special-character, or non-ustar-header issue in this specific .tar.gz) still prevents the real contents from being enumerated.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.12884	Metacheck code: archive listed but never downloaded or opened	RECLASSIFIED 2026-10-08 (CLASSIFIER_ARCHIVE_NOT_EXPANDED): In the current res_repo_check.RData, 'Supporting info s7.tar' (931,840 bytes) is listed with data_type 'unknown', i.e. it was never opened. The cause was not established in this review. Earlier explanation: 'Supporting info s7.tar' (931.84 KB) genuinely contains MATLAB scripts, R code, and an Excel occupancy data file (confirmed via live Dryad fetch), matching the paper's own description ('Data S7. Occupancy data files and scripts'). The tar's expansion step did not execute for this file in this run despite being well under all size caps (100MB/500MB) and despite .expand_tar() being designed to handle exactly this format.
10.1111/1365-2664.14288	Metacheck code: archive listed but never downloaded or opened	RECLASSIFIED 2026-10-08 (CLASSIFIER_ARCHIVE_NOT_EXPANDED): Sole file Morford_etal-2022_JAE.zip (18,596,974,642 bytes) contains data products and modelling code per the Zenodo depositor's description. In the current res_repo_check.RData it is listed with data_type 'unknown'. The archive is far larger than Metacheck's archive download cap (2 x max_download_size, at most about 1 GB with the 500 MB default; R/repo-download.R line 1551), so it is never downloaded or opened. An earlier version of this row blamed the peek_zips default; peek_zips now defaults to TRUE, so that explanation is out of date.
10.1111/1365-2745.13537	Metacheck code: archive listed but never downloaded or opened	RECLASSIFIED 2026-10-08 (CLASSIFIER_ARCHIVE_NOT_EXPANDED): Sole file BadenEtAlPlantago2020.zip (15,792,162,474 bytes) contains the paper's core research data (confirmed via Dryad's file description). In the current res_repo_check.RData it is listed with data_type 'unknown'. The archive is far larger than Metacheck's archive download cap (2 x max_download_size, at most about 1 GB with the 500 MB default; R/repo-download.R line 1551), so it is never downloaded or opened. An earlier version of this row blamed the peek_zips default; peek_zips now defaults to TRUE, so that explanation is out of date.
10.1111/2041-210X.12820	Metacheck code: archive listed but never downloaded or opened	ADDED 2026-10-12 after the archive-download rerun (script 13, then script 14 with every rate limit waited out). repo_check finds the Dryad dataset (doi:10.5061/dryad.j2v6q) and lists its README plus two members of the dataset archive, 'A_Simulated Data.zip' and 'B_Empirical data.zip' -- archives inside the dataset archive. Metacheck reads only one level of archive (intended behaviour, confirmed by the Metacheck developer), so these inner archives are neither opened nor counted as data, and data_check classifies no file as data. The earlier corpus run (older metacheck) had counted the two inner .zip files themselves as data files, which is why this paper was not in the population before.
10.1111/2041-210X.13459	Metacheck code: archive listed but never downloaded or opened	RECLASSIFIED 2026-10-08 (CLASSIFIER_ARCHIVE_NOT_EXPANDED): Three small tar.gz archives (DogExperiment.tar.gz 20,629 bytes; FieldExperiment.tar.gz 1,048,909 bytes; OTUs.tar.gz 4,447 bytes) are genuine DNA-sequencing data per the paper's availability statement. In the current res_repo_check.RData all three are listed with data_type 'unknown', i.e. never opened. An earlier version of this row blamed Dryad's requirement for an OAuth token; the installed metacheck now supports Dryad OAuth client credentials (dryad_auth()), so that explanation is out of date. The cause of the non-expansion in the recorded run was not established.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.13795	Metacheck code: an R package's own source files excluded from the file listing	RECLASSIFIED 2026-10-07 (still a genuine, currently-reproducible gap -- only the explanation below changed, not the classification). A live rerun today confirms repo_check() still reports only 9 files for this repo, but the ROOT CAUSE is not a shallow/non-recursive crawl: github_tree_files() (R/archive-github.R) correctly retrieves all 238 files in the repo, including 96 real .R source files under R/ -- confirmed directly by calling it live. The 229 files get dropped afterwards, inside .is_r_package_file() (R/data_check_helpers.R, lines 671-732): this function provisionally treats the whole DESCRIPTION+NAMESPACE package tree as an excludable vendored dependency, then only keeps that exclusion if nothing OUTSIDE the package classifies as real data/code. For biomonitoR the 9 non-package files include .travis.yml and codecov.yml, both CI-service config files; a separate, unrelated blanket rule in data_classify_files() (line 172-173) classifies any .yml/.yaml file as 'code' by extension alone, with no exception for CI config. That false 'has_real_content = TRUE' signal stops the function's own reset logic from firing, so all 229 genuine package files (the actual deposited analysis package) stay wrongly excluded, same as if biomonitoR were a vendored dependency rather than the paper's real code. The function already special-cases .Rproj out of this same check for an identical reason (its own comment: 'packaging config, not a real analysis script') but was never extended to .travis.yml/codecov.yml/similar CI-only files. Confirmed empirically by running the function body by hand against the live 238-file listing.
10.1111/2041-210X.12784	Metacheck code: an R package's own source files excluded from the file listing	RECLASSIFIED 2026-10-07 (still a genuine, currently-reproducible gap -- only the explanation below changed, not the classification). A live rerun today confirms repo_check() still under-reports this repo's file count. The root cause, confirmed in detail for a sibling case in this same audit category (10.1111/1365-2664.13795, biomonitoR), is NOT a shallow/non-recursive GitHub crawl -- github_tree_files() correctly retrieves the full file tree. The files are instead dropped afterwards by .is_r_package_file() (R/data_check_helpers.R): a root-level R package's real content gets wrongly treated as an excludable vendored dependency whenever some file outside the package (e.g. a CI-service config file) is misclassified as 'code' by the unrelated blanket '.yml/.yaml is code' rule, which stops the function's own exclusion-reset logic from firing. This paper's repository is also a root-level R package of the same shape, so the same mechanism plausibly applies, but was not independently re-confirmed file-by-file for this specific repo today -- treat the mechanism as a strong hypothesis carried over from the biomonitoR case, not an independently re-verified fact for this paper.
10.1111/2041-210X.13009	Metacheck code: an R package's own source files excluded from the file listing	RECLASSIFIED 2026-10-07 (still a genuine, currently-reproducible gap -- only the explanation below changed, not the classification). A live rerun today confirms repo_check() still under-reports this repo's file count. The root cause, confirmed in detail for a sibling case in this same audit category (10.1111/1365-2664.13795, biomonitoR), is NOT a shallow/non-recursive GitHub crawl -- github_tree_files() correctly retrieves the full file tree. The files are instead dropped afterwards by .is_r_package_file() (R/data_check_helpers.R): a root-level R package's real content gets wrongly treated as an excludable vendored dependency whenever some file outside the package (e.g. a CI-service config file) is misclassified as 'code' by the unrelated blanket '.yml/.yaml is code' rule, which stops the function's own exclusion-reset logic from firing. This paper's repository is also a root-level R package of the same shape, so the same mechanism plausibly applies, but was not independently re-confirmed file-by-file for this specific repo today -- treat the mechanism as a strong hypothesis carried over from the biomonitoR case, not an independently re-verified fact for this paper.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/2041-210X.13503	Metacheck code: an R package's own source files excluded from the file listing	RECLASSIFIED 2026-10-07 (still a genuine, currently-reproducible gap -- only the explanation below changed, not the classification). A live rerun today confirms repo_check() still under-reports this repo's file count. The root cause, confirmed in detail for a sibling case in this same audit category (10.1111/1365-2664.13795, biomonitoR), is NOT a shallow/non-recursive GitHub crawl -- github_tree_files() correctly retrieves the full file tree. The files are instead dropped afterwards by .is_r_package_file() (R/data_check_helpers.R): a root-level R package's real content gets wrongly treated as an excludable vendored dependency whenever some file outside the package (e.g. a CI-service config file) is misclassified as 'code' by the unrelated blanket '.yml/.yaml is code' rule, which stops the function's own exclusion-reset logic from firing. This paper's repository is also a root-level R package of the same shape, so the same mechanism plausibly applies, but was not independently re-confirmed file-by-file for this specific repo today -- treat the mechanism as a strong hypothesis carried over from the biomonitoR case, not an independently re-verified fact for this paper.
10.1111/2041-210X.13750	Metacheck code: an R package's own source files excluded from the file listing	RECLASSIFIED 2026-10-07 (still a genuine, currently-reproducible gap -- only the explanation below changed, not the classification). A live rerun today confirms repo_check() still under-reports this repo's file count. The root cause, confirmed in detail for a sibling case in this same audit category (10.1111/1365-2664.13795, biomonitoR), is NOT a shallow/non-recursive GitHub crawl -- github_tree_files() correctly retrieves the full file tree. The files are instead dropped afterwards by .is_r_package_file() (R/data_check_helpers.R): a root-level R package's real content gets wrongly treated as an excludable vendored dependency whenever some file outside the package (e.g. a CI-service config file) is misclassified as 'code' by the unrelated blanket '.yml/.yaml is code' rule, which stops the function's own exclusion-reset logic from firing. This paper's repository is also a root-level R package of the same shape, so the same mechanism plausibly applies, but was not independently re-confirmed file-by-file for this specific repo today -- treat the mechanism as a strong hypothesis carried over from the biomonitoR case, not an independently re-verified fact for this paper.

Table 5 lists, in the same way, every one of the 28 papers in the main text's cluster of repositories that contain data or code Metacheck did not classify: 16 where the data or code were not shared in a FAIR format, and 12 where they are in a format Metacheck does not yet support.

Table 8: Table 5. Full paper-by-paper detail for every paper whose repository contains data or code that Metacheck did not classify.

Paper (DOI)	Category	Why Metacheck misses it
10.1002/2688-8319.12149	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data file is a rich-text (.rtf) document rather than a machine-readable data file. Earlier explanation: The deposit's two files (both .rtf) are the genuine data file (moose density estimates) and its README; both land on 'documentation' because .rtf has no entry in .ext_registry. file_category() returns NA for the data file so Tier 2 doesn't fire either; Tier 3's coarse crosswalk maps .rtf's coarse type 'text' to 'documentation', which correctly identifies the README but incorrectly also catches the actual data file saved in the same format.

Paper (DOI)	Category	Why Metacheck misses it
10.1002/pan3.10334	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data (letter transcripts) are .docx files named by anonymised codes, not labelled as data. Earlier explanation: All 39 files in the OSF repo are .docx anonymized individual letter transcripts, confirmed by the OSF project's own title and description to be the actual dataset. .docx has no row in .ext_registry, file_category() returns NA for these filenames, and none contain a Tier-2 keyword token (anonymized site codes, e.g. '37yam_3_1.docx'). Tier 3's coarse crosswalk resolves .docx to coarse type 'text', mapped to 'documentation'. Directly reproduced empirically: data_classify_files() on three real filenames returned 'documentation' for all three.
10.1002/pan3.6	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data (interview transcripts) are shared as PDF files whose names (CP01.pdf ...) do not label them as data. Earlier explanation: All 33 genuine interview-transcript files (CP01.pdf...CP35.pdf, redacted/anonymized, exactly matching the paper's stated raw data) are classified 'documentation'. .pdf has no row in .ext_registry (deliberately left unlocked, since a PDF can be materials/documentation/preprint depending on content). Filenames like 'CP01.pdf' contain no Tier-2 keyword token, so Tier 3's coarse crosswalk fires: metacheck::file_types maps .pdf to coarse type 'text', and the crosswalk maps text->'documentation'. This is a real miss on genuinely primary raw data (anonymized qualitative transcripts), not materials/questionnaires/preprints, with no keyword signal to rescue it.
10.1111/1365-2435.13500	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data are serialized R objects saved with the .Rd extension, which is R's documentation-file extension, so the file name mislabels the content. Earlier explanation: NOTE: this paper does not exactly fit the stated premise of 'classified none of the files as data or code' -- 5 of 6 files WERE classified 'code', just wrongly. The 5 real data files (SitesMassBins.Rd etc., each 0.9-1.7MB, MIME application/gzip per the Figshare API, i.e. genuine serialized data objects, not R documentation) were saved with extension '.Rd', colliding with R's own .Rd package-documentation-markup extension. .ext_registry has no Tier-1 entry for 'rd'; Tier 3's coarse crosswalk hardcodes ext=='rd' -> type 'code' (confirmed directly), with no check against file size or binary content that would flag these multi-megabyte gzip blobs as implausible R documentation source.
10.1111/1365-2435.13735	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The simulation outputs are bare .txt files (inside a zip) whose names do not label them as data; a .txt file needs to be labelled as data to be findable. File content not checked directly in this pass. Earlier explanation: 18 genuine simulation-output .txt files (e.g. Coord_row_sim_s0000.txt, Trait_sim_s0000.txt) inside the deposited ZIP are simulation-generated DATA, confirmed by the paper's text and Dryad's own listing, but none were ever assigned data_type=='data'. .txt is deliberately not format-locked in .ext_registry since it can be prose or data; Tier 2 keyword rules require tokens like 'data'/'output'/'results', none of which appear in these filenames; Tier 3's crosswalk routes generic 'text' filetype to 'documentation'. NOTABLY, the RDS snapshot actually used in the manuscript run shows these files as 'code' (not 'documentation' as today's vendored code produces when rerun), indicating the classifier code changed between when the snapshot was generated and now -- a reproducibility concern flagged separately from the root classification gap. Either way, genuine simulation-result data is mistyped into a non-data bucket.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2435.14060	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The files are space-separated .txt tables of analysis outputs, named only by variable (e.g. aggression_rate.txt), and not labelled as data. Earlier explanation: 8 of 9 files in the Dryad repo (aggression_rate.txt, group_size.txt, N_groups_day.txt, N_individuals_day.txt, steepness.txt, strength_associations.txt, time_groups.txt, time_individuals.txt) are genuine tabular research data but all classified data_type='documentation'. .ext_registry leaves .txt's data_type as NA (deliberately not format-locked). Tier 2 keyword rules only override for bounded name tokens 'data'/'code'/etc; none of these 8 filenames contain such a token. All 8 fall to Tier 3's coarse crosswalk: filetype() returns 'text', and .file_type_crosswalk['text'] resolves to 'documentation'.
10.1111/1365-2435.14337	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): Chen_2023_gross.txt is a tab-separated table preceded by a quoted title line (checked directly), shared as a bare .txt file not labelled as data. Metacheck's after-download content check also misses it (metacheck issue #470). Earlier explanation: Chen_2023_gross.txt is almost certainly the paper's core numeric dataset (gross soil N mineralization/nitrification rates). .txt is deliberately unlocked at Tier 1. The content-based txt_classify_content() upgrade path only recognizes comma/semicolon/tab/pipe delimiters and explicitly, by documented design, does not detect space/whitespace-delimited tables -- plausibly the file's actual format, so it never upgrades past the default 'documentation' guess. README_Chen_2023_gross.txt correctly resolves via the 'readme' keyword.
10.1111/1365-2656.12671	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The data are shared as tables inside Word .doc files rather than as machine-readable data files. Earlier explanation: .doc files on Dryad containing genuine research data were classified under a non-data category by the generic document-extension handling.
10.1111/1365-2656.12732	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): Dataset_Outomuro&Johansson.txt is a tab-separated table with a header row (checked directly), shared as a bare .txt file. Metacheck's name rule does not treat 'Dataset' as a data label, and its after-download content check misses the table because of trailing empty cells (metacheck issue #470). Earlier explanation: A genuine data file named with the word 'dataset' (not the bounded token 'data') was missed by Tier 2's keyword regex, which requires 'data' bounded by separators and does not match 'dataset' or similar compound words.
10.1111/1365-2656.12978	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The interaction matrices are bare .txt files named P#_###_###.txt that do not label them as data. File content not checked directly in this pass. Earlier explanation: ~65 files named 'P#_###_###.txt' are genuine ant-plant interaction-count matrices (confirmed via Dryad's own file description). .txt is unlocked at Tier 1; Tier 2 keyword rules do not match these filenames. txt_classify_content()'s delimiter sniffer only tests comma/semicolon/tab/pipe and explicitly does not detect whitespace-delimited tables -- exactly the format of these species-interaction matrices -- so all fall to the Tier 3 'text'->documentation crosswalk default.
10.1111/1365-2656.13145	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): Male_SES_encounter_history_matrix.txt is a space-separated .txt matrix (checked directly) whose name does not label it as data. Earlier explanation: Single file Male_SES_encounter_history_matrix.txt is the paper's primary mark-recapture dataset (34-yr encounter history matrix, 6,245 individuals, 58,177 records), explicitly titled 'Data from: Encounter history matrix' in the paper's own reference citation. .ext_registry leaves .txt's data_type as NA. Tier 2 keyword rules require a bounded 'data' token; the filename contains none. Tier 3 crosswalks the coarse 'text' filetype to 'documentation'.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2656.13222	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): CTs.txt is a tab-separated table with a header row (checked directly), but a bare .txt file whose name does not label it as data. Separately, Metacheck's after-download content check, which should recognise tab tables, fails on this file because of a non-UTF-8 byte (metacheck issue #470). Earlier explanation: CTs.txt is the primary data file (critical thermal limits for juvenile Rana temporaria, confirmed via Dryad's own dataset description). .txt is left unlocked at Tier 1. Tier 2 keyword rules find no match in the filename 'CTs' (short for 'critical thermals'). Tier 3 crosswalks the coarse 'text' filetype to 'documentation'. Verified directly: data_classify_files('CTs.txt') returns 'documentation'.
10.1111/1365-2745.13026	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The cited Zenodo deposit (https://zenodo.org/records/1291377) contains LICENSE.md, README.md and a supplement PDF. The PDF contains a table with data. The data in this table should have been shared in a FAIR format (e.g., a csv file) for Metacheck to classify the repository as containing data. So there is data, but it is not in a FAIR format; Metacheck's classification of the PDF as documentation is as designed.
10.1111/2041-210X.13332	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The C++ source is committed without any file extension ('C++ code'), so its language cannot be determined from the file type. Earlier explanation: The repo's sole source file is committed with no extension at all, literally named 'C++ code' (confirmed via live GitHub fetch to be genuine Arduino/C++ source). data_check's own classifier already gets this right (Tier 2 keyword match on the bare word 'code', data_type='code'). The actual miss is downstream in code_check.R's code_lang(), which determines language purely via tools::file_ext() matched against .ext_registry$code_lang; an empty extension matches nothing, so code_lang() returns NA and the file never enters code_check's candidate set, leaving code_check_repo_urls empty despite data_check already having typed the file correctly.
10.1111/2041-210X.13396	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The repository's only file is a Word supporting-information document, Nishida_supplementary.docx (https://datadryad.org/downloads/file_stream/292473), opened and read directly. Its Tables S1-S7 report means and standard deviations, and Tables S2-S4 and S6 also give specimen-level values. These are data, but shared as tables in a Word document rather than as a machine-readable file (e.g., a csv file), so Metacheck classifies the file as documentation.
10.1111/2041-210X.13979	Data or code not shared in a FAIR format	RECLASSIFIED 2026-10-08 (DATA_OR_CODE_NOT_IN_FAIR_FORMAT): The R code is saved as 'R code.txt' rather than with a .R extension, so its language cannot be determined from the file type. Earlier explanation: Sole file 'R code.txt' genuinely is R source code (confirmed via Zenodo's own title/description: implements five richness estimators). data_check's data_classify_files() already correctly tags it 'code' via a Tier-2 keyword match on 'code' bounded by space/period. But code_check()'s separate code_lang() function keys purely on file extension via .ext_registry$code_lang; .txt's code_lang is NA by design, so code_lang('R code.txt') returns NA, the file is filtered out of code_check's relevant-languages set, and code_check_repo_urls comes back empty -- a cross-module inconsistency between data_check's keyword logic and code_check's extension-only logic.

Paper (DOI)	Category	Why Metacheck misses it
10.1002/pan3.10200	Format Metacheck does not yet support	All 18 files are ESRI/GDAL GeoTIFF raster sidecar files (.tfw, .tif, .tif.aux.xml, .tif.ovr, .tif.vat.cpg, .tif.vat.dbf) for the paper's 3 conservation-prioritization output layers. None of these extensions appear anywhere in .ext_registry (data_check_helpers.R). tools::file_ext() truncates to the last dot only, so compound extensions like .tif.aux.xml resolve to 'xml'. Reran data_classify_files() directly and reproduced the exact observed output: .tif crosswalks to 'materials' (generic image bucket), .tif.aux.xml crosswalks via 'xml' to 'code' (false positive), the rest fall to 'unknown'. No GIS raster family extension exists anywhere in the registry.
10.1111/1365-2435.13183	Format Metacheck does not yet support	data_check correctly typed the repo's one Julia source file (wns-simple.jl) as 'code' via metacheck::file_types, where jl->code is registered. However, code_check (whose output is what the manuscript's n_mc_code aggregate actually counts) gates everything through code_lang(), which looks up language ONLY via .ext_registry$code_lang -- and that table has no row for extension 'jl' at all (Julia appears only as a possible notebook kernel, never as its own .jl entry). So code_lang() returns NA, the file is never flagged as a code-check candidate, and code_check_repo_urls is empty despite data_check correctly flagging the same file. This is an internal inconsistency between two classifier tables that should agree but don't. CORRECTION 2026-10-08: a live rerun confirmed .ext_registry$code_lang now HAS a row for extension 'jl' (added 2026-10-04), and data_classify_files() correctly tags wns-simple.jl as data_type = 'code'. But this row's original diagnosis was already about a DIFFERENT, downstream gate: code_check.R's own `listed_langs` whitelist (R, Python, SAS, SPSS, Stata, Mplus, MATLAB, JASP) never included 'Julia' as a possible language at all, so code_check's own `table` (what the manuscript's Cooper-vs-Metacheck comparison counts, via code_table_slim_obj) still has zero rows for this paper even though data_check's classification is now correct. Confirmed directly: a fresh rerun through module_run() produced res_code_check$table with 0 rows for this paper_id. The extension-registry fix and the code_check language-whitelist gap are two separate, independent defects; only the first has been fixed.
10.1111/1365-2435.13315	Format Metacheck does not yet support	The deposit's only file, DataDryad.7z, is a 7-Zip archive. Metacheck's zip-peek.R explicitly documents that only .zip, the .tar family, and single-file .gz/.bz2/.xz can be opened with base R -- .7z is deliberately left unextracted ('left as other, never fetched') since Metacheck has no libarchive/7z dependency. Since the archive is never opened, data_classify_files() only ever sees the archive's own filename. A second, independent factor compounds this: even the Tier-2 keyword fallback that would normally rescue a misnamed data file fails, since the bounded-token regex requires a separator before 'data' and 'DataDryad.7z' has 'data' glued directly to 'dryad' with no boundary character (confirmed by running the exact regex).
10.1111/1365-2435.13848	Format Metacheck does not yet support	ANSYS Fluent simulation files (.cas/.dat.gz) are genuine simulation data/output files with no entry in .ext_registry, a domain-specific engineering/CFD format gap.
10.1111/1365-2435.13921	Format Metacheck does not yet support	Bruker NMR extensionless raw files plus associated .txt result tables are genuine spectroscopy data/output files not recognized by the classifier's extension rules.

Paper (DOI)	Category	Why Metacheck misses it
10.1111/1365-2664.13096	Format Metacheck does not yet support	Sole file 'EEEworm Model V1.0.0 Johnston et al..nlogo' is NetLogo model source code (confirmed as 'programme code' by the paper's own text). .ext_registry has no 'nlogo' entry anywhere. Tier 2 keyword rules find no match in the filename. Tier 3 coarse crosswalk also has no NetLogo mapping. Directly executed: data_classify_files() on this filename returns 'unknown'. CORRECTION 2026-10-08: a live rerun confirmed .ext_registry now HAS an 'nlogo' entry (added 2026-10-04), and data_classify_files() correctly tags the .nlogo model file as data_type = 'code' (no longer 'unknown' as this row originally found). But code_check.R's own `listed_langs` whitelist (R, Python, SAS, SPSS, Stata, Mplus, MATLAB, JASP) has never included 'NetLogo', so code_check's own `table` still has zero rows for this paper -- confirmed directly via a fresh rerun. Same underlying class of gap as the Julia case above: the extension-registry fix and code_check's separate language-whitelist gate are independent defects, and only the registry side has been addressed.
10.1111/1365-2664.13538	Format Metacheck does not yet support	Single .rar archive genuinely contains R scripts and tabular data files (confirmed via Dryad's own file-content description). .rar is absent from .ext_registry entirely. zip-peek.R's own comment states .rar/.7z 'need an external binary or the libarchive-backed archive package, which metacheck does NOT depend on' -- a deliberately-documented limitation, not an oversight. .is_readable_archive() returns FALSE for 'rar', so it is never downloaded or opened.
10.1111/1365-2664.14746	Format Metacheck does not yet support	Three GeoTIFF spatial-raster data layers (microcorrs500m.tif, PAconns500m.tif, betweenness500m.tif) are exactly what the paper's Data Availability Statement names as shared data. .ext_registry has no entry for 'tif'/'tiff'. Tier 2 finds no keyword match. Tier 3 resolves via metacheck::file_types to coarse type 'image', and .file_type_crosswalk hard-maps image->'materials' -- a rule for ordinary photographs, not disambiguated for georeferenced raster data.
10.1111/1365-2745.13006	Format Metacheck does not yet support	Sole file 'To submission.rar' genuinely contains research data and R code (Figshare's own description: 'contains research materials including a dataset and analytical code'). .ext_registry has no entry for 'rar'. Archive-expansion in data_check.R only unpacks zip/tar-family/gz/bz2/xz; code comments explicitly state .7z/.rar are not carved out and stay excluded, with repo_check instead warning the author to re-upload as .zip.
10.1111/2041-210X.13124	Format Metacheck does not yet support	Sole .rar file genuinely contains two GIS raster data files (congruence.grd, pr_Litocranius_walleri.grd), confirmed via Dryad's usage notes. .ext_registry has no entry for 'rar' (or '7z'). data_check.R's archive-expansion comment explicitly states '.7z/.rar are NOT here -- base R cannot read them (repo_check warns instead)' -- a deliberate design limitation.
10.1111/1365-2664.14504	Format Metacheck does not yet support (read as image material)	Four GeoTIFF raster layers (DailyAbsorpWinter_AllConflict.tif, DailyAbsorpWinter_Livestock.tif, FidelityWinter.tif, ResistanceWinter.tif) are genuine SAMC model input layers for a wolf-recolonization connectivity analysis. .ext_registry has no row for '.tif'/'.tiff'. Tier 2 finds no keyword match. Tier 3's coarse crosswalk buckets .tif under coarse type 'image', and .file_type_crosswalk hardcodes image->'materials' unconditionally -- a rule built for photographs/figures, not GIS raster data (unlike .nii/.dcm/.mnc/.mgz, which the registry already special-cases to 'data').
10.1111/2041-210X.14457	Format Metacheck does not yet support (read as image material)	64 .JPG drone photographs are the paper's primary measurement data for 3D structure-from-motion photogrammetry (explicitly: 'Data required to recreate analysis with ground truthing images'), not participant-facing stimuli. .ext_registry has no entry for 'jpg'/'jpeg'; classification falls to the coarse crosswalk, which hard-codes image->'materials' -- a rule correct for the common case (a stimulus photo) but wrong for this domain where photographs are literally the quantitative primary data.

References

Cooper, N., & the BES Data and Code Hackathon Group. (2026a). Data and code for MEE paper v1.0. Zenodo. https://doi.org/10.5281/zenodo.19737911

Cooper, N., & the BES Data and Code Hackathon Group. (2026b). Data- and code-archiving in the British Ecological Society journals: Present status and recommendations for future improvements. Methods in Ecology and Evolution, 17, 1954–1966. https://doi.org/10.1111/2041-210x.70338

DeBruine, L., Mesquida, C., Werner, J., & Lakens, D. (2026). Metacheck: Check research outputs for best practices. https://github.com/scienceverse/metacheck

Riedel, N., Kip, M., & Bobrov, E. (2020). ODDPub – a Text-Mining Algorithm to Detect Data Sharing in Biomedical Publications. Data Science Journal, 19(1). https://doi.org/10.5334/dsj-2020-042

Vilhuber, L., & Cavanagh, J. (2025). Report of the AEA Data Editor. AEA Papers and Proceedings, 115, 944–957. https://doi.org/10.1257/pandp.115.944

Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., … Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 3(1), 160018. https://doi.org/10.1038/sdata.2016.18